

Grado en Ingeniería Informática
2025-2026

Trabajo Fin de Grado

“Predicción mediante aprendizaje
automático de la calidad de la reducción
de bases de retículo y selección
adaptativa de parámetros”

Fernando Martín Arencibia

Tutor

Francisco Javier Blanco Romero

Leganés, junio de 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

El avance de la computación cuántica compromete los criptosistemas de clave pública en uso (señaladamente RSA y Diffie-Hellman) y ha motivado la adopción de criptografía post-cuántica basada en retículos, estandarizada por el NIST en 2024 (ML-KEM, ML-DSA). La seguridad de estos esquemas reposa sobre el coste real de reducir bases de retículo; sin embargo, los métodos disponibles para estimarlo (cotas del peor caso, heurística Gaussiana o ejecución directa de LLL y BKZ) resultan, respectivamente, demasiado conservadores, insensibles a la instancia concreta o computacionalmente costosos.

Este Trabajo Fin de Grado diseña, implementa y evalúa un sistema de aprendizaje automático supervisado que predice la calidad de la reducción LLL de un retículo a partir únicamente de descriptores algebraicos y geométricos de su base sin reducir, sin necesidad de ejecutar el algoritmo. Sobre un conjunto de datos reproducible de 224 000 instancias, cuatro familias estructurales de retículos (uniforme, gaussiana, *sparse* y *q*-aria), nueve dimensiones y cuatro valores del parámetro δ , se entrenan modelos de *Random Forest*, *gradient boosting* (XGBoost) y *redes de perceptrón multicapa*, bajo estrategias de entrenamiento global y por dimensión, adoptando el *root Hermite factor* (RHF) como métrica rectora.

Los modelos predicen el defecto de ortogonalidad y la norma del primer vector reducido con $R^2 \approx 0,9997$ y $R^2 \approx 0,98$, respectivamente; la especialización por dimensión eleva la predicción del RHF hasta $R^2 > 0,99$ para dimensiones $d \geq 100$. Sobre estas predicciones se construye un oráculo de selección adaptativa de δ que, en su régimen estricto ($\tau = 0$), elige un valor $\delta < 0,99$ en el 36,6 % de las bases con una degradación media de calidad de tan solo +0,123 % respecto del valor por defecto. La validación temporal sobre bases no vistas muestra que estas selecciones producen aceleraciones moderadas del tiempo de ejecución real (hasta $\approx 1,3\times$) sin coste apreciable de calidad; sobre los retículos *q*-arios, la familia que subyace al problema *Learning With Errors* en que se fundamentan los esquemas post-cuánticos estandarizados, el oráculo conserva acertadamente $\delta = 0,99$, evitando la degradación que una reducción más agresiva introduciría en esas instancias. El análisis de atribución mediante valores de Shapley revela que el sistema no reconstruye la ley volumétrica de la heurística Gaussiana, cuya forma fuerte queda refutada, sino que aprende un refinamiento dependiente de la instancia de la teoría asintótica de reducción, sustentado en propiedades geométricas interpretables de la base. Todo ello sitúa la propuesta como herramienta auxiliar para la estimación de seguridad de la criptografía post-cuántica.

Palabras clave: criptografía post-cuántica, retículos, algoritmo LLL, reducción de bases, machine learning, Random Forest, Gradient Boosting, Shortest Vector Problem, Learning With Errors, estimación de seguridad.

ABSTRACT

The advance of quantum computing compromises the public-key cryptosystems in current use, notably RSA and Diffie-Hellman, and has driven the adoption of lattice-based post-quantum cryptography, standardised by NIST in 2024 (ML-KEM, ML-DSA). The security of these schemes rests on the actual cost of reducing lattice bases; however, the methods available to estimate it (worst-case bounds, the Gaussian heuristic, or the direct execution of LLL and BKZ) are, respectively, too conservative, insensitive to the specific instance, or computationally expensive.

This Bachelor's Thesis designs, implements, and empirically evaluates a supervised machine learning system that predicts the quality of the LLL reduction of a lattice solely from algebraic and geometric descriptors of its unreduced basis, without executing the algorithm. On a reproducible dataset of 224 000 instances, four structural lattice families (uniform, Gaussian, sparse, and q -ary), nine dimensions, and four values of the parameter δ , *Random Forest*, *gradient boosting* (XGBoost) and *Multilayer Perceptron* models are trained under both global and per-dimension strategies, adopting the root Hermite factor (RHF) as the governing metric.

The models predict the orthogonality defect and the norm of the first reduced vector with $R^2 \approx 0,9997$ and $R^2 \approx 0,98$, respectively; per-dimension specialisation raises the prediction of the RHF to $R^2 > 0,99$ for dimensions $d \geq 100$. Building on these predictions, an adaptive δ -selection oracle, in its strict regime ($\tau = 0$), selects a value $\delta < 0,99$ for 36,6 % of the bases with a mean quality degradation of only +0,123 % relative to the default value. Temporal validation on unseen bases shows that these selections yield moderate speed-ups in actual running time, up to $\approx 1,3\times$, with no appreciable loss of quality; on q -ary lattices, the family underlying the *Learning With Errors* problem on which the standardised post-quantum schemes are built, the oracle correctly retains $\delta = 0,99$, avoiding the degradation that a more aggressive reduction would introduce on those instances. An attribution analysis based on Shapley values reveals that the system does not reconstruct the volumetric law of the Gaussian heuristic, whose strong form is in fact refuted, but rather learns an instance-dependent refinement of the asymptotic theory of reduction, grounded in interpretable geometric properties of the basis. Together, these results position the proposal as an auxiliary tool for the security estimation of post-quantum cryptography.

Keywords: post-quantum cryptography, lattices, LLL algorithm, basis reduction, machine learning, Random Forest, Gradient Boosting, Shortest Vector Problem, Learning With Errors, security estimation.

DEDICATORIA

Le dedico este trabajo a mi madre, por su apoyo incondicional durante toda mi vida académica y durante el desarrollo de este proyecto, y a mi padre, por su ejemplo de esfuerzo, perseverancia y trabajo duro. A ambos, por enseñarme que la educación es el único valor que nadie te puede quitar, y que además es el motor que impulsa el progreso y la superación personal.

ÍNDICE GENERAL

1. INTRODUCCIÓN.	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Marco regulatorio y ético	4
1.3.1. Protección de datos y seguridad de la información	4
1.3.2. Control de exportación y uso dual	5
1.3.3. Estándares técnicos aplicables	5
1.3.4. Propiedad intelectual e industrial	6
1.3.5. Marco deontológico y compromiso ético	6
1.4. Estructura de la memoria	7
2. ESTADO DEL ARTE.	9
2.1. Criptografía clásica y la amenaza cuántica.	9
2.2. Retículos: definición y propiedades fundamentales	10
2.2.1. Definición formal	11
2.2.2. Multiplicidad de bases y matrices unimodulares	11
2.2.3. Dominio fundamental y determinante	12
2.2.4. Norma, vector más corto y mínimos sucesivos	12
2.2.5. Ortogonalización de Gram-Schmidt y perfil del retículo	13
2.2.6. Retículos q -arios	15
2.3. Problemas computacionales sobre retículos	15
2.3.1. El Problema del Vector más Corto (SVP)	16
2.3.2. El Problema del Vector más Cercano (CVP).	16
2.3.3. Learning With Errors (LWE)	17
2.4. Algoritmos de reducción de bases	18
2.4.1. El algoritmo LLL.	19
2.4.2. El algoritmo BKZ	20
2.4.3. Lattice sieving	21

2.5. Aplicación de Machine Learning a problemas sobre retículos	22
2.5.1. Ataques ML al problema LWE	22
2.5.2. Reducción de retículos mediante deep learning	23
2.5.3. Lattice sieving como algoritmo evolutivo	24
2.5.4. Predicción de la calidad de reducción LLL: el hueco en la literatura	24
2.6. Herramientas y tecnologías relacionadas.	25
2.6.1. Bibliotecas de reducción de retículos: fplll y fpylll	25
2.6.2. Aprendizaje automático: XGBoost y scikit-learn	26
2.6.3. Redes neuronales: PyTorch	27
2.6.4. Estimación concreta de seguridad: el <i>lattice-estimator</i>	27
2.6.5. Esquemas post-cuánticos estandarizados: ML-KEM y ML-DSA	28
3. METODOLOGÍA Y DISEÑO DEL SISTEMA	29
3.1. Visión general del sistema.	29
3.1.1. Justificación de las decisiones de diseño	31
3.2. Generación del dataset.	32
3.2.1. Tipos de retículo	32
3.2.2. Dimensiones y volumen del dataset	34
3.2.3. Extracción de características	35
3.2.4. Variables objetivo	36
3.3. Modelos de Machine Learning	37
3.3.1. Modelos globales.	38
3.3.2. Modelos por dimensión	39
3.4. Selección adaptativa del parámetro δ	39
3.4.1. Formulación inicial: regresión del RHF con regla de umbral	40
3.4.2. Reformulación como clasificación supervisada de δ^*	40
3.4.3. Oráculo de referencia y métricas de evaluación	41
3.4.4. Protocolo experimental	42
3.5. Entorno de desarrollo y reproducibilidad	43
4. RESULTADOS Y EVALUACIÓN.	46
4.1. Resultados de los modelos globales	46
4.2. Resultados de los modelos por dimensión	47

4.3. Análisis de importancia de features.	53
4.4. Evaluación de la selección adaptativa de δ	64
4.4.1. Diagnóstico de la formulación inicial	64
4.4.2. Comparación de las dos formulaciones	65
4.4.3. Validación del oráculo reformulado	67
4.4.4. Limitaciones y trabajo futuro	71
4.5. Discusión de resultados	72
5. PLANIFICACIÓN Y PRESUPUESTO	77
5.1. Planificación temporal	77
5.2. Presupuesto	80
5.3. Impacto social y ambiental	82
6. CONCLUSIONES Y TRABAJO FUTURO	85
6.1. Conclusiones	85
6.2. Trabajo Futuro	87
BIBLIOGRAFÍA	91

ÍNDICE DE FIGURAS

3.1	Pipeline completo del sistema. Las flechas indican el flujo de datos; la base original alimenta simultáneamente la ejecución de LLL (para obtener las etiquetas o targets) y la extracción de descriptores (para construir las entradas al modelo).	29
4.1	Dispersión de valores predichos frente a valores reales del modelo global (XGBRegressor) sobre el conjunto de test, para los tres <i>targets</i> : log defecto de ortogonalidad, $\log \ \mathbf{b}_1^{\text{red}}\ $ y RHF. La línea discontinua representa la identidad $y = x$ (predicción perfecta). La mayor dispersión del RHF respecto a la diagonal refleja su menor coeficiente de determinación a escala global.	47
4.2	MAE en test de los modelos por dimensión y del modelo global, evaluados sobre las <i>mismas</i> bases de test de cada dimensión, para los tres <i>targets</i> (eje vertical en escala logarítmica). Comparar sobre instancias idénticas neutraliza el efecto de la varianza y hace que las diferencias de error sean directamente interpretables.	50
4.3	Comparativa del R^2 por dimensión entre el MLP y las dos variantes de XGBoost por dimensión, para los tres <i>targets</i> . Para el RHF (panel derecho) se aprecia la divergencia del MLP a partir de $d = 100$, donde XGBoost mantiene $R^2 > 0,99$ mientras el MLP se degrada progresivamente.	51
4.4	R^2 del RHF por dimensión desglosado por tipo de retículo, para el modelo XGBRegressor por dimensión. El desglose intra-tipo revela valores sensiblemente inferiores al R^2 agregado y el colapso de la métrica en los retículos q -arios a dimensiones altas.	53
4.5	Importancia de <i>ganancia</i> de XGBoost (<i>gain</i> normalizado), promediada de forma ponderada sobre las dimensiones $d \in \{10, \dots, 200\}$, para las quince <i>features</i> de mayor peso. (a) Al incluir las indicadoras <i>one-hot</i> del tipo, la del tipo q -ario sobrepondera de forma artefactual el resto de predictores. (b) Al excluirlas y ponderar nuevamente, la jerarquía entre <i>features</i> geométricas comparte con la obtenida mediante SHAP un núcleo común de predictores, aunque no la <i>feature</i> principal, lo que evidencia el sesgo de la métrica de <i>ganancia</i> y justifica relegarla al papel de control.	55

- 4.6 Importancia SHAP global (media de valor absoluto, ponderada sobre dimensiones) para los tres *targets*, mostrando las quince *features* de mayor peso. El recuadro de cada panel indica la entropía de Shannon H de la distribución de importancias. El defecto se explica esencialmente por una única *feature* estructural (H baja), mientras que $\log \|\mathbf{b}_1^{\text{red}}\|$ y el RHF reparten la atribución entre más predictores (H alta). 56
- 4.7 Diagrama SHAP *beeswarm* para $\log \delta(\mathbf{B}_{\text{red}})$ en $d = 80$. La *feature* dominante es la fracción de coeficientes de Gram–Schmidt con $|\mu_{i,j}| > 0,5$: los valores altos (puntos rojos) producen contribuciones SHAP fuertemente positivas y los bajos (puntos azules), marcadamente negativas, lo que evidencia una relación monótona creciente entre la lejanía de la base de entrada respecto de la condición de *size-reduction* ($|\mu_{i,j}| \leq 1/2$, justamente la que el algoritmo LLL impone) y el defecto de ortogonalidad alcanzado tras la reducción. El modelo se apoya así en un *proxy* estructural directo de cuán reducida está la base de partida, en lugar de reconstruir término a término la identidad $\log \delta(\mathbf{B}) = \sum_i \log \|\mathbf{b}_i\| - \log |\det(\mathbf{B})|$. El parámetro de reducción δ aparece en segundo plano con un efecto *inverso* sobre el defecto: los valores altos (puntos rojos), que refuerzan la condición de Lovász y producen una base más ortogonal, desplazan la predicción hacia defectos menores (SHAP negativo), mientras que los bajos la elevan. La concentración casi total de la atribución en una única *feature* es coherente con la mínima entropía de este *target* (Figura 4.6). 57
- 4.8 Diagrama SHAP *beeswarm* para $\log \|\mathbf{b}_1^{\text{red}}\|$ en $d = 80$. La atribución se reparte entre varios predictores, en coherencia con la elevada entropía de este *target*. El parámetro δ encabeza la jerarquía con un efecto *inverso*: los valores altos (puntos rojos), asociados a una reducción más agresiva, desplazan la predicción hacia log-normas menores (SHAP negativo), mientras que los valores bajos la elevan. La *feature* $\min_i \|\mathbf{b}_i\|$ exhibe, por el contrario, una relación *directa* y nítida: una norma mínima de entrada pequeña (puntos azules) arrastra sistemáticamente la predicción a la baja, en coherencia con que un vector de partida ya corto impone una cota inferior a la longitud alcanzable tras la reducción. El logaritmo del determinante $\log |\det(\mathbf{B})|$, predictor que la heurística Gaussiana situaría en cabeza, presenta un efecto comparativamente débil y de signo poco definido, lo que refuta la forma fuerte de dicha heurística a d fijo: con el volumen del retículo de variabilidad acotada dentro de cada dimensión, la log-norma queda gobernada por la agresividad de la reducción y por la estructura métrica de la base de entrada, no por el término volumétrico. 58

4.9	Diagrama SHAP <i>beeswarm</i> para el RHF en $d = 80$. La fracción de coeficientes $ \mu_{i,j} > 0,5$ encabeza la atribución con un efecto monótono creciente y claramente separado del resto: las instancias con valores altos (puntos rojos) reciben las contribuciones positivas de mayor amplitud. El parámetro δ ocupa la segunda posición, pero contribuye con ambos signos, lo que apunta a un efecto mediado por interacciones con la estructura de la base más que a una dependencia directa de signo constante. Por detrás emergen descriptores estructurales finos, la indicadora del tipo q -ario y varias magnitudes del perfil de Gram–Schmidt (pendiente, adyacencia y dispersión de residuos), en coherencia con que el RHF, al normalizar $\ \mathbf{b}_1^{\text{red}}\ $ por $ \det(\Lambda) ^{1/d}$, cancela el término volumétrico dominante y deja la varianza residual gobernada por la geometría de la base. Nótese la escala SHAP comprimida ($\approx \pm 0,015$) frente a la del <i>beeswarm</i> de la norma, reflejo de la reducida varianza intrínseca de este <i>target</i>	59
4.10	Evolución de la importancia SHAP (media de $ \text{SHAP} $) de las ocho <i>features</i> principales de cada <i>target</i> en función de la dimensión d . Se observa el relevo de predictores con la dimensión, especialmente acusado para $\log \ \mathbf{b}_1^{\text{red}}\ $, lo que concuerda con su mayor entropía de atribución. Nótese el contraste de escalas verticales: la atribución del defecto crece sin saturación con d , mientras que la del RHF se colapsa.	60
4.11	Efecto intra-base de δ en la predicción del modelo RHF (azul), comparado con el MAE del modelo (rojo) y la desviación estándar real entre bases (verde), por dimensión. El rango intra-base es prácticamente nulo, lo que evidencia que el modelo no captura la señal de δ	65
4.12	Distribución de δ^* (global, por tipo de retículo y por dimensión). El $\delta = 0,99$ no es el óptimo para el 47,1 % de las bases.	65
4.13	Frontera de Pareto: degradación media $\Delta q_{\text{vs}0,99}$ frente al porcentaje de bases en que se elige $\delta < 0,99$ ($\tau = 0$). El clasificador δ^* (Opción B) se sitúa entre los baselines conservadores y la cota de fuerza bruta, con una degradación muy inferior a <i>siempre</i> $\delta = 0,55$	67
4.14	Efecto de τ sobre el porcentaje de bases con $\delta < 0,99$, la exactitud y la degradación de calidad del clasificador δ^* , comparado con la cota de fuerza bruta y los baselines triviales. El parámetro τ produce una transición abrupta en $\tau \approx 0,01$	68
4.15	Matrices de confusión del clasificador δ^* ($\tau = 0$): por conteos (izquierda) y normalizada por fila (derecha). Los errores se concentran entre las clases $\delta = 0,90$ y $\delta = 0,99$	69
4.16	Mapa de calor de la exactitud (izquierda) y la degradación media $\Delta q_{\text{vs}0,99}$ (derecha) del clasificador δ^* por dimensión d y tipo de retículo ($\tau = 0$). . .	70

4.17	Aceleración (mediana $\pm$ IQR, izquierda) y porcentaje de tiempo ahorrado (mediana, derecha) del clasificador δ^* frente a $\delta = 0,99$ sobre retículos frescos, por tipo y dimensión.	71
5.1	Diagrama de Gantt de la planificación temporal del proyecto (1 de febrero a 12 de junio de 2026; el mes de junio aparece truncado en la fecha de entrega). Las barras representan actividades; los rombos, los hitos principales y las reuniones de seguimiento con el tutor. El solapamiento entre fases refleja el carácter iterativo e incremental del desarrollo.	79

ÍNDICE DE TABLAS

3.1	Distribución de muestras por dimensión en <code>lattice_dataset_v2.csv</code> . Cada fila del dataset corresponde a un par (base, δ); cada matriz base origina 4 filas (una por valor de δ).	35
3.2	Variables objetivo extraídas sobre la base reducida $\mathbf{B}_{\text{red}}$	36
3.3	Dependencias principales del entorno <code>delta_predictor_release</code>	44
4.1	Coeficiente de determinación R^2 de los modelos globales sobre el conjunto de test (20 % del dataset v2, 224 000 instancias totales).	46
4.2	R^2 de los modelos por dimensión para las dos variantes de XGBoost: RF (XGBRFRegressor) y GB (XGBRegressor). La última fila reproduce el R^2 del modelo global como referencia, aunque no es directamente comparable ya que es un agregado.	48
4.3	Modelos por dimensión y global del RHF evaluados sobre las <i>mismas</i> bases de test de cada dimensión. El MAE se expresa en unidades de 10^{-3} . El R^2 es equiparable en dimensiones altas, mientras que el error absoluto del modelo por dimensión es sistemáticamente menor.	49
4.4	R^2 del RHF por dimensión: comparativa entre XGBRFRegressor, XGBRegressor y MLP por dimensión. Se marca con $\checkmark$ cuando el MLP es competitivo ($ \Delta < 0,01$) y con $\times$ cuando presenta degradación significativa.	51
4.5	R^2 del RHF por dimensión y tipo de retículo (XGBRegressor por dimensión), evaluado de forma estratificada sobre el conjunto de test.	52
4.6	Contraste entre las hipótesis iniciales sobre la <i>feature</i> SHAP dominante y los resultados observados para cada <i>target</i>	63
4.7	Comparativa de estrategias de selección de δ ($\tau = 0$, conjunto de test, $n = 11\,200$ bases). Acc. = fracción de aciertos sobre $\delta^*(\tau = 0)$; OB1 = tolerancia ± 1 nivel de δ	66
5.1	Hitos principales del proyecto.	78
5.2	Costes de personal.	80
5.3	Amortización del equipo informático.	81
5.4	Presupuesto total del proyecto.	82

1. INTRODUCCIÓN

1.1. Motivación

La seguridad de la infraestructura digital contemporánea (desde el comercio electrónico y la banca en línea hasta las comunicaciones gubernamentales) descansa, en última instancia, sobre la dificultad computacional de un reducido conjunto de problemas matemáticos, señaladamente la factorización de enteros y el cálculo del logaritmo discreto, en los que se fundamentan esquemas de clave pública tan extendidos como RSA o Diffie–Hellman. Esta confianza, sin embargo, es de naturaleza condicional: el algoritmo de Shor [1] demuestra que un computador cuántico de escala suficiente resolvería ambos problemas en tiempo polinómico y comprometería, por tanto, en la práctica, la totalidad de los criptosistemas de clave pública desplegados. La amenaza no es meramente prospectiva, pues la estrategia conocida como *harvest now, decrypt later* expone desde hoy cualquier información cuya confidencialidad deba preservarse a largo plazo. El alcance de esta amenaza se analiza en detalle en la Sección 2.1.

Frente a este horizonte, la criptografía post-cuántica persigue sustituir las primitivas vulnerables por esquemas resistentes a adversarios cuánticos. Este esfuerzo culminó en 2024 con la publicación, por parte del *National Institute of Standards and Technology* (NIST), de los tres primeros estándares post-cuánticos: ML-KEM (FIPS 203) para el encapsulado de claves y ML-DSA (FIPS 204) para la firma digital, ambos basados en retículos, junto con SLH-DSA (FIPS 205), un esquema de firma basado en funciones *hash* [2], [3], [4], [5]. Los dos primeros pertenecen a la familia de la criptografía basada en retículos, consolidada como la candidata de referencia para la era post-cuántica por su eficiencia, la versatilidad de sus construcciones y su aparente resistencia frente al cómputo cuántico [6], [7], [8].

La seguridad de estos esquemas no reposa, no obstante, sobre una garantía absoluta, sino sobre la dificultad práctica de los problemas reticulares que los sustentan, en particular el problema *Learning With Errors* (Sección 2.3.3), cuyos mejores ataques conocidos se reducen, en su núcleo, a la reducción de bases de retículos mediante algoritmos como LLL y BKZ (Sección 2.4). En consecuencia, calibrar el nivel de seguridad efectivo de un conjunto de parámetros equivale, en buena medida, a anticipar la calidad que alcanzará la reducción de una base en dimensiones criptográficamente relevantes: cuanto mejor sea la reducción que un adversario pueda obtener, menor será el margen de seguridad real del esquema.

Esta dependencia sitúa la estimación de la calidad de reducción en el centro del análisis de seguridad, y es precisamente ahí donde los métodos disponibles muestran sus limitaciones. Las cotas teóricas del peor caso resultan excesivamente conservadoras, ya que el comportamiento de los algoritmos de reducción sobre instancias aleatorias es no-

tablemente mejor que el garantizado por dichas cotas; la heurística Gaussiana ofrece una estimación promedio más ajustada, pero ignora por completo la estructura específica de cada instancia; y la ejecución directa de LLL o BKZ, aunque fiable, conlleva un coste computacional que se vuelve prohibitivo cuando debe repetirse sobre grandes volúmenes de instancias, como exige la exploración sistemática de espacios de parámetros. Este compromiso entre precisión, sensibilidad a la instancia y coste, examinado con mayor detalle en la Sección 2.5.4, delimita el problema que motiva el presente trabajo.

Más allá de su interés teórico, la cuestión posee una relevancia profesional inmediata. La migración hacia la criptografía post-cuántica constituye una de las transformaciones de ingeniería de mayor calado en curso sobre las infraestructuras de seguridad, con implicaciones directas en protocolos y sistemas de uso masivo como TLS, las infraestructuras de clave pública (PKI) o las comunicaciones de Estado. En este escenario, la selección responsable de los parámetros de los esquemas estandarizados y la verificación de sus márgenes de seguridad reclaman herramientas de estimación que sean, a la vez, precisas, sensibles a las particularidades de cada instancia y computacionalmente asequibles. Disponer de procedimientos que satisfagan simultáneamente estas tres exigencias es, hoy, un problema abierto y de impacto práctico considerable, y es esta necesidad (la de estimar con rigor y a bajo coste la dificultad real de reducir una base de retículo) la que confiere sentido y pertinencia al trabajo desarrollado en esta memoria.

1.2. Objetivos

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y evaluar empíricamente un sistema de aprendizaje automático supervisado capaz de predecir, a partir únicamente de la base sin reducir y sin ejecutar el algoritmo, la calidad que alcanzará la reducción LLL de un retículo, así como de aprovechar dicha capacidad predictiva para seleccionar de forma adaptativa el parámetro δ de LLL. Con ello se busca responder al compromiso entre precisión, sensibilidad a la instancia y coste computacional que la Sección 1.1 identifica como problema abierto: un estimador aprendido proporciona una predicción sensible a la geometría concreta de cada base (a diferencia de las cotas del peor caso o de la heurística Gaussiana) a un coste de inferencia despreciable frente a la ejecución directa de la reducción. El trabajo se plantea, por tanto, como una prueba de viabilidad de la predicción de la calidad de reducción en cuanto herramienta auxiliar para el análisis de seguridad de la criptografía basada en retículos, y no como un ataque criptoanalítico completo.

Este objetivo general se concreta en los siguientes objetivos específicos, formulados de modo que su grado de consecución pueda verificarse a partir de los resultados del Capítulo 4:

1. Desarrollar generadores de bases de retículo para cuatro familias estructurales (uniforme, gaussiana, *sparse* y *q*-aria) y nueve dimensiones, y reducir cada base con

cuatro valores del parámetro δ mediante la biblioteca `fpv111` [9], conformando un conjunto de datos etiquetado de 224 000 instancias con semillas fijadas que garanticen su reproducibilidad (Sección 3.2).

2. Diseñar y extraer un vector de descriptores algebraicos y geométricos de la base original, antes de aplicar reducción alguna, fundamentado en el perfil de Gram-Schmidt y en magnitudes de ortogonalidad, que actúe como entrada de baja dimensionalidad a los modelos y admita una interpretación criptográfica (Sección 3.2.3).
3. Caracterizar la calidad de la reducción mediante el defecto de ortogonalidad, la norma del primer vector reducido y el *root Hermite factor* (RHF), adoptando este último, por tratarse de una medida normalizada e independiente de la dimensión [10], como métrica rectora del sistema (Sección 3.2.4).
4. Entrenar y evaluar modelos de bosque aleatorio (*Random Forest*) y de *gradient boosting* sobre árboles (implementados con `XGBoost` [11]) y, como extensión exploratoria, un perceptrón multicapa (*Multilayer Perceptron*), bajo dos estrategias de entrenamiento: un modelo global y modelos por dimensión. La capacidad predictiva se cuantifica con el coeficiente de determinación R^2 sobre datos no vistos, empleando una partición *por base* que evita la fuga de información (*data leakage*) entre los conjuntos de entrenamiento y prueba (Sección 3.3).
5. Construir, sobre las predicciones del RHF, un oráculo que seleccione para cada base el menor valor de δ cuya calidad estimada no se degrade más de un umbral relativo τ respecto del valor por defecto, y cuantificar el compromiso resultante entre la frecuencia con que se selecciona un $\delta < 0,99$, la degradación de calidad y la aceleración del tiempo de ejecución real (Sección 3.4).
6. Analizar la importancia y el efecto de las características mediante técnicas de atribución, en particular valores de Shapley [12], a fin de relacionar las predicciones con la geometría del retículo y extraer una lectura de los resultados en términos de estimación de seguridad criptográfica (Sección 4.3).

A estos objetivos técnicos se suma un objetivo transversal de carácter profesional: enmarcar el trabajo en el contexto normativo, ético y de impacto que corresponde a un proyecto de ingeniería informática (señaladamente la naturaleza de uso dual del criptoanálisis, el empleo de software de código abierto y la relación del trabajo con el proceso de estandarización de la criptografía post-cuántica), según se desarrolla en la Sección 1.3.

Conviene, por último, delimitar el alcance de estos objetivos. El estudio se circunscribe al algoritmo LLL y a las distribuciones de retículos generadas para este trabajo; la extensión a algoritmos de reducción más potentes como BKZ (más próximos a los niveles de seguridad efectivamente estandarizados) y el estudio sistemático de la transferibilidad entre distribuciones distintas quedan fuera del ámbito de la memoria y se recogen como líneas de trabajo futuro. El grado de consecución de cada uno de los objetivos enunciados

se valora de forma explícita en el Capítulo 6, con el que se mantiene la coherencia exigida entre lo propuesto y lo alcanzado.

1.3. Marco regulatorio y ético

En cuanto proyecto de naturaleza profesional en el ámbito de la ingeniería informática, este trabajo ha de situarse en el contexto normativo y deontológico que le es propio. Su relación con la regulación presenta, no obstante, una particularidad: el trabajo no trata datos personales ni entrega un producto desplegable, sino que opera sobre la materia que buena parte de la normativa de seguridad digital presupone (la criptografía) y, más concretamente, sobre las herramientas que sirven para estimar su fortaleza. Esta sección examina, en este orden, la legislación aplicable en materia de protección de datos y seguridad de la información, el régimen de control de exportación y uso dual, los estándares técnicos de referencia, la propiedad intelectual e industrial del instrumental y de los resultados, y el marco deontológico que rige la práctica. El análisis aplicado del impacto social y ambiental del trabajo, así como la argumentación sustantiva sobre el riesgo de uso indebido y su mitigación, se desarrollan en la Sección 5.3, a la que se remite para evitar duplicidades.

1.3.1. Protección de datos y seguridad de la información

El conjunto de datos empleado se compone exclusivamente de bases de retículo generadas de forma sintética con semillas fijadas (Sección 3.2), por lo que el trabajo no realiza tratamiento alguno de datos personales y queda, en sentido estricto, fuera del ámbito de aplicación del Reglamento (UE) 2016/679 (RGPD) [13] y de la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [14]. La conexión con esta normativa es, sin embargo, material y no meramente formal: la criptografía constituye la salvaguarda técnica que dichas normas presuponen. El propio artículo 32 del RGPD enumera el cifrado de datos personales entre las medidas de seguridad del tratamiento exigibles al responsable, de modo que la robustez de los esquemas criptográficos condiciona de forma directa la posibilidad de cumplir las obligaciones de confidencialidad e integridad que el Reglamento impone. La irrupción del cómputo cuántico (Sección 2.1) desplaza este requisito hacia las primitivas post-cuánticas, cuya calibración de seguridad es precisamente el sector de aplicación del presente trabajo.

En el plano nacional, el Esquema Nacional de Seguridad, regulado por el Real Decreto 311/2022 [15], obliga a las administraciones públicas y a las entidades de su sector a emplear mecanismos criptográficos acreditados para la protección de la información, y es sensible al horizonte post-cuántico en la medida en que la estrategia *harvest now, decrypt later* compromete desde hoy la confidencialidad a largo plazo de la información cifrada con esquemas clásicos. La selección responsable de los parámetros de los esquemas reticulares que sustituirán a los actuales (tarea a la que este trabajo aporta una herramien-

ta auxiliar de estimación) es, por tanto, un requisito previo para la futura adopción de mecanismos criptográficos aprobados en el marco del Esquema Nacional de Seguridad.

1.3.2. Control de exportación y uso dual

Las herramientas criptográficas y criptoanalíticas tienen la consideración de productos de doble uso, sujetos en el ámbito de la Unión Europea al Reglamento (UE) 2021/821 [16], que establece un régimen común de control de las exportaciones y refunde la normativa anterior. Su Anexo I, derivado del Acuerdo de Wassenaar, incluye el soporte lógico y la tecnología de seguridad de la información en su Categoría 5, lo que en principio sometería a control la transferencia de cierto software criptográfico hacia destinos fuera del territorio aduanero de la Unión. El propio Reglamento contempla, no obstante, dos exenciones de control directamente pertinentes para un trabajo de esta naturaleza: la del *dominio público* y la de la *investigación científica básica*, cuyo alcance precisan las orientaciones de la Recomendación (UE) 2021/1700. El presente trabajo se apoya de forma exclusiva en algoritmos y bibliotecas de dominio público (Sección 2.6), se inscribe en la investigación académica y documenta sus resultados de manera reproducible, por lo que se acoge a dichas exenciones y no requiere autorización de exportación.

Esta circunstancia ilustra la ambivalencia característica del criptoanálisis, en el que un mismo conocimiento puede orientarse a fortalecer un sistema o a atacarlo. El análisis sustantivo de ese riesgo de uso indebido (y de los rasgos del trabajo que lo acotan, señaladamente su carácter predictivo y no recuperatorio, el régimen dimensional reducido de los experimentos y su transparencia) se desarrolla en la Sección 5.3.

1.3.3. Estándares técnicos aplicables

Los estándares que constituyen el referente último del trabajo son los dos estándares post-cuánticos basados en retículos publicados por el NIST en 2024: ML-KEM (FIPS 203) para el encapsulado de claves y ML-DSA (FIPS 204) para la firma digital [2], [3], descritos con mayor detalle en la Sección 2.6.5. Se trata de estándares abiertos, fruto de un proceso de normalización público y transparente que es, en sí mismo, una forma de criptoanálisis abierto: la confianza en estos esquemas descansa sobre el escrutinio sostenido de la comunidad, y no sobre el secreto de su diseño. El trabajo no implementa ninguno de estos estándares, pero las magnitudes que sus modelos aprenden a predecir (la calidad de reducción de una base) son las que, en última instancia, determinan el margen de seguridad de las primitivas que dichos estándares regulan. La normalización internacional de la criptografía post-cuántica prosigue además en organismos como ISO/IEC, lo que refuerza la pertinencia de disponer de herramientas de estimación de seguridad sensibles a la instancia.

1.3.4. Propiedad intelectual e industrial

El instrumental software empleado es íntegramente de código abierto, circunstancia que elimina restricciones de propiedad intelectual sobre las herramientas y garantiza la reproducibilidad de los experimentos. La biblioteca de reducción de retículos `fp111` se distribuye bajo licencia LGPL-2.1, y la interfaz de Python efectivamente utilizada, `fpyl11`, bajo licencia GPL-2.0 [9]; los modelos de conjunto se construyen con XGBoost (licencia Apache-2.0) [11] y con la infraestructura de soporte de `scikit-learn` (licencia BSD de tres cláusulas) [17]; y la extensión exploratoria con redes neuronales emplea PyTorch, también bajo licencia BSD [18]. El uso realizado en este trabajo (de carácter investigador y sin redistribución de obra derivada) se ajusta a los términos de estas licencias, que se citan en la bibliografía a efectos de trazabilidad.

En lo relativo a la propiedad industrial, conviene precisar que los esquemas estandarizados se publican como estándares abiertos concebidos para una implementación libre de regalías, una vez gestionados por el NIST los derechos de patente concurrentes durante el proceso de normalización [5]. Por último, los productos originales de este Trabajo Fin de Grado (el código de generación y análisis a excepción de `lattice_utils.py`, el conjunto de datos etiquetado y los resultados) son obra propia del autor; su reproducibilidad queda garantizada mediante la fijación de semillas (Sección 3.2), y la memoria se depositará en el archivo institucional de la Universidad bajo la correspondiente licencia Creative Commons, conforme a la normativa de la Escuela Politécnica Superior.

1.3.5. Marco deontológico y compromiso ético

Más allá de la legislación vigente, la práctica profesional de la ingeniería informática se rige por códigos deontológicos como el Código Ético del IEEE [19] y el Código Ético de la ACM [20], que comprometen al profesional a actuar en interés público, a evitar daños y a obrar con honestidad e integridad. Aplicados a la investigación en seguridad, estos principios respaldan el rechazo de la *seguridad por oscuridad*: la evaluación abierta de la fortaleza de los esquemas criptográficos beneficia, antes que a ningún otro agente, a quien los diseña y despliega, en coherencia con el principio de Kerckhoffs sobre el que se asienta la criptografía moderna. El planteamiento de este trabajo se ajusta a dicho marco: su finalidad es la calibración defensiva de la seguridad, opera sobre algoritmos y datos de dominio público y expone sus métodos y resultados de forma verificable. La concreción de este compromiso en el análisis del impacto del trabajo y en la valoración de su uso responsable se desarrolla en la Sección 5.3, con la que esta sección mantiene una división deliberada de cometidos: aquí se fija el marco de normas y deberes; allí, su aplicación al caso concreto.

1.4. Estructura de la memoria

Expuestos la motivación, los objetivos y el marco normativo del trabajo, este último apartado describe la organización del resto de la memoria. El documento sigue la estructura recomendada para los Trabajos Fin de Grado de la Escuela Politécnica Superior y se articula en seis capítulos, completados por la bibliografía, un anexo de código fuente y un anexo de declaración de uso de IA generativa. Su hilo conductor reproduce el recorrido natural de un proyecto de ingeniería, del planteamiento del problema a su resolución y evaluación, de modo que cada capítulo se apoya en los precedentes y prepara los siguientes.

El presente Capítulo 1 sitúa y delimita el trabajo. La Sección 1.1 expone la motivación, ligando la amenaza del cómputo cuántico sobre la criptografía de clave pública con la necesidad de estimar a bajo coste la calidad de la reducción de bases de retículo; la Sección 1.2 fija el objetivo general y los objetivos específicos, formulados de manera que su grado de consecución sea verificable; y la Sección 1.3 encuadra el proyecto en el contexto regulatorio, deontológico y de propiedad intelectual que corresponde a un trabajo de naturaleza profesional en ingeniería informática.

El Capítulo 2 construye el marco teórico sobre el que se sustenta el desarrollo. Parte de la amenaza cuántica y la respuesta post-cuántica, introduce los fundamentos de la teoría de retículos (bases, determinante, mínimos sucesivos, ortogonalización de Gram-Schmidt y factor de Hermite) y los problemas computacionales que dotan de seguridad a estos esquemas (SVP, CVP y, en particular, *Learning With Errors*). A continuación presenta los algoritmos de reducción de bases (LLL, BKZ y *sieving*) y revisa de forma crítica las aplicaciones recientes del aprendizaje automático a problemas reticulares, lo que permite identificar con precisión el hueco en la literatura que este trabajo aborda. El capítulo se cierra con las herramientas y tecnologías sobre las que se apoya la implementación.

El Capítulo 3 constituye el núcleo metodológico y describe el diseño del sistema. Tras una visión general de la arquitectura, detalla la generación del conjunto de datos (cuatro familias estructurales de retículos y nueve dimensiones, reducidas con varios valores del parámetro δ hasta conformar un *dataset* etiquetado y reproducible (Sección 3.2)) la representación geométrica de la base sin reducir empleada como entrada y las variables objetivo que caracterizan la calidad de la reducción, con el *root Hermite factor* como métrica rectora. Se exponen después las familias de modelos consideradas y las dos estrategias de entrenamiento, global y por dimensión, (Sección 3.3), el diseño del oráculo de selección adaptativa de δ (Sección 3.4) y, por último, el entorno de desarrollo y las medidas que garantizan la reproducibilidad de los experimentos.

El Capítulo 4 presenta la evaluación empírica del sistema y discute sus implicaciones. Reporta el desempeño predictivo de los modelos globales y por dimensión sobre datos no vistos, analiza la importancia y el efecto de las características mediante valores de Shapley (Sección 4.3) (de donde emerge la conexión con la heurística Gaussiana como

hallazgo interpretativo) y cuantifica el compromiso entre ahorro de cómputo, degradación de calidad y aceleración que ofrece la selección adaptativa de δ . Una sección de discusión integra estos resultados, los confronta con el estado del arte y extrae su lectura en términos de estimación de seguridad criptográfica.

El Capítulo 5 aborda la dimensión de gestión y responsabilidad profesional del proyecto. Recoge la planificación temporal del trabajo, su presupuesto detallado y el análisis del impacto social y ambiental (Sección 5.3), incluida la valoración del carácter de uso dual del criptoanálisis y de los rasgos del trabajo que acotan ese riesgo.

El Capítulo 6 cierra la memoria con un balance del trabajo realizado. Revisa el grado de consecución de cada uno de los objetivos específicos enunciados en la Sección 1.2 y plantea las líneas de trabajo futuro (Sección 6.2), entre ellas la extensión a dimensiones mayores, a métricas de perfil completo y al algoritmo BKZ.

La memoria concluye con la bibliografía, que reúne las referencias citadas siguiendo el estilo IEEE, y con el Anexo A, que reproduce los fragmentos de código más representativos del desarrollo a efectos de trazabilidad y reproducibilidad.

2. ESTADO DEL ARTE

2.1. Criptografía clásica y la amenaza cuántica

La criptografía constituye el conjunto de técnicas que protegen la comunicación digital frente a terceros no autorizados, garantizando que únicamente las partes legítimas puedan acceder al contenido de los mensajes intercambiados [21]. Desde la formulación del paradigma de clave pública por Diffie y Hellman en 1976 [22], la seguridad de las comunicaciones a gran escala, desde el comercio electrónico hasta las infraestructuras de Estado, ha descansado de manera casi universal sobre un reducido número de problemas matemáticos. Los esquemas hoy desplegados de forma mayoritaria, como el cifrado RSA [23] o el intercambio de claves de Diffie–Hellman [22], fundamentan su seguridad en la presunta dificultad computacional de dos problemas estrechamente relacionados: la factorización de números enteros y el cálculo del logaritmo discreto. La premisa es que, con los parámetros adecuados, no existe ningún algoritmo clásico capaz de resolver estos problemas en un tiempo razonable, de modo que un adversario con recursos de cómputo convencionales no puede recuperar la clave privada a partir de la información pública.

Esta confianza, sin embargo, es de naturaleza condicional: depende por completo de las capacidades del modelo de cómputo del adversario. El trabajo seminal de Shor en 1997 [1] demostró de forma teórica que tanto la factorización de enteros como el problema del logaritmo discreto admiten algoritmos de tiempo polinómico cuando se ejecutan sobre un computador cuántico. La consecuencia es directa y de gran calado: la mayor parte de los sistemas de clave pública actualmente en uso quedarían completamente comprometidos en el momento en que se disponga de un computador cuántico de escala suficiente [21]. Conviene subrayar el contraste cuantitativo que introduce el algoritmo de Shor. Frente a los mejores ataques clásicos conocidos, cuyo coste sobre RSA o sobre el logaritmo discreto es de carácter subexponencial respecto del parámetro de seguridad, el algoritmo cuántico reduce dicho coste a una dependencia meramente polinómica [21]. Es precisamente este salto de complejidad (de subexponencial a polinómica) el que convierte en inviables los esquemas clásicos en el escenario cuántico, y no una mera aceleración marginal.

La amenaza, además, no es puramente prospectiva. Aunque la construcción de un computador cuántico criptográficamente relevante pueda demorar años, el riesgo opera ya en el presente. Un adversario puede almacenar hoy comunicaciones cifradas con tecnología clásica y posponer su descifrado hasta el momento en que disponga de la capacidad cuántica necesaria [21]. Esta estrategia, conocida como *harvest now, decrypt later* (recoger ahora, descifrar después), implica que cualquier información cuya confidencialidad deba preservarse a largo plazo se encuentra expuesta desde este mismo instante. Por consiguiente, los métodos criptográficos clásicos representan un riesgo tangible incluso antes de que la propia amenaza cuántica se materialice de forma efectiva.

Ante este horizonte, la comunidad académica y la industria han redirigido su atención hacia el diseño de primitivas criptográficas resistentes a adversarios cuánticos, agrupadas bajo la denominación de criptografía post-cuántica (*Post-Quantum Cryptography*, PQC). Este esfuerzo se ha canalizado a través de iniciativas de estandarización de alcance internacional, entre las que destacan el proceso promovido por el *National Institute of Standards and Technology* (NIST) de los Estados Unidos [5] y los trabajos del *European Telecommunications Standards Institute* (ETSI) en Europa [24]. El objetivo de estas iniciativas es sustituir, de manera ordenada y verificable, los algoritmos vulnerables por nuevos esquemas cuya seguridad no se vea amenazada por el cómputo cuántico.

Entre las distintas familias propuestas para la era post-cuántica, la criptografía basada en retículos se ha consolidado como una de las candidatas de referencia, al ofrecer construcciones versátiles, eficientes y con tamaños de clave reducidos [6], [7], [8]. Su atractivo no radica únicamente en su eficiencia práctica, sino en una propiedad fundamental: a diferencia de la factorización o el logaritmo discreto, los problemas sobre retículos que sustentan estos esquemas no parecen admitir aceleraciones significativas mediante algoritmos cuánticos. La técnica de búsqueda de periodicidad en la que se apoya el algoritmo de Shor no resulta aplicable a la estructura de los retículos, y los intentos de resolver estos problemas por vía cuántica, ensayados desde el propio descubrimiento de Shor a mediados de la década de 1990, no han logrado mejoras sustanciales sobre los mejores métodos clásicos [6]. Aun cuando se conocen algoritmos cuánticos ligeramente más rápidos para ciertos problemas reticulares, su coste temporal sigue creciendo de forma exponencial con la dimensión del retículo, en marcado contraste con la reducción a coste polinómico que el cómputo cuántico provoca sobre los esquemas clásicos [21]. Es esta resistencia, sustentada tanto en la evidencia experimental acumulada como en la ausencia de avances algorítmicos relevantes, la que confiere a la criptografía basada en retículos su carácter de candidata sólida para el mundo post-cuántico, y la que justifica el interés por analizar con rigor la dificultad real de los problemas que la fundamentan, objeto central del presente trabajo.

2.2. Retículos: definición y propiedades fundamentales

El criptoanálisis de la criptografía basada en retículos exige, como punto de partida, una descripción precisa del objeto matemático sobre el que se construyen tanto los esquemas criptográficos como los algoritmos diseñados para atacarlos. Esta sección establece la definición formal de retículo, introduce las nociones de base, dominio fundamental, determinante y mínimos sucesivos, y presenta los resultados clásicos (en particular el teorema de Minkowski) que acotan la longitud del vector más corto. Estas magnitudes constituyen el vocabulario geométrico con el que, en secciones posteriores, se formularán los problemas computacionales difíciles (SVP, CVP, SIVP) y se analizará en qué medida los modelos de aprendizaje automático pueden aproximar sus soluciones.

2.2.1. Definición formal

Intuitivamente, un retículo puede describirse como una malla regular de puntos distribuidos de forma periódica en el espacio euclídeo $\mathbb{R}^d$ [25]. Formalmente, dado un conjunto de d vectores linealmente independientes $\mathbf{b}_1, \dots, \mathbf{b}_d \in \mathbb{R}^d$, el retículo generado por ellos se define como el conjunto de todas sus combinaciones lineales con coeficientes enteros [6], [26]:

$$\mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_d) = \left\{ \sum_{i=1}^d x_i \mathbf{b}_i : x_i \in \mathbb{Z} \text{ para } 1 \leq i \leq d \right\}. \quad (2.1)$$

Al conjunto de vectores $\{\mathbf{b}_1, \dots, \mathbf{b}_d\}$ se le denomina *base* del retículo. Resulta cómodo agrupar dichos vectores como columnas de una matriz $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_d] \in \mathbb{R}^{d \times d}$, de modo que el retículo admite la representación compacta $\mathcal{L}(\mathbf{B}) = \{\mathbf{B}\mathbf{x} : \mathbf{x} \in \mathbb{Z}^d\}$, donde $\mathbf{B}\mathbf{x}$ denota el producto matriz-vector habitual [6].

Desde una perspectiva algebraica más abstracta, un retículo d -dimensional se caracteriza como un subgrupo discreto de $(\mathbb{R}^d, +)$ de rango máximo [25]. La condición de discretitud (la existencia de una distancia mínima no nula entre puntos distintos) es precisamente lo que distingue a un retículo de un subespacio vectorial continuo y lo que confiere a los problemas geométricos asociados su dificultad computacional. El número de vectores de la base, d , se denomina *rango* o *dimensión* del retículo; cuando coincide con la dimensión del espacio ambiente se dice que el retículo es de *rango completo*, situación habitual en las construcciones criptográficas.

2.2.2. Multiplicidad de bases y matrices unimodulares

Una propiedad esencial de los retículos, y de hecho el fundamento geométrico de buena parte de las construcciones criptográficas, es que un mismo retículo admite infinitas bases distintas. Concretamente, dos matrices $\mathbf{B}$ y $\mathbf{B}'$ generan el mismo retículo si, y solo si, existe una matriz *unimodular* $\mathbf{U}$ (esto es, una matriz entera cuadrada con determinante ± 1) tal que $\mathbf{B}' = \mathbf{B}\mathbf{U}$ [6]. Puesto que la multiplicación por una matriz unimodular constituye un cambio de base que preserva el conjunto de puntos del retículo, cada retículo posee bases muy diversas en cuanto a su calidad geométrica.

Esta multiplicidad es el origen de la asimetría que explota la criptografía basada en retículos: mientras que algunas bases están formadas por vectores cortos y casi ortogonales (y, por tanto, hacen sencillos los problemas geométricos), otras están compuestas por vectores largos y muy oblicuos que ocultan la estructura del retículo [26]. Una base “buena” puede desempeñar el papel de clave privada, en tanto que una base “mala” del mismo retículo, computacionalmente difícil de mejorar, puede hacerse pública sin comprometer el secreto. El proceso de transformar una base de baja calidad en otra de mayor calidad se conoce como *reducción de bases* y se aborda en detalle en la Sección 2.4.

2.2.3. Dominio fundamental y determinante

A cada base de un retículo le corresponde una región del espacio denominada *paralelepípedo fundamental*, definido como

$$\mathcal{P}(\mathbf{B}) = \{\mathbf{B}\mathbf{x} : 0 \leq x_i < 1 \text{ para } 1 \leq i \leq d\}. \quad (2.2)$$

Los trasladados de esta región por todos los puntos del retículo recubren el espacio sin solaparse; es decir, los conjuntos $\mathbf{v} + \mathcal{P}(\mathbf{B})$, indexados por $\mathbf{v} \in \mathcal{L}(\mathbf{B})$, forman una partición de $\text{span}(\mathbf{B})$ [6]. Cualquier región con esta propiedad de teselación recibe el nombre de *región fundamental* del retículo; además del paralelepípedo, otros ejemplos relevantes son el paralelepípedo centrado y la *celda de Voronoi*, formada por los puntos más próximos al origen que a cualquier otro punto del retículo [6].

El *determinante* del retículo, denotado $\det(\mathcal{L})$, se define como el valor absoluto del determinante de cualquier matriz base, $\det(\mathcal{L}(\mathbf{B})) = |\det(\mathbf{B})|$, y coincide con el volumen d -dimensional del paralelepípedo fundamental. Una propiedad fundamental es que este valor es *invariante respecto de la base elegida*: dado que las distintas bases se relacionan mediante matrices unimodulares de determinante ± 1 , todas ellas conducen al mismo volumen [6]. Geométricamente, el determinante es inversamente proporcional a la densidad de los puntos del retículo en $\mathbb{R}^d$, de modo que retículos con determinante pequeño concentran más puntos por unidad de volumen.

El grado en que una base se aparta de la ortogonalidad se cuantifica mediante el *defecto de ortogonalidad*, definido como

$$\delta(\mathbf{B}) = \frac{\prod_i \|\mathbf{b}_i\|}{|\det(\mathbf{B})|}. \quad (2.3)$$

Por la desigualdad de Hadamard se cumple $\delta(\mathbf{B}) \geq 1$, con igualdad si, y solo si, los vectores de la base son mutuamente ortogonales [25]. Cuanto más se aproxima el defecto de ortogonalidad a la unidad, más sencillos resultan los problemas geométricos sobre el retículo, lo que justifica el interés en hallar bases lo más ortogonales posible.

2.2.4. Norma, vector más corto y mínimos sucesivos

La longitud de un vector del retículo se mide habitualmente mediante la norma euclídea $\|\mathbf{x}\| = \sqrt{\sum_i x_i^2}$, si bien los problemas pueden formularse respecto de cualquier norma [6]. La magnitud geométrica más relevante es la longitud del vector no nulo más corto del retículo, denotada $\lambda_1(\mathcal{L})$:

$$\lambda_1(\mathcal{L}) = \min_{\mathbf{v} \in \mathcal{L} \setminus \{0\}} \|\mathbf{v}\|. \quad (2.4)$$

Esta cantidad generaliza al concepto de *mínimos sucesivos* $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_d$, donde λ_k representa el menor radio r tal que la bola centrada en el origen de radio r contiene al menos k vectores del retículo linealmente independientes. Los mínimos sucesivos describen la “forma” geométrica del retículo y aparecen de manera natural en el análisis de los algoritmos de reducción de bases [6].

Aunque el cálculo exacto de $\lambda_1(\mathcal{L})$ es, en general, computacionalmente difícil, su valor puede acotarse. El resultado clásico en este sentido es el *teorema de Minkowski*, que establece que todo cuerpo convexo y centralmente simétrico de volumen estrictamente mayor que $2^n \det(\mathcal{L})$ contiene necesariamente un punto no nulo del retículo [26]. Aplicando este teorema a una bola euclídea se obtiene la cota superior

$$\lambda_1(\mathcal{L}) \leq \sqrt{d} \cdot \det(\mathcal{L})^{1/d}, \quad (2.5)$$

que relaciona la longitud del vector más corto con el determinante y la dimensión del retículo [26]. Conviene subrayar que el teorema de Minkowski es un resultado de *existencia*: garantiza que existe un vector corto, pero no proporciona ningún procedimiento eficiente para hallarlo, ni siquiera de forma aproximada para factores significativamente mejores que los exponenciales en la dimensión [6]. Es precisamente esta brecha entre la existencia garantizada de soluciones y la dificultad de calcularlas la que sustenta la seguridad de los criptosistemas basados en retículos y, en consecuencia, motiva la exploración de enfoques basados en aprendizaje automático para el criptoanálisis abordada en este trabajo.

2.2.5. Ortogonalización de Gram-Schmidt y perfil del retículo

El análisis de la calidad de una base y, en consecuencia, de la dificultad de los problemas geométricos sobre el retículo, se apoya de forma central en la *ortogonalización de Gram-Schmidt* (GSO) de la base. Dada una base $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_d]$, el procedimiento de Gram-Schmidt construye un conjunto de vectores ortogonales $\mathbf{b}_1^*, \dots, \mathbf{b}_d^*$ mediante la recurrencia

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^*, \quad \mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle}, \quad (2.6)$$

donde los escalares $\mu_{i,j}$ se denominan *coeficientes de Gram-Schmidt* [26], [27]. Conviene subrayar que los vectores $\mathbf{b}_i^*$ no pertenecen, en general, al retículo: constituyen una construcción auxiliar que captura la componente de cada vector de la base ortogonal al subespacio generado por los anteriores. Pese a ello, el producto de sus normas reproduce exactamente el determinante del retículo, $\det(\mathcal{L}) = \prod_{i=1}^d \|\mathbf{b}_i^*\|$, lo que pone de manifiesto su papel como descomposición geométrica de la base [26].

Los coeficientes $\mu_{i,j}$ ofrecen una medida directa del grado de solapamiento entre los vectores de la base. En particular, una base se dice *reducida en tamaño* (*size-reduced*) cuando $|\mu_{i,j}| \leq 1/2$ para todo $j < i$; esta condición, junto con la condición de Lovász, define las bases reducidas según el algoritmo LLL, tratado en la Sección 2.4 [27], [28]. Por tanto, tanto la magnitud de los coeficientes $\mu_{i,j}$ como la fracción de ellos que excede el umbral de $1/2$ informan sobre la cercanía de una base a su forma reducida, hecho que se explota más adelante en la construcción del modelo de aprendizaje.

Perfil de Gram-Schmidt

La secuencia de las normas logarítmicas de los vectores de Gram-Schmidt, $\ell_i = \log \|\mathbf{b}_i^*\|$ para $i = 1, \dots, d$, recibe el nombre de *perfil* del retículo (o *perfil GSO*) y constituye una de las herramientas analíticas más informativas para caracterizar el estado de una base [25], [29]. Su importancia radica en que resume en una única secuencia tanto la calidad global de la reducción como la distribución de la masa geométrica del retículo a lo largo de sus dimensiones.

En el caso de una base bien reducida, el perfil exhibe una tendencia aproximadamente lineal y decreciente. Esta regularidad se formaliza mediante la denominada *Geometric Series Assumption* (GSA), introducida por Schnorr [30], según la cual las normas de Gram-Schmidt de una base reducida decrecen geoméricamente, esto es, $\|\mathbf{b}_{i+1}^*\| \approx r \cdot \|\mathbf{b}_i^*\|$ para una razón constante $0 < r < 1$. Bajo esta hipótesis, el perfil logarítmico ℓ_i se comporta como una progresión aritmética, de modo que su *pendiente* caracteriza de forma compacta la calidad de la reducción: pendientes más planas corresponden a bases más reducidas, en las que la longitud de los vectores decrece más lentamente. Tanto la pendiente del ajuste lineal del perfil como la dispersión de los residuos respecto de dicho ajuste sintetizan, respectivamente, la tendencia y la regularidad del perfil, y constituyen descriptores geométricos de primer orden del estado de la base.

Factor de Hermite

Estrechamente ligado al perfil se encuentra el *factor de Hermite*, una medida adimensional de la calidad de una base reducida ampliamente utilizada en el análisis de seguridad de los esquemas basados en retículos [10], [29]. Para una base de un retículo d -dimensional, el *root Hermite factor* δ_0 se define a partir de la longitud del primer vector de la base reducida como

$$\delta_0 = \left(\frac{\|\mathbf{b}_1\|}{\det(\mathcal{L})^{1/d}} \right)^{1/d}. \quad (2.7)$$

El factor de Hermite normaliza la longitud del vector más corto hallado respecto de la cota dada por el determinante (2.5), lo que permite comparar de manera homogénea la calidad de la reducción entre retículos de distinta dimensión y determinante. Valores de δ_0 próximos a la unidad indican reducciones de alta calidad; en la práctica, los algoritmos de reducción alcanzan valores característicos de δ_0 que dependen del esfuerzo computacional invertido, lo que convierte a esta magnitud en un indicador estándar de la potencia de un ataque por reducción de bases [10]. Esta cantidad constituye, además, una de las variables que el modelo desarrollado en este trabajo pretende predecir a partir de las características de la base y la que se utiliza principalmente como indicadora de la calidad de la reducción a lo largo del trabajo.

2.2.6. Retículos q -arios

En el contexto criptográfico revisten especial importancia los retículos q -arios, definidos como aquellos retículos $\mathcal{L}$ que satisfacen $q\mathbb{Z}^n \subseteq \mathcal{L} \subseteq \mathbb{Z}^n$ para algún entero q [6]. En estos retículos la pertenencia de un vector queda determinada por sus coordenadas módulo q , lo que los pone en correspondencia biunívoca con los códigos lineales sobre $\mathbb{Z}_q^n$ y permite representarlos de forma compacta. La mayoría de las construcciones criptográficas modernas, entre ellas las basadas en el problema *Learning With Errors* (LWE), tratado más adelante, emplean retículos q -arios como soporte de su problema difícil en promedio [6], [31]. Dada una matriz $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, se definen los dos retículos q -arios m -dimensionales

$$\Lambda_q(\mathbf{A}) = \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{y} = \mathbf{A}^\top \mathbf{s} \text{ mód } q \text{ para algún } \mathbf{s} \in \mathbb{Z}^n\}, \quad (2.8)$$

$$\Lambda_q^\perp(\mathbf{A}) = \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{A}\mathbf{y} = \mathbf{0} \text{ mód } q\}, \quad (2.9)$$

que son duales entre sí salvo normalización y que constituyen el marco natural sobre el que se formulan los esquemas estudiados en este trabajo [6].

Dentro de la familia de los retículos q -arios merecen una mención particular los *retículos NTRU*, asociados al criptosistema del mismo nombre [32]. Estos retículos constituyen un caso particular de retículo q -ario dotado de una estructura algebraica adicional: sus bases presentan una forma de bloques construida a partir de polinomios en un anillo cociente, lo que introduce una periodicidad cíclica de la que carecen los retículos q -arios generados a partir de una matriz $\mathbf{A}$ uniforme. Esta estructura, heredada de la aritmética de ideales del anillo subyacente, los sitúa en la base de esquemas post-cuánticos estandarizados y de variantes estructuradas del problema LWE, como *Ring-LWE* y *Module-LWE* [31]. Aunque el presente trabajo no incorpora retículos NTRU en su conjunto experimental (centrado en los tipos de retículo descritos en la Sección 3.2.1) su relevancia criptográfica los convierte en una extensión natural del estudio, según se discute en la Sección 6.2.

2.3. Problemas computacionales sobre retículos

La seguridad de la criptografía basada en retículos no descansa sobre la estructura del retículo en sí, sino sobre la dificultad computacional de ciertos problemas geométricos formulados a partir de una base arbitraria. La premisa fundamental es que, dada una base «mala» (es decir, formada por vectores largos y marcadamente no ortogonales), recuperar información geométrica precisa sobre el retículo resulta intratable a medida que crece la dimensión [6], [8]. Esta sección formaliza los tres problemas que articulan dicha dificultad: el problema del vector más corto (SVP), el problema del vector más cercano (CVP) y el problema *Learning With Errors* (LWE), que constituye el puente entre la teoría de retículos y los esquemas post-cuánticos estandarizados. Todos ellos comparten una propiedad común que motiva el presente trabajo: su resolución se ve facilitada por bases «buenas», de modo que la calidad de la reducción de bases, objeto del análisis empírico de esta memoria, determina directamente la dificultad efectiva del ataque.

2.3.1. El Problema del Vector más Corto (SVP)

El problema del vector más corto es el problema computacional más estudiado sobre retículos y el de mayor relevancia histórica [29]. En su formulación exacta, dado un retículo $\mathcal{L}$ representado mediante una base $\mathbf{B}$, el SVP consiste en hallar un vector no nulo del retículo de norma mínima, esto es, un vector $\mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\}$ tal que $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$, donde $\lambda_1(\mathcal{L})$ denota el primer mínimo sucesivo del retículo introducido en la Sección 2.2.4 [6]. Aunque la formulación admite cualquier norma, la euclídea $\|\mathbf{x}\| = (\sum_i x_i^2)^{1/2}$ es la considerada de forma estándar en el ámbito criptográfico [6].

En la práctica criptográfica rara vez se exige una solución exacta. Por ello resulta más pertinente la *variante de aproximación*, denotada SVP_γ , en la que se admite como solución cualquier vector no nulo del retículo cuya norma no exceda en más de un factor $\gamma \geq 1$ a la del vector más corto, es decir, $0 < \|\mathbf{v}\| \leq \gamma \cdot \lambda_1(\mathcal{L})$ [6]. El parámetro γ , denominado factor de aproximación, gobierna la frontera entre lo tratable y lo intratable: para factores exponenciales en la dimensión el problema es resoluble en tiempo polinómico mediante reducción de bases, mientras que para factores pequeños permanece fuera del alcance de los algoritmos conocidos. Junto a la variante de búsqueda se define también su versión de decisión, GapSVP_γ , que pregunta únicamente si $\lambda_1(\mathcal{L})$ es menor que un umbral dado o mayor que dicho umbral multiplicado por γ , y que constituye la formulación habitual en los análisis de complejidad y en las demostraciones de reducción [31].

La relevancia criptográfica del SVP se sustenta en sólidos resultados de dificultad. La conjetura de su carácter NP-difícil, formulada por van Emde Boas en 1981, permaneció abierta hasta que Ajtai demostró en 1998 que el SVP exacto es NP-difícil bajo reducciones aleatorizadas; posteriormente Micciancio probó que ni siquiera es posible aproximarlos en tiempo polinómico dentro de cualquier factor inferior a $\sqrt{2}$, salvo que se colapsen ciertas clases de complejidad [29]. Este resultado se ha reforzado para factores de aproximación cuasipolinómicos [33]. No obstante, conviene matizar la relación entre esta dureza en el peor caso y la seguridad criptográfica: Goldreich y Goldwasser establecieron que, bajo hipótesis estándar, el SVP no puede ser NP-difícil de aproximar para factores polinómicos pequeños del orden de $O(\sqrt{n/\log n})$, lo que delimita la región de factores de aproximación en la que cabe fundamentar de forma plausible la construcción de primitivas seguras [29]. Es precisamente en ese régimen intermedio donde operan tanto los algoritmos de reducción clásicos como los enfoques basados en aprendizaje automático examinados en este trabajo.

2.3.2. El Problema del Vector más Cercano (CVP)

El problema del vector más cercano constituye la contrapartida inhomogénea del SVP. Dado un retículo $\mathcal{L}$ con base $\mathbf{B}$ y un vector objetivo $\mathbf{t} \in \mathbb{R}^d$, que en general no pertenece al retículo, el CVP consiste en hallar el punto del retículo $\mathbf{v} \in \mathcal{L}$ que minimiza la distancia $\|\mathbf{v} - \mathbf{t}\|$ [6]. Análogamente al caso anterior, su variante de aproximación CVP_γ admite como

solución cualquier vector del retículo situado a una distancia del objetivo no superior a γ veces la distancia mínima [6]. Intuitivamente, mientras que el SVP busca el punto del retículo más próximo al origen, el CVP generaliza dicha búsqueda a un objetivo arbitrario del espacio, lo que lo convierte en el modelo natural de los escenarios de descifrado en los que un texto cifrado se interpreta como una versión perturbada de un punto del retículo.

Ambos problemas están estrechamente relacionados. El SVP se reduce al CVP, incluso en sus respectivas variantes de aproximación, por lo que el CVP es al menos tan difícil como el SVP [25]. Esta relación también se manifiesta en el plano algorítmico: la dificultad de uno y otro disminuye conforme la base de partida es más ortogonal, hasta el punto de que, ante una base ortogonal, el SVP se resuelve inspeccionando las columnas de la base y el CVP se reduce a redondear las coordenadas del objetivo [25]. La técnica de inmersión de Kannan (*Kannan's embedding*) explota esta conexión en sentido inverso, al transformar una instancia de CVP en una instancia de SVP sobre un retículo de dimensión incrementada en una unidad, construido añadiendo el vector objetivo como una columna adicional de la base [6]. Esta reducción reviste interés práctico porque permite atacar el CVP (y, por extensión, los problemas de descifrado que de él derivan) reutilizando la maquinaria de reducción de bases y de resolución del SVP. En el plano algorítmico, el procedimiento clásico para el CVP aproximado combina una reducción previa de la base mediante LLL con el algoritmo del plano más cercano (*Nearest Plane*) de Babai, alcanzando un factor de aproximación exponencial en tiempo polinómico [27]; este esquema ilustra, una vez más, que la calidad de la base reducida condiciona de forma decisiva la precisión de la solución obtenida.

2.3.3. Learning With Errors (LWE)

El problema *Learning With Errors*, introducido por Regev en 2005, constituye el fundamento de la mayor parte de la criptografía basada en retículos desplegada en la actualidad [7], [8]. A diferencia del SVP y el CVP, formulados de forma puramente geométrica, el LWE se enuncia en términos algebraicos como un problema de recuperación de un secreto a partir de ecuaciones lineales ruidosas. Sea q un entero, n la dimensión del secreto y $\mathbf{s} \in \mathbb{Z}_q^n$ un vector secreto. Dada una matriz uniformemente aleatoria $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ y un vector de error $\mathbf{e} \in \mathbb{Z}_q^m$ muestreado de una distribución de probabilidad concentrada en torno a cero (típicamente una gaussiana discreta de desviación pequeña), el problema LWE consiste en recuperar el secreto $\mathbf{s}$ a partir del par

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \text{ mód } q), \quad (2.10)$$

es decir, a partir de un sistema de ecuaciones lineales módulo q perturbado por el término de error [7]. Es precisamente la presencia de este ruido la que confiere al problema su dificultad: sin él, el secreto se recuperaría trivialmente mediante eliminación gaussiana, pero su introducción transforma la tarea en un problema computacionalmente duro. Junto a esta versión de búsqueda se considera una versión de decisión, consistente en distinguir

muestras de la forma (2.10) de pares uniformemente aleatorios, ambas equivalentes en los regímenes de parámetros de interés [31].

La solidez del LWE como base criptográfica deriva de su conexión formal con los problemas geométricos sobre retículos. El paradigma de fundamentar la seguridad sobre reducciones del peor caso al caso medio fue introducido por Ajtai, quien construyó la primera familia de retículos aleatorios para la que hallar un vector corto en el caso medio resulta tan difícil como resolver problemas reticulares clásicos en el peor caso [34]. Sobre esta base, Regev demostró posteriormente una reducción cuántica que vincula la dificultad del LWE en el caso promedio con la dificultad en el peor caso de problemas reticulares clásicos como GapSVP_γ y el problema de los vectores independientes más cortos (SIVP) [7], [31]. Esta garantía es de gran valor: asegura que romper un esquema basado en LWE para una elección aleatoria de parámetros implicaría resolver eficientemente instancias de SVP o SIVP en el peor caso, una hazaña que se considera inalcanzable con la tecnología algorítmica actual, tanto clásica como cuántica. La conexión geométrica subyacente se hace explícita al observar que recuperar el secreto a partir de (2.10) equivale a un problema de tipo CVP sobre el retículo q -ario $\Lambda_q(\mathbf{A})$ definido en la Sección 2.2.6, en el que $\mathbf{b}$ desempeña el papel de vector objetivo y $\mathbf{e}$ el de desplazamiento respecto al punto reticular más cercano [31].

Por motivos de eficiencia, las construcciones prácticas emplean variantes estructuradas del problema. *Ring-LWE* sustituye los vectores y matrices por elementos de un anillo de polinomios cociente, lo que reduce drásticamente el tamaño de las claves y acelera la aritmética a costa de introducir estructura algebraica adicional [8], [31]; *Module-LWE* ofrece un compromiso intermedio entre la versión genérica y la versión sobre anillos, combinando la flexibilidad de la primera con buena parte de la eficiencia de la segunda. La relevancia de estas variantes trasciende el ámbito teórico: Module-LWE constituye el fundamento de los dos estándares post-cuánticos basados en retículos publicados por el NIST en 2024, el mecanismo de encapsulado de claves ML-KEM [2] y el esquema de firma digital ML-DSA [3]. En consecuencia, la evaluación rigurosa de la dificultad real del LWE (y, de manera indirecta, la del SVP que lo sustenta) no es un ejercicio puramente académico, sino un requisito para calibrar la seguridad de las infraestructuras criptográficas que sustituirán a los esquemas clásicos vulnerables al cómputo cuántico, objetivo último que motiva el análisis desarrollado en este trabajo.

2.4. Algoritmos de reducción de bases

Los problemas geométricos descritos en las secciones anteriores, en particular el SVP y sus variantes aproximadas, carecen de algoritmos eficientes que los resuelvan de forma exacta en dimensión arbitraria. En la práctica, el ataque a los esquemas basados en retículos se reconduce a la búsqueda de una *base de buena calidad*: un conjunto de vectores generadores que sean cortos y aproximadamente ortogonales, a partir de los cuales se obtienen soluciones aproximadas a los problemas reticulares. Este es el cometido de los

algoritmos de reducción de bases, que transforman una base arbitraria del retículo en otra equivalente (esto es, que genera el mismo retículo, lo que equivale a multiplicar por una matriz unimodular) pero geoméricamente más favorable [6], [29]. La familia de estos algoritmos abarca un espectro continuo entre coste y calidad, cuyos dos extremos canónicos son el algoritmo LLL, de coste polinómico y calidad modesta, y los algoritmos de cribado (*sieving*), de coste exponencial y calidad óptima, con el algoritmo BKZ ocupando una posición intermedia y ajustable. La caracterización precisa de la calidad que cada uno alcanza constituye el fundamento de la estimación de seguridad de la criptografía post-cuántica y, por tanto, el punto de partida del problema abordado en este trabajo.

2.4.1. El algoritmo LLL

El algoritmo de Lenstra, Lenstra y Lovász, publicado en 1982 e introducido originalmente como herramienta para la factorización de polinomios con coeficientes racionales [28], constituye el primer procedimiento capaz de reducir una base de un retículo en *tiempo polinómico* respecto de la dimensión. Su relevancia histórica y práctica es difícil de exagerar: pese a producir una aproximación cuya garantía se degrada exponencialmente con la dimensión, su eficiencia y robustez lo han convertido en el bloque elemental sobre el que se construyen prácticamente todos los algoritmos de reducción posteriores [29].

El algoritmo recibe como entrada una base $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_d]$ y un parámetro $\delta \in (1/4, 1]$ que regula el compromiso entre la calidad de la base resultante y el coste computacional: valores próximos a la unidad (típicamente $\delta = 0,99$) producen reducciones de mayor calidad a costa de un mayor número de iteraciones [26], [28]. Una base se considera *LLL-reducida* con parámetro δ cuando satisface simultáneamente dos condiciones, formuladas sobre la ortogonalización de Gram-Schmidt introducida en la Sección 2.2.5. La primera es la condición de *reducción en tamaño* (*size reduction*),

$$|\mu_{i,j}| \leq \frac{1}{2}, \quad \text{para todo } 1 \leq j < i \leq d, \quad (2.11)$$

que limita la proyección de cada vector sobre los precedentes y acota así su solapamiento mutuo. La segunda es la *condición de Lovász*,

$$\|\mathbf{b}_k^*\|^2 \geq \left(\delta - \mu_{k,k-1}^2\right) \|\mathbf{b}_{k-1}^*\|^2, \quad (2.12)$$

que impone una decadencia controlada del perfil de Gram-Schmidt entre vectores consecutivos, impidiendo que las normas ortogonalizadas decrezcan con demasiada rapidez [27], [28].

Operativamente, el algoritmo recorre la base de forma iterativa alternando dos acciones. Mientras un vector incumple la reducción en tamaño, se le resta el múltiplo entero adecuado de los vectores precedentes para forzar (2.11); y cuando dos vectores consecutivos violan la condición de Lovász (2.12), se intercambian sus posiciones, $\mathbf{b}_k \leftrightarrow \mathbf{b}_{k-1}$. Estas dos operaciones se repiten hasta que ninguna condición resulta violada, momento en el que la base resultante posee vectores más cortos y considerablemente más ortogonales

que los de partida [26]. Puesto que ambas operaciones se corresponden con multiplicaciones por matrices unimodulares, el retículo generado permanece invariante a lo largo de todo el proceso.

En cuanto a las garantías, una base LLL-reducida con parámetro δ verifica que la longitud de su primer vector está acotada por

$$\|\mathbf{b}_1\| \leq \left(\frac{2}{\sqrt{4\delta - 1}} \right)^{n-1} \lambda_1(\mathcal{L}), \quad (2.13)$$

de modo que el factor de aproximación al vector más corto es, en el peor caso, exponencial en la dimensión n [28], [29]. El coste computacional, en cambio, es polinómico: con aritmética entera, la versión clásica requiere del orden de $O(n^4 \log B)$ operaciones de bit, donde B acota la norma de los vectores de entrada [29]. Conviene subrayar, no obstante, que esta cota del peor caso resulta notablemente pesimista frente al comportamiento empírico: en la práctica, LLL alcanza de forma sistemática factores de Hermite (2.7) muy inferiores a los que la ecuación (2.13) permitiría, con valores característicos del *root Hermite factor* en torno a $\delta_0 \approx 1,02$ con independencia de la instancia concreta [10]. Esta brecha entre la garantía teórica del peor caso y el comportamiento medio observado es, precisamente, una de las motivaciones centrales de este trabajo: si el resultado de la reducción es en la práctica mucho más regular y predecible de lo que sugieren las cotas, cabe esperar que pueda anticiparse mediante modelos aprendidos a partir de las características de cada base, sin necesidad de ejecutar el algoritmo.

2.4.2. El algoritmo BKZ

El algoritmo *Block Korkine-Zolotareff* (BKZ) generaliza el esquema de LLL con el objetivo de obtener reducciones de mayor calidad a cambio de un coste superior, ofreciendo además un mecanismo para graduar ese compromiso de forma explícita. La idea esencial consiste en sustituir el tratamiento de pares de vectores consecutivos, propio de LLL, por el de *bloques* de vectores de mayor tamaño [29]. El algoritmo se gobierna mediante un parámetro de *bloque* β , con $2 \leq \beta \leq d$, que fija el número de vectores considerados conjuntamente en cada paso: sobre cada ventana de β vectores consecutivos se resuelve (o aproxima) el SVP del subretículo de proyección correspondiente, y el vector corto hallado se incorpora a la base mediante operaciones unimodulares, recorriéndose la base de forma repetida hasta la convergencia.

El parámetro β interpola de manera continua entre los dos regímenes extremos: para $\beta = 2$, BKZ se reduce esencialmente al algoritmo LLL, mientras que para $\beta = d$ proporciona una base óptimamente reducida en el sentido de Hermite-Korkine-Zolotareff, equivalente a resolver el SVP sobre el retículo completo [29]. El coste crece de forma acusada con β , pues cada paso requiere resolver un SVP en dimensión β (tarea de complejidad exponencial en dicho parámetro) al tiempo que la calidad de la base, medida por el factor de Hermite, mejora monótonamente. Esta dependencia explícita y ajustable entre β , coste y calidad es lo que convierte a BKZ en la herramienta de referencia para el

criptoanálisis de retículos: variando el parámetro de bloque se recorre todo el espectro de esfuerzo computacional disponible para un adversario.

La variante *BKZ 2.0*, propuesta por Chen y Nguyen, incorpora una serie de mejoras (entre ellas la poda extrema (*extreme pruning*) en la enumeración del SVP, el preprocesado de los bloques y una estrategia de terminación temprana) que reducen sustancialmente el coste práctico del algoritmo y permiten alcanzar parámetros de bloque elevados en dimensiones de interés criptográfico [35]. Esta optimización resultó decisiva porque desplazó las estimaciones de seguridad de los esquemas basados en retículos hacia valores más realistas, y sus modelos de coste constituyen hoy el núcleo de las herramientas de estimación concreta de seguridad, como el ampliamente utilizado *lattice-estimator* [35], [36]. En consecuencia, predecir con precisión la calidad de reducción alcanzable para un parámetro de bloque dado equivale, en términos prácticos, a estimar el margen de seguridad de los esquemas post-cuánticos estandarizados. El presente trabajo se centra en la predicción de la reducción LLL, el caso $\beta = 2$, como paso fundacional, dejando la extensión natural al parámetro de bloque β de BKZ como una de las líneas de trabajo futuro.

2.4.3. Lattice sieving

En el extremo opuesto del espectro coste-calidad se sitúan los algoritmos de *cribado* o *sieving*, que constituyen la familia de métodos asintóticamente más eficiente conocida para resolver el SVP de forma exacta. A diferencia de los enfoques basados en enumeración, que exploran de manera estructurada el espacio de combinaciones de la base, el cribado parte de una gran colección de vectores del retículo y los combina iterativamente, restando unos de otros, para generar vectores progresivamente más cortos, hasta aislar el vector más corto del retículo [6], [37]. El coste de estos métodos es exponencial tanto en tiempo como en memoria, lo que los hace inviables más allá de dimensiones moderadas; sin embargo, su escalado asintótico es el más favorable conocido y, por ello, son los que determinan las estimaciones de seguridad para las dimensiones criptográficas reales.

Los avances en cribado han ido ligados a las técnicas de búsqueda del vecino más próximo (*nearest neighbor searching*): aplicando estructuras de búsqueda por proximidad para identificar de manera eficiente pares de vectores cuya diferencia es corta, Becker *et al.* establecieron las complejidades de referencia actuales, situando el coste en $2^{0,292n+o(n)}$ en el modelo clásico y en $2^{0,265n+o(n)}$ en el modelo cuántico [37]. Estos exponentes son los que, en última instancia, fijan los niveles de seguridad declarados para esquemas como ML-KEM y ML-DSA, pues representan el coste del mejor ataque conocido sobre el problema subyacente.

Resulta de particular interés para la perspectiva de este trabajo la lectura del cribado como un *algoritmo evolutivo* propuesta por Laarhoven [21]. Bajo esta analogía, la colección de vectores cortos mantenida por el algoritmo desempeña el papel de una *población*, la retención de los vectores más cortos equivale a un proceso de *selección*, y la combinación de pares de vectores para generar otros más cortos se interpreta como un

operador de *recombinación* análogo al cruce y la mutación de la computación evolutiva. Esta reinterpretación, además de ofrecer un marco conceptual unificador, abre la puerta a la incorporación de técnicas metaheurísticas y de aprendizaje en el diseño de algoritmos de reducción, y conecta de forma natural con la motivación general de esta memoria: la aplicación de métodos de aprendizaje automático e inteligencia computacional al criptoanálisis de la criptografía basada en retículos.

2.5. Aplicación de Machine Learning a problemas sobre retículos

2.5.1. Ataques ML al problema LWE

La línea de trabajo más visible en la aplicación de técnicas de aprendizaje automático al criptoanálisis de retículos es la familia de ataques iniciada por SALSA [38] y continuada en sucesivas iteraciones hasta el trabajo de Stevens *et al.* [39]. Todos ellos abordan directamente el problema LWE descrito en la Sección 2.3.3 y comparten una misma arquitectura conceptual, articulada en dos fases. En la primera, de *preprocesado*, se aplican algoritmos clásicos de reducción de bases (LLL y BKZ, junto con variantes recientes) a las muestras LWE con el fin de acortar las coordenadas de la matriz $\mathbf{A}$ y atenuar el ruido, transformando el problema en una instancia más cercana a una regresión lineal. En la segunda fase se entrena un modelo de tipo *transformer* que actúa como distinguidor criptográfico: el modelo aprende a predecir el término $\mathbf{b}$ a partir de $\mathbf{a}$ y, al hacerlo, codifica implícitamente el secreto $\mathbf{s}$, que se recupera posteriormente comparando las predicciones del modelo ante entradas que difieren en una única posición [39].

El interés de estos trabajos para la presente memoria es doble. Por un lado, ilustran que el aprendizaje automático puede integrarse en un ataque criptoanalítico completo y no meramente como heurística auxiliar. Por otro, y de manera más significativa, evidencian que el rendimiento del ataque queda determinado en gran medida por la fase de preprocesado mediante reducción de retículos: las aportaciones de SALSA FRESCA [39] (el empleo de algoritmos de reducción más rápidos, una arquitectura *encoder-only* con codificación angular y el uso de preentrenamiento) persiguen precisamente reducir el coste de esta etapa, que constituye el principal cuello de botella del método. Gracias a ellas, dicho trabajo logra las primeras recuperaciones de secretos en dimensión $n = 1024$ (la menor empleada en aplicaciones prácticas de cifrado homomórfico) y rebaja sustancialmente el tiempo de ataque en dimensión $n = 512$ [39].

No obstante, una valoración crítica de estos resultados exige acotar su alcance. Los ataques de esta familia operan exclusivamente sobre secretos binarios o ternarios y *rales* (de bajo peso de Hamming), un régimen de parámetros propio de ciertos esquemas de cifrado homomórfico pero ajeno a las distribuciones de error empleadas por los estándares ML-KEM y ML-DSA. Los propios autores reconocen que su técnica no compromete, por el momento, los esquemas post-cuánticos estandarizados [39]. Más allá de su impacto directo, lo relevante para este trabajo es la constatación de que la eficacia del aprendizaje

automático en este contexto descansa sobre la calidad de la reducción de bases obtenida en el preprocesado, magnitud cuyo análisis y predicción constituye el objeto central de la presente memoria.

2.5.2. Reducción de retículos mediante deep learning

Una segunda vía de investigación, complementaria de la anterior, no emplea el aprendizaje automático para atacar un esquema concreto, sino para resolver directamente el problema de reducción de bases descrito en la Sección 2.4. El trabajo de referencia en esta línea es el de Marchetti *et al.* [25], que plantea la siguiente cuestión: ¿es posible parametrizar el espacio de algoritmos de reducción mediante redes neuronales y descubrir un algoritmo sin recurrir a datos supervisados? Su propuesta consiste en una red φ que, dada una base $\mathbf{B}$, produce una matriz unimodular de cambio de base $\mathbf{Q}$ tal que la base reducida $\mathbf{B}' = \mathbf{BQ}$ minimiza el defecto de ortogonalidad introducido en (2.3). El entrenamiento es *autosupervisado*: al no disponerse en general de la matriz de cambio de base óptima como verdad de referencia, el modelo se optimiza penalizando directamente la falta de ortogonalidad de la base de salida [25].

El principal mérito técnico de esta aproximación reside en la incorporación de las simetrías del problema al diseño de la red, en línea con el *aprendizaje profundo geométrico*. Dado que el defecto de ortogonalidad es invariante frente a isometrías y a cambios de escala del espacio ambiente, el modelo recibe como entrada la matriz de Gram normalizada en lugar de la base, lo que garantiza dicha invariancia por construcción; asimismo, se impone equivarianza respecto al grupo hiperoctaédrico, un subgrupo finito del grupo unimodular formado por las permutaciones y cambios de signo de los vectores de la base [25]. La salida se factoriza como un producto de transformaciones elementales (*movimientos de Gauss*), una construcción que asegura que la matriz resultante sea simultáneamente entera e invertible.

En cuanto a su evaluación, el método alcanza un rendimiento y una complejidad comparables a los del algoritmo LLL en conjuntos de prueba de dimensión reducida, y resulta especialmente ventajoso en su variante convolucional, concebida para reducir de forma conjunta retículos espacialmente correlacionados (como los que aparecen en la detección MIMO de las comunicaciones inalámbricas) amortizando el coste sobre el conjunto [25]. Ahora bien, una lectura rigurosa de sus resultados obliga a subrayar sus límites: el modelo no supera de forma consistente a LLL y, de hecho, fracasa en igualarlo a medida que crecen la dimensión y la correlación. Los propios autores lo presentan como una prueba de viabilidad de un enfoque basado en aprendizaje profundo para un problema NP-duro, antes que como un sustituto práctico de los algoritmos clásicos [25]. Esta distinción es pertinente para el presente trabajo: mientras Marchetti *et al.* emplean el aprendizaje para *ejecutar* la reducción, la presente memoria lo orienta a *predecir* su resultado, un objetivo distinto que se precisa en la Sección 2.5.4.

2.5.3. Lattice sieving como algoritmo evolutivo

Una tercera perspectiva, de carácter más conceptual, es la propuesta por Laarhoven [21], ya anticipada al cierre de la Sección 2.4. Su aportación consiste en mostrar que el cribado de retículos (*lattice sieving*), el método más rápido conocido para el SVP en dimensiones elevadas, admite una reformulación natural como *algoritmo evolutivo*. Bajo esta analogía, la lista de vectores reticulares que el algoritmo mantiene desempeña el papel de una *población*; la norma euclídea de cada vector actúa como función de *aptitud* (cuanto menor es la norma, más apto es el individuo); la selección de pares de vectores próximos equivale a la *selección de progenitores*; y la combinación de dos vectores $\mathbf{v}, \mathbf{w}$ para formar la diferencia $\mathbf{u} = \mathbf{v} - \mathbf{w}$ se interpreta como un operador de *recombinación*, garantizado en el retículo por la propiedad elemental $\mathbf{v}, \mathbf{w} \in \mathcal{L} \Rightarrow \mathbf{v} - \mathbf{w} \in \mathcal{L}$ [21].

Laarhoven identifica además dos diferencias entre el cribado clásico y el paradigma evolutivo estándar: la ausencia de *mutaciones* (la descendencia se genera de forma determinista) y el reemplazo local e inmediato de los progenitores en lugar de una selección de supervivientes a escala global. Sobre esta base, el autor reinterpreta diversas mejoras del cribado como instancias de técnicas evolutivas ya conocidas (la búsqueda por proximidad como un mecanismo de *especiación*, el cribado progresivo como una estrategia de preferencias progresivas o la paralelización como un *modelo de islas*) y explora experimentalmente la incorporación de operadores hasta entonces ausentes, como la selección global y las mutaciones, observando que pueden mejorar la calidad de la población o ayudar a evitar óptimos locales [21].

La relevancia de este trabajo para la presente memoria no radica en un resultado criptoanalítico concreto, sino en el marco conceptual unificador que ofrece. Al situar los algoritmos de reducción dentro del dominio de la inteligencia computacional, legitima y motiva la exploración de métodos de aprendizaje y metaheurísticas en el criptoanálisis de retículos, que es la premisa de fondo de este trabajo. Conviene, no obstante, matizar que se trata de una analogía descriptiva: su valor reside en la transferencia de ideas entre disciplinas más que en una mejora cuantificada de las complejidades asintóticas establecidas en [37].

2.5.4. Predicción de la calidad de reducción LLL: el hueco en la literatura

Las tres líneas anteriores comparten un rasgo común: emplean el aprendizaje automático para *resolver* un problema sobre retículos (recuperar un secreto LWE, ejecutar una reducción o cribar vectores cortos). Sin embargo, ninguna de ellas aborda la cuestión que articula el presente trabajo: *predecir* la calidad que alcanzará la reducción de una base antes de ejecutarla. Este problema es central para la estimación de la seguridad de los esquemas post-cuánticos, pues, como se ha argumentado a lo largo de este capítulo, la dificultad efectiva de los ataques depende directamente de la calidad de reducción alcanzable, medida por magnitudes como el defecto de ortogonalidad o el factor de Hermite

de (2.7).

Los procedimientos actualmente empleados para estimar esta calidad presentan limitaciones bien diferenciadas. Las cotas teóricas del peor caso, derivadas de resultados como el teorema de Minkowski o las garantías de aproximación del algoritmo LLL (Sección 2.4.1), resultan en la práctica excesivamente conservadoras, ya que el comportamiento de los algoritmos de reducción sobre instancias aleatorias es notablemente mejor que el que garantizan dichas cotas. La heurística Gaussiana ofrece una estimación promedio más ajustada, pero, por su propia naturaleza, ignora la estructura específica de cada instancia concreta. Por último, los simuladores del perfil de Gram-Schmidt (Sección 2.2.5) o la ejecución directa de LLL y BKZ proporcionan estimaciones fiables a costa de un esfuerzo computacional que puede resultar prohibitivo cuando debe repetirse sobre un gran número de instancias.

En este contexto se sitúa la originalidad de la presente memoria. La propuesta consiste en entrenar modelos supervisados que, a partir de características algebraicas y geométricas extraídas de la base original, predigan métricas de calidad de la reducción (en particular, el defecto de ortogonalidad de la base reducida, la longitud del primer vector reducido $\log\|\mathbf{b}_1\|$ y el factor de Hermite) sin necesidad de ejecutar el algoritmo. Hasta donde alcanza la revisión bibliográfica realizada, ninguno de los trabajos descritos en las secciones precedentes aborda este problema específico, lo que delimita con precisión la aportación de este trabajo y justifica su pertinencia para la calibración de la seguridad de las infraestructuras criptográficas post-cuánticas, según se detalla en los capítulos siguientes.

2.6. Herramientas y tecnologías relacionadas

El desarrollo de este trabajo se apoya en un conjunto reducido de herramientas software y de estándares normativos que conviene presentar de forma diferenciada según el papel que desempeñan en la metodología. Cabe distinguir tres planos: las bibliotecas que generan los datos experimentales mediante la ejecución efectiva de la reducción de retículos, el entorno de aprendizaje automático con el que se construyen los modelos predictivos y, por último, las herramientas y estándares que sitúan la pertinencia práctica del trabajo en el contexto de la criptografía post-cuántica desplegada. Un rasgo común a todo el instrumental empleado es su condición de software libre y de acceso abierto, circunstancia que no solo elimina restricciones de propiedad intelectual sobre las herramientas utilizadas, sino que garantiza la reproducibilidad de los experimentos descritos en los capítulos posteriores.

2.6.1. Bibliotecas de reducción de retículos: `fp111` y `fp111`

La biblioteca `fp111` constituye la implementación de referencia, hoy ampliamente aceptada por la comunidad, de los principales algoritmos de reducción de bases de retículos.

los. Escrita en C++ con aritmética de coma flotante de precisión arbitraria, proporciona implementaciones eficientes del algoritmo LLL (Sección 2.4.1), de BKZ y de su variante BKZ 2.0 (Sección 2.4.2), así como de las rutinas de enumeración del vector más corto y del cálculo de la ortogonalización de Gram-Schmidt [9], [35]. Por su parte, `fpyl1l` ofrece una interfaz en Python sobre dicha biblioteca que expone tanto los algoritmos de reducción como las estructuras internas de la base (en particular, los coeficientes $\mu_{i,j}$ y las normas de Gram-Schmidt $\|\mathbf{b}_i^*\|$ que definen el perfil GSO (Sección 2.2.5)) de manera directamente manipulable desde el ecosistema científico de Python [9].

En el marco de este trabajo, `fpyl1l` cumple una doble función insustituible. En primer lugar, es la herramienta que produce las *etiquetas* del conjunto de entrenamiento: al ejecutar LLL con un valor controlado del parámetro δ sobre cada base generada, permite medir con exactitud las métricas de calidad que el modelo aprende a predecir, el defecto de ortogonalidad, la longitud del primer vector reducido y el factor de Hermite de (2.7). En segundo lugar, posibilita la extracción del perfil de Gram-Schmidt de la base original, fuente de buena parte de las características geométricas que alimentan el predictor. La biblioteca actúa, por tanto, como el oráculo experimental frente al cual se contrasta la capacidad predictiva de los modelos.

2.6.2. Aprendizaje automático: XGBoost y scikit-learn

El núcleo del modelado predictivo se construye sobre dos familias de modelos de conjunto basados en árboles de decisión (*Random Forest* y *gradient boosting*, ambas formuladas como problemas de regresión), implementadas mediante la biblioteca XGBoost, distribuida bajo licencia Apache 2.0 y ampliamente consolidada para el aprendizaje sobre datos tabulares [11]. En concreto, se emplean los estimadores `XGBRFRegressor` (bosque aleatorio) y `XGBRegressor` (*gradient boosting*), cuya ejecución con aceleración GPU (`device='cuda'`, `tree_method='hist'`) permite entrenar sobre las 224 000 instancias del conjunto de datos en tiempos razonables, lo que resultaba inviable con implementaciones equivalentes no aceleradas. La biblioteca `scikit-learn`, distribuida bajo licencia BSD y consolidada como estándar de facto para el modelado sobre datos tabulares [17], complementa este núcleo aportando la infraestructura de soporte del *pipeline*: el escalado de características (`StandardScaler`) y las métricas de evaluación (`r2_score`, `mean_absolute_error`). La partición en conjuntos de entrenamiento y prueba, en cambio, no se delega en `train_test_split`: al requerir una división *por base* (Sección 3.3), se implementa en un módulo propio (`split_utils.py`) que agrupa las cuatro filas de cada base antes de asignarlas a uno u otro conjunto.

La elección de estas familias de modelos no es arbitraria, sino que responde a la naturaleza del problema planteado. Las características extraídas de cada base son heterogéneas en escala y origen (combinan magnitudes algebraicas, como el logaritmo del determinante, con descriptores geométricos derivados del perfil GSO), y los árboles de decisión son robustos frente a estas diferencias de escala, al tiempo que capturan de forma natu-

ral las interacciones no lineales entre variables. Resulta especialmente relevante para los objetivos de la memoria que ambos modelos proporcionen una medida intrínseca de la importancia de cada característica, pues dicha información es la que permite interpretar los resultados en términos criptográficos y contrastar si el modelo reproduce, de manera implícita, el comportamiento descrito por la heurística Gaussiana (Sección 2.5.4).

2.6.3. Redes neuronales: PyTorch

Como ampliación de carácter exploratorio respecto al núcleo del trabajo, basado en los modelos de conjunto descritos en la subsección anterior, se contempla el empleo de una red neuronal de tipo perceptrón multicapa (*Multilayer Perceptron*, MLP) sobre el mismo problema de predicción. Para ello se recurre a PyTorch, marco de trabajo de aprendizaje profundo de código abierto que proporciona diferenciación automática, gestión transparente del cálculo sobre GPU y un esquema de grafos de cómputo dinámicos que facilita la definición y el entrenamiento de arquitecturas neuronales [18].

La motivación de esta extensión es doble. Por un lado, permite contrastar el rendimiento de los modelos de conjunto basados en árboles (que operan de forma natural sobre características tabulares) frente al de un modelo de representación distribuida, evaluando si este último es capaz de capturar estructura adicional en el perfil de Gram-Schmidt que los primeros no aprovechan. Por otro, conecta el enfoque de la memoria con la línea de investigación más reciente, que explora la aplicación de técnicas de aprendizaje profundo directamente a la reducción de retículos [25]. No obstante, conviene precisar el alcance de esta aportación: el MLP se plantea como un complemento destinado a delimitar las ventajas y limitaciones de cada paradigma sobre el problema concreto abordado, y no como el método principal, cuyo análisis interpretativo descansa sobre los modelos de conjunto basados en árboles (XGBoost) por su mayor transparencia.

2.6.4. Estimación concreta de seguridad: el *lattice-estimator*

El *lattice-estimator* (heredero de la metodología de estimación concreta de la dureza del LWE introducida por Albrecht, Player y Scott [40]) constituye la herramienta de referencia para estimar la dificultad concreta de los problemas sobre retículos en los que se sustenta la criptografía post-cuántica [36]. A diferencia de las cotas asintóticas, esta herramienta agrega los modelos de coste de los ataques conocidos (singularmente las estrategias primal y dual, que reducen el problema LWE a una reducción de bases gobernada por los modelos de coste de BKZ 2.0) para devolver una estimación del nivel de seguridad efectivo, expresado en bits, de un conjunto de parámetros dado [35], [36].

La relación de esta herramienta con el presente trabajo es directa y motiva su inclusión. Como se argumentó en la Sección 2.4.2, la calibración de la seguridad de los esquemas reticulares depende de poder anticipar la calidad de reducción alcanzable. El *lattice-estimator* apoya esa estimación en modelos de coste de carácter promedio que ig-

noran la estructura de cada instancia concreta; los modelos predictivos propuestos en esta memoria pueden interpretarse, por consiguiente, como un refinamiento complementario y sensible a la instancia de la misma tarea que esta herramienta aborda a escala global.

2.6.5. Esquemas post-cuánticos estandarizados: ML-KEM y ML-DSA

El destinatario último de cualquier estimación de seguridad sobre retículos son los esquemas criptográficos efectivamente desplegados. En agosto de 2024, el NIST culminó su proceso de estandarización con la publicación de los primeros estándares post-cuánticos basados en retículos [5]: el mecanismo de encapsulado de claves ML-KEM, recogido en el estándar FIPS 203 y derivado del esquema CRYSTALS-Kyber [2], y el esquema de firma digital ML-DSA, recogido en FIPS 204 y derivado de CRYSTALS-Dilithium [3]. Ambos fundamentan su seguridad en variantes estructuradas de los problemas reticulares, Module-LWE y Module-SIS, cuya dificultad se reduce, en última instancia, al coste de la reducción de bases.

Estos estándares cierran el círculo argumental del capítulo: las magnitudes que los modelos de este trabajo aprenden a predecir son precisamente las que determinan el margen de seguridad de las primitivas hoy en proceso de adopción a escala global. Predecir con precisión la calidad de reducción no constituye, por tanto, un ejercicio meramente teórico, sino una herramienta con aplicación directa a la verificación y la calibración de seguridad de las primitivas post-cuánticas llamadas a sustituir a los esquemas clásicos vulnerables al cómputo cuántico.

3. METODOLOGÍA Y DISEÑO DEL SISTEMA

3.1. Visión general del sistema

El sistema desarrollado en este trabajo es un *pipeline* de aprendizaje automático supervisado cuyo propósito es predecir, sin ejecutar el algoritmo LLL, la calidad de la reducción que este producirá sobre una base de retículo dada. La figura 3.1 muestra su arquitectura de alto nivel, articulada en seis etapas que se describen de forma resumida a continuación y se detallan en las secciones siguientes.

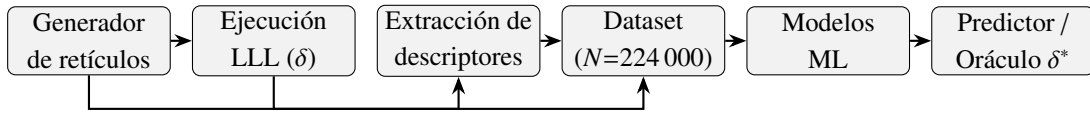


Fig. 3.1. Pipeline completo del sistema. Las flechas indican el flujo de datos; la base original alimenta simultáneamente la ejecución de LLL (para obtener las etiquetas o targets) y la extracción de descriptores (para construir las entradas al modelo).

Etapas 1. Generación de retículos. Se generan bases de retículo de forma aleatoria (con semillas fijadas para garantizar la reproducibilidad) cubriendo cuatro tipos estructurales (uniforme, gaussiano, *sparse* y *q*-ario) y nueve dimensiones $d \in \{10, 20, 30, 50, 80, 100, 120, 150, 200\}$, mediante los generadores disponibles en `lattice_utils.py` (véase la nota de la Sección 3.2.1). La diversidad de tipos persigue un doble objetivo: maximizar la representatividad del conjunto de entrenamiento respecto a las estructuras que aparecen en la práctica criptográfica (en particular mediante una versión simplificada de los retículos *q*-arios, relacionados con el problema LWE de la Sección 2.3.3) y evaluar en qué medida la capacidad predictiva del modelo se traslada entre estructuras distintas. La justificación detallada de los tipos, las dimensiones y el número de instancias generadas se ofrece en la Sección 3.2.

Etapas 2. Ejecución de LLL. Cada base generada se reduce mediante el algoritmo LLL de la Sección 2.4.1 con cuatro valores del parámetro $\delta \in \{0.55, 0.75, 0.90, 0.99\}$, usando la biblioteca `fpylll` (Sección 2.6.1) como oráculo experimental. Una decisión de diseño fundamental es que *la misma base* se reduce con los cuatro valores de δ , de modo que las filas del conjunto de datos correspondientes a distintos δ comparten la misma base original (esta decisión puede inducir *data leakage* si no se separan los conjuntos de *test* y *train* correctamente). Esta estructura *multi-delta* permite comparar directamente el efecto del parámetro sobre una instancia idéntica y es lo que hace posible la selección adaptativa descrita en la Sección 3.4.

Etapa 3. Extracción de descriptores. A partir de la base original, antes de aplicar ninguna reducción, se calcula un vector de 25 características (*features*) numéricas algebraicas y geométricas, agrupadas en cinco bloques: (i) magnitudes globales (dimensión d , $\log |\det(\mathbf{B})|$ y la desviación típica de las entradas de $\mathbf{B}$); (ii) estadísticas de las normas originales $\|\mathbf{b}_i\|$; (iii) estadísticas de las normas del perfil GSO $\|\mathbf{b}_i^*\|$ y de su ajuste log-lineal; (iv) los funcionales globales del perfil GSO (masa, entropía y producto de adyacencia); (v) magnitudes de ortogonalidad y condicionamiento de la base (el logaritmo del defecto de ortogonalidad y el logaritmo del número de condición $\log \kappa(\mathbf{B})$) y estadísticas de los coeficientes $\mu_{i,j}$ y de la matriz de Gram. A estas 25 características se añade el parámetro δ como entrada adicional. La relación completa y su definición formal se presentan en la Sección 3.2.3. El principio subyacente es que la geometría del retículo antes de la reducción (capturada mediante el perfil GSO y las magnitudes derivadas) contiene información suficiente para anticipar la calidad del resultado; este es el supuesto que el trabajo se propone validar empíricamente. Además, el uso de características derivadas en lugar de la base original como vector de entrada reduce la dimensionalidad del espacio de entrada, lo que facilita el entrenamiento de los modelos y su interpretación criptográfica.

Etapa 4. Construcción del conjunto de datos. La combinación de 56 000 bases generadas y los cuatro valores de δ produce un conjunto de datos (*dataset*) de 224 000 filas, almacenado en `lattice_dataset_v2.csv`. Cada fila corresponde a un par (base, δ) y reúne 30 columnas: las 25 características numéricas descritas, la columna categórica que identifica el tipo de retículo, el parámetro δ y las tres variables objetivo (el logaritmo del defecto de ortogonalidad de la base reducida $\log \delta(\mathbf{B}_{\text{red}})$, el logaritmo de la norma del primer vector reducido $\log \|\mathbf{b}_1^{\text{red}}\|$ y el *root Hermite factor* δ_0 (2.7)). En la fase de entrenamiento, el tipo de retículo se codifica mediante cuatro columnas *one-hot*, de modo que el vector de entrada del modelo global consta de las 25 características numéricas (incluida d), las cuatro columnas *one-hot* y el parámetro δ ; en los modelos por dimensión (Etapa 5), d se elimina por ser constante. La definición formal y la motivación criptográfica de las variables objetivo se exponen en la Sección 3.2.4.

Etapa 5. Entrenamiento de los modelos. Sobre el conjunto anterior se entrenan dos familias de modelos de regresión basados en conjuntos de árboles: *Random Forest* (RF) y *Gradient Boosting* (GB), ambos implementados mediante **XGBoost** con aceleración por GPU. El trabajo considera dos modalidades de entrenamiento, cuya comparación constituye uno de los ejes de la evaluación. En la modalidad *global* se entrena un único modelo con todas las dimensiones conjuntamente, empleando d y el tipo de retículo como características adicionales; en la modalidad *por dimensión* se entrena un modelo independiente para cada valor de d , eliminando d como variable y permitiendo al modelo especializarse en la geometría propia de cada régimen dimensional. Como extensión exploratoria se entrena además un perceptrón multicapa (*Multilayer Perceptron*, MLP) en PyTorch, cuya comparación con **XGBoost** permite determinar si la mayor capacidad de representación

de las redes neuronales aporta ventaja sobre datos tabulares de esta naturaleza. En todos los casos la evaluación se realiza sobre datos no vistos durante el entrenamiento; dado que una misma base origina cuatro filas, una por cada valor de δ , la partición entre entrenamiento y prueba se efectúa *por base* y no por fila, asignando en bloque las cuatro filas de cada base, lo que evita la fuga de información (*data leakage*) entre ambos conjuntos. Dicha partición es *canónica* y estratificada por dimensión y tipo de retículo: se define una sola vez y la comparten todos los experimentos, según se detalla en la Sección 3.3.

Etapas 6. Predictor y selección adaptativa de δ . El producto final del *pipeline* es un predictor que, dada una nueva base $\mathbf{B}$ cuyas características se han extraído según la Etapa 3, devuelve estimaciones de las tres métricas de calidad para cada uno de los cuatro valores de δ , sin necesidad de ejecutar LLL en ningún momento. Sobre este predictor se articula la contribución aplicada del trabajo: un *oráculo adaptativo* que, para cada base, selecciona el valor de δ más conveniente. Su diseño se aborda de forma iterativa en la Sección 3.4; aquí se introduce únicamente la *formulación inicial*, sobre la que se construye el resto.

Esta primera formulación, gobernada por una única métrica, el *root Hermite factor* de la base reducida, selecciona el menor valor de δ cuya calidad *predicha* no empeora más de un umbral relativo τ respecto de la que se obtendría con $\delta = 0,99$:

$$\delta^*(\mathbf{B}) = \min\{\delta \in \{0,55, 0,75, 0,90, 0,99\} : \hat{\delta}_0(\mathbf{B}, \delta) \leq (1 + \tau) \hat{\delta}_0(\mathbf{B}, 0,99)\}, \quad (3.1)$$

donde $\hat{\delta}_0(\mathbf{B}, \delta)$ denota la predicción del *root Hermite factor* δ_0 (2.7) de la base reducida y $\tau \geq 0$ es el umbral de tolerancia. La regla descansa en el supuesto de que la calidad de la reducción mejora de forma monótona con δ (de modo que $\delta = 0,99$ proporciona la mejor de las cuatro reducciones y actúa como referencia), supuesto habitualmente avalado por el comportamiento empírico de LLL. Bajo este diseño el coste de la selección se reduce a cuatro evaluaciones del modelo, despreciable frente a una única ejecución de LLL.

Esta formulación inicial, sin embargo, no es la finalmente adoptada. La evaluación del Capítulo 4 (Sección 4.4.1) muestra que la regla (3.1) fracasa por motivos medibles (las cuatro predicciones de una misma base difieren entre sí por menos que el error del propio modelo, y el supuesto de monotonía solo se cumple en parte de las instancias), lo que obliga a reformular el mecanismo de selección. El oráculo definitivo, construido como un clasificador supervisado que predice δ^* directamente, se presenta en la Sección 3.4.2 y se valida en la Sección 4.4.

3.1.1. Justificación de las decisiones de diseño

Dos decisiones de diseño definen la identidad metodológica del sistema y merecen una justificación explícita antes de abordar el detalle de cada etapa. La primera es la elección de predecir la calidad a partir de la *base original*, antes de ninguna reducción. Esta decisión distingue al sistema de los simuladores del perfil de Gram-Schmidt (que modelan la

evolución interna del algoritmo) y lo sitúa como un estimador de caja negra que responde a la pregunta práctica relevante: dado el retículo tal y como se tiene, ¿cuánta reducción cabe esperar? La hipótesis subyacente es que el perfil GSO de la base original, aunque lejos del perfil regularizado descrito por la GSA (Sección 2.2.5), ya contiene información geométrica suficiente para anticipar la calidad del resultado. El Capítulo 4 contrasta esta hipótesis de forma sistemática y permite delimitar los regímenes dimensionales en los que se sostiene.

La segunda decisión es el diseño *multi-delta*: generar varias filas del conjunto de datos a partir de la misma base, variando únicamente δ . Esta elección tiene dos consecuencias directas. En primer lugar, enriquece el conjunto de datos sin incrementar el coste de generación de retículos, que es la operación más costosa del *pipeline*. En segundo lugar, y de mayor relevancia, permite al modelo aprender la calidad de la reducción como función de δ para cada instancia concreta, que es precisamente la información que requiere el oráculo de la Sección 3.4 para adoptar decisiones bien calibradas. El análisis de importancia de características de la Sección 4.3 permite verificar en qué medida el modelo captura el efecto de δ sobre la calidad de la reducción.

3.2. Generación del dataset

La construcción del dataset constituye la base experimental del proyecto, dado que la capacidad predictiva de los modelos depende directamente de la representatividad de los datos de entrenamiento.

El pipeline de generación implementado en `gen_features_and_target.py` sigue un esquema de tres etapas secuenciales: (i) generación de una base de retículo $\mathbf{B} \in \mathbb{Z}^{d \times d}$ mediante un generador estocástico parametrizado por tipo y dimensión; (ii) extracción del vector de características sobre la base sin reducir, a través del objeto `GSO.Mat` de `fpynll` [9]; y (iii) aplicación del algoritmo LLL con cada uno de los valores de δ del conjunto discreto $\{0,55; 0,75; 0,90; 0,99\}$ y extracción de las variables objetivo sobre la base reducida resultante.

Este diseño, denominado en adelante *esquema multi-delta por matriz*, garantiza que las cuatro filas generadas a partir de una misma matriz base son directamente comparables entre sí: el vector de características de entrada es idéntico y únicamente varía el parámetro δ . Esta propiedad es metodológicamente necesaria para que el modelo aprenda la influencia de δ como variable de control independiente, sin confundirla con la variabilidad geométrica intrínseca de la base.

3.2.1. Tipos de retículo

Con el objetivo de que el dataset cubra estructuras geométricas variadas y criptográficamente relevantes, se emplean cuatro familias de generadores disponibles en el módulo

`lattice_utils.py`.¹

En el retículo uniforme, la base $\mathbf{B}$ se genera con entradas enteras independientes distribuidas uniformemente en $\{-10, \dots, 10\}$. Este tipo constituye el *baseline* más sencillo de la literatura de reducción de bases y sirve como punto de referencia para los demás generadores.

En el retículo gaussiano, las entradas de $\mathbf{B}$ se muestrean como $\lfloor X_{ij} \rfloor$ con $X_{ij} \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 5$. La distribución gaussiana genera matrices con mayor concentración de valores próximos a cero y colas pesadas, produciendo perfiles GSO cualitativamente distintos a los uniformes.

En el retículo disperso (*sparse*), se parte de una matriz uniforme a la que se aplica una máscara de densidad $\rho = 0,5$: cada entrada se anula con probabilidad $1 - \rho$, con excepción de la diagonal principal, que se fuerza a ser entera positiva para garantizar la no singularidad. Este tipo modela situaciones donde la base presenta estructura de soporte escaso.

En el retículo q -ario, se construye un retículo *half-rank* q -ary con $q = 101^2$ siguiendo la estructura clásica

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_k & \mathbf{0} \\ \mathbf{A} & \mathbf{I}_{d-k} \end{pmatrix}, \quad k = \lfloor d/2 \rfloor, \quad (3.2)$$

donde $\mathbf{A} \in \mathbb{Z}^{(d-k) \times k}$ es aleatoria uniforme módulo q y $\det(\mathbf{B}) = q^k$. Conviene precisar el estatus de esta familia para no atribuirle una relación con los estándares post-cuánticos que la construcción no posee. Se trata de una versión *simplificada* y *no estructurada* de retículo q -ario: la matriz $\mathbf{A}$ se muestrea de forma uniforme e independiente (sin la estructura algebraica de anillo cociente propia de las variantes *Ring-LWE* y *Module-LWE*) y el módulo se fija en un valor reducido, $q = 101$, elegido por tratabilidad computacional y no por correspondencia con ningún conjunto de parámetros real. Como tal, esta familia se corresponde con los retículos q -arios aleatorios que sustentan el problema *Learning With Errors* (LWE) en su formulación *plana* [6], [7], y debe interpretarse como un modelo geométrico de esa clase de instancias. En particular, **no** reproduce los esquemas ML-KEM ni ML-DSA: estos descansan sobre variantes *estructuradas* del problema (Module-LWE y Module-SIS) definidas sobre el anillo $\mathbb{Z}_q[X]/(X^{256} + 1)$, con módulos primos muy superiores al aquí empleado ($q = 3329$ en ML-KEM y $q = 8380417$ en ML-DSA) y dimensiones de retículo de varios centenares a más de mil coeficientes [2], [3] (Sección 3.2.2), propiedades ausentes en el generador descrito. Su inclusión en el conjunto experimental persigue, por tanto, evaluar si el oráculo generaliza a una estructura geométrica radical-

¹Los generadores de retículos contenidos en `lattice_utils.py` fueron facilitados por el tutor del trabajo, Javier Blanco Romero, y se utilizan aquí como infraestructura de generación de datos. El resto del *pipeline* (extracción de características, entrenamiento y evaluación de los modelos, y selección adaptativa de δ) es de elaboración propia.

²El módulo $q = 101$ se elige con fines ilustrativos y de tratabilidad computacional; queda varios órdenes de magnitud por debajo de los módulos primos de los esquemas estandarizados ($q = 3329$ en ML-KEM y $q = 8380417$ en ML-DSA [2], [3]), lo que confirma el carácter simplificado de esta familia.

mente distinta de las densas aleatorias, y no establecer una equivalencia con las instancias criptográficas desplegadas en la práctica.

En los tres generadores aleatorios densos (uniforme, gaussiano y *sparse*) se verifica que la matriz generada tenga rango completo antes de proceder con la extracción de características; si esta condición no se satisface tras un máximo de 500 intentos, el ejemplo se descarta. El generador q -ario, en cambio, produce matrices no singulares por construcción, ya que $\det(\mathbf{B}) = q^k \neq 0$, por lo que no requiere reintentos ni verificación.

3.2.2. Dimensiones y volumen del dataset

Las dimensiones d consideradas son $\{10, 20, 30, 50, 80, 100, 120, 150, 200\}$. Esta elección responde a tres criterios complementarios.

En primer lugar, el **gradiente de complejidad estadística**. Las dimensiones bajas ($d \leq 30$) permiten verificar el comportamiento del modelo en regímenes donde el Root Hermite Factor (RHF) presenta alta varianza relativa. Las dimensiones intermedias ($d \in \{50, 80\}$) cubren el rango habitual de los experimentos de seguridad en la literatura.

En segundo lugar, la **relevancia criptográfica**. Conviene precisar aquí la estructura de los parámetros de ML-KEM para no confundir el grado del anillo con la dimensión del módulo. En todos sus conjuntos de parámetros el grado del anillo polinómico es fijo, $n = 256$ (el esquema opera sobre el anillo $\mathbb{Z}_q[X]/(X^{256} + 1)$ con $q = 3329$), y lo que distingue a cada variante es el rango del módulo k : $k = 2$ en ML-KEM-512 y $k = 3$ en ML-KEM-768 [2]. La dimensión del retículo asociado al problema Module-LWE (la magnitud verdaderamente relevante para los ataques de reducción de bases) es el producto $k \cdot n$, es decir, 512 y 768 coeficientes respectivamente; son estas cifras, y no el grado del anillo, las que dan nombre a cada variante. Dichas dimensiones quedan por encima del rango explorado en este trabajo, pero el régimen $d \geq 100$ cubre la zona en la que los ataques de reducción de bases resultan ya pertinentes para la estimación de seguridad.

En tercer lugar, el **coste computacional**. Para $d \geq 100$ la aplicación de LLL es notablemente más costosa (su coste polinómico $O(n^4 \log B)$ crece de forma acusada con la dimensión (Sección 2.4.1)), por lo que el número de muestras se reduce a 1 000 matrices por par (dimensión, tipo) frente a las 2 000 empleadas para $d \leq 80$.

Con 4 tipos de retículo, 4 valores de δ y los volúmenes de muestreo indicados, el dataset final contiene **224 000 filas** y 30 columnas (25 características numéricas, la columna categórica `lattice_type`, el parámetro δ y 3 variables objetivo). La distribución de muestras se resume en la Tabla 3.1.

La generación se ejecutó en paralelo utilizando `multiprocessing.Pool` sobre 12 hilos lógicos (6 núcleos físicos), con un tiempo total de ejecución de aproximadamente 36,9 minutos. Cada *worker* recibe exclusivamente datos serializables (dimensión y tipo de retículo), de modo que los objetos `fpylll`, no serializables por *pickle*, se instancian y destruyen dentro del propio proceso hijo, evitando condiciones de carrera.

Dimensión d	Matrices / tipo	Tipos	Valores δ	Filas totales
10, 20, 30, 50, 80	2 000	4	4	$5 \times 2000 \times 4 \times 4 = 160\,000$
100, 120, 150, 200	1 000	4	4	$4 \times 1000 \times 4 \times 4 = 64\,000$
Total				224 000

TABLA 3.1. DISTRIBUCIÓN DE MUESTRAS POR DIMENSIÓN EN LATTICE_DATASET_V2.CSV. CADA FILA DEL DATASET CORRESPONDE A UN PAR (BASE, δ); CADA MATRIZ BASE ORIGINA 4 FILAS (UNA POR VALOR DE δ).

3.2.3. Extracción de características

Para cada base $\mathbf{B}$ sin reducir se extrae un vector de 25 características escalares accesibles a través del objeto `GSO.Mat` de `fpylll` sin necesidad de ejecutar LLL. Las características se agrupan en cinco bloques funcionales.

El primer bloque recoge las características globales de la base: la dimensión d , la desviación típica de las entradas de $\mathbf{B}$, y el logaritmo del determinante $\log |\det(\mathbf{B})|$, obtenido como la mitad del valor retornado por `M.get_log_det()`, dado que `fpylll` devuelve el logaritmo del cuadrado del determinante.

El segundo bloque reúne estadísticas de las normas de los vectores base: media, desviación típica, máximo, mínimo y cociente máximo/mínimo de las normas euclídeas $\|\mathbf{b}_i\|$, $i = 1, \dots, d$. Estas estadísticas capturan la dispersión geométrica de los vectores antes de la reducción.

El tercer bloque corresponde al perfil GSO y a su ajuste log-lineal. Las normas $\|\mathbf{b}_i^*\|$ del proceso de Gram-Schmidt ortogonal (GSO), obtenidas como $\sqrt{\mathbf{M}.get_r(i, i)}$, son el indicador más directo de la calidad estructural de la base [10]. De estas normas se calculan media, desviación típica, máximo, mínimo y cociente máximo/mínimo. Adicionalmente, sobre el perfil logarítmico $\{p_i = \log \|\mathbf{b}_i^*\|\}_{i=1}^d$ se ajusta por mínimos cuadrados una recta $p_i \approx s \cdot i + c$, de la que se retienen como características la pendiente s y la desviación típica de los residuos del ajuste. La pendiente s es especialmente informativa: una pendiente negativa pronunciada caracteriza un perfil ya relativamente reducido, mientras que una pendiente próxima a cero corresponde a bases cuasi-ortogonales o a retículos q -arios [10].

El cuarto bloque añade funcionales globales del perfil GSO. Para capturar la forma global del perfil de manera compacta, se añaden tres funcionales invariantes a la escala:

- *Masa del perfil*: $\sum_{i=1}^d |p_i|$, que mide la energía total del perfil logarítmico.
- *Entropía del perfil*: entropía de Shannon sobre la distribución de pesos $|p_i| / \sum_j |p_j|$, que cuantifica la uniformidad del perfil.
- *Producto de adyacencia*: $\sum_{i=1}^{d-1} p_i \cdot p_{i+1}$, sensible a caídas abruptas características de los retículos q -arios.

El quinto bloque agrupa las características de ortogonalidad y los coeficientes μ . El logaritmo del defecto de ortogonalidad

$$\log \delta(\mathbf{B}) = \sum_{i=1}^d \log \|\mathbf{b}_i\| - \log |\det \mathbf{B}|, \quad (3.3)$$

la media y el máximo de las entradas fuera de la diagonal de la matriz de Gram $\mathbf{B}\mathbf{B}^\top$, el logaritmo del número de condición $\log \kappa(\mathbf{B})$, y estadísticas de los coeficientes de Gram-Schmidt $\mu_{i,j}$ (media, máximo y fracción con $|\mu_{i,j}| > 0,5$, umbral relevante pues la condición de *size-reduction* del algoritmo LLL exige $|\mu_{i,j}| \leq 1/2$ [28]).

El número de condición $\kappa(\mathbf{B}) = \|\mathbf{B}\| \|\mathbf{B}^{-1}\|$ cuantifica el grado de degeneración numérica de la base: valores altos indican una base mal condicionada, vectores casi linealmente dependientes o de escalas muy dispares, una situación geoméricamente próxima a la pérdida de ortogonalidad que LLL debe corregir.

Por último, el valor del parámetro de reducción LLL δ se incluye explícitamente como característica de entrada, lo que permite al modelo aprender la relación funcional entre la calidad de reducción y el parámetro de control sin necesidad de entrenar un modelo independiente por valor de δ .

3.2.4. Variables objetivo

Para cada par $(\mathbf{B}, \delta)$ se aplica `LLL.reduction(B_copy, delta= $\delta$ )` sobre una copia de la matriz original y se extraen tres variables objetivo sobre la base reducida $\mathbf{B}_{\text{red}}$, descritas en la Tabla 3.2.

Variable	Definición formal	Rango típico
<code>target_log_orth_defect</code>	$\log \delta(\mathbf{B}_{\text{red}}) = \sum_i \log \ \mathbf{b}_i^{\text{red}}\ - \log \det \mathbf{B} $	$[0, +\infty)$, decreciente con δ
<code>target_log_norm_b1</code>	$\log \ \mathbf{b}_1^{\text{red}}\ $	depende de d y δ
<code>target_rhf</code>	$\delta_0 = \left(\ \mathbf{b}_1^{\text{red}}\ / \det(\Lambda) ^{1/d} \right)^{1/d}$	$[1,00, 1,08]$ para $d \geq 30$

TABLA 3.2. VARIABLES OBJETIVO EXTRAÍDAS SOBRE LA BASE REDUCIDA $\mathbf{B}_{\text{RED}}$.

El defecto de ortogonalidad (ecuación 3.3) cuantifica la distancia de la base a la ortogonalidad completa: el valor cero corresponde a una base perfectamente ortogonal. La log-norma del primer vector es el indicador más directo de la calidad de reducción desde la perspectiva del problema del vector más corto (SVP). El Root Hermite Factor δ_0 es la métrica estándar en la literatura de criptografía de retículos para evaluar la calidad de un algoritmo de reducción de forma independiente de la dimensión y del volumen del retículo [6], [10]: valores de δ_0 próximos a 1 indican bases casi reducidas de forma óptima,

mientras que valores superiores a 1,05 caracterizan bases poco reducidas. Precisamente por reunir estas propiedades (ser una medida de calidad normalizada, comparable entre dimensiones y de interpretación directa en términos criptográficos), el *root Hermite factor* se adopta como la métrica rectora sobre la que opera la selección adaptativa del parámetro δ descrita en la Sección 3.4: el oráculo predice esta cantidad para cada valor candidato de δ y la emplea como criterio único para decidir el nivel de reducción más conveniente en cada instancia.

3.3. Modelos de Machine Learning

En este trabajo se emplean tres familias de modelos supervisados para la tarea de regresión descrita en la Sección 3.2.4: bosques aleatorios (*Random Forest*), implementados mediante `XGBRFRegressor` de la librería `XGBoost` [11]; modelos de *gradient boosting* sobre árboles de decisión, implementados mediante `XGBRegressor` de la misma librería; y redes neuronales de tipo perceptrón multicapa (*Multilayer Perceptron*, MLP), implementadas en `PyTorch`. Ambas familias basadas en árboles se benefician de aceleración GPU (`device='cuda'`, `tree_method='hist'`), lo que permite su entrenamiento sobre el dataset completo de 224 000 instancias en tiempos razonables. Las tres familias se evalúan bajo dos estrategias de entrenamiento: un modelo *global*, entrenado sobre el conjunto de datos completo, y modelos *por dimensión*, entrenados de forma independiente para cada valor de $d \in \{10, 20, 30, 50, 80, 100, 120, 150, 200\}$.

La modalidad *por dimensión* se fundamenta en el carácter dimensional de las métricas de calidad de la reducción: en el análisis empírico de Gama y Nguyen se establece que el factor de Hermite alcanzado por los algoritmos de reducción converge a una constante característica que depende de la dimensión y presenta una dispersión reducida dentro de cada régimen dimensional fijo [10]. Así, al eliminar d como variable de entrada, cada modelo se especializa en la geometría propia de su dimensión, lo que cabe esperar que beneficie especialmente a la predicción del *root Hermite factor*, cuya baja varianza intradimensional dificulta su estimación por un modelo global.

En todos los casos las características de entrada se estandarizan mediante `StandardScaler` (media cero, varianza unitaria), ajustado únicamente sobre el conjunto de entrenamiento para evitar la fuga de información hacia el conjunto de prueba. Aunque los modelos basados en árboles son invariantes a transformaciones monótonas de las características y no requieren esta normalización, se aplica de forma uniforme en todo el *pipeline* por ser imprescindible para la convergencia del MLP. La evaluación se realiza sobre una *partición canónica por base*, común a todos los experimentos de este trabajo. Como cada base genera cuatro filas, una por cada valor de δ , la división en entrenamiento (80 %) y prueba (20 %) se efectúa agrupando esas cuatro filas en bloque, de manera que ninguna base aparezca simultáneamente en ambos conjuntos; en caso contrario, las filas de una misma base repartidas entre entrenamiento y prueba introducirían una fuga de información (*data leakage*) que inflaría de forma artificial las métricas. La asignación se realiza

de forma estratificada por dimensión y tipo de retículo (de modo que cada estrato (d , tipo) quede representado con misma proporción en el conjunto de entrenamiento y prueba) con semilla fija, y el conjunto de test resultante (11 200 bases) se materializa una sola vez y se reutiliza de forma idéntica en los modelos globales, los modelos por dimensión, el MLP y el oráculo. El coeficiente de determinación R^2 se adopta como métrica principal aunque en ciertas partes como la comparación del modelo general con los modelos por dimensión sea necesario recurrir al MAE.

3.3.1. Modelos globales

El modelo global recibe como entrada las 25 características numéricas (incluida la dimensión d), las cuatro columnas *one-hot* que codifican el tipo de retículo y el parámetro δ , lo que conforma un vector de entrada de 30 componentes. Entrenado sobre las 224 000 filas del dataset v2, actúa como *baseline* universal y permite la inferencia sobre dimensiones no vistas durante el entrenamiento.

Configuración de XGBoost

Se emplean dos variantes de XGBoost con aceleración GPU:

- XGBRFRegressor (*Random Forest*) con 200 estimadores, `max_depth = 15`, `subsample = 0.8`, `colsample_bynode = 0.8`.
- XGBRegressor (*Gradient Boosting*) con 200 rondas de boosting, `max_depth = 5`, `learning_rate = 0.1`, `subsample = 0.8`, `colsample_bytree = 0.8`.

Los hiperparámetros de profundidad y número de estimadores (200 estimadores con `max_depth = 15` para el modelo de bosque aleatorio y 200 rondas de *boosting* con profundidad `max_depth = 5` para el modelo de *gradient boosting*) constituyen una configuración estándar de capacidad media-alta, adecuada para datasets tabulares de esta escala sin incurrir en sobreajuste severo [11]. Los parámetros de submuestreo (`subsample`, `colsample_bynode/bytree`) introducen el componente estocástico característico de estos métodos de conjunto. El recurso a XGBoost con aceleración GPU responde a una necesidad práctica: el entrenamiento sobre 224 000 instancias resultaba inviable en CPU con implementaciones no aceleradas para el volumen de datos generado.

Configuración del MLP global

La arquitectura `ConditionedMLP` consiste en tres bloques `LINEAR-BATCHNORM-GELU-DROPOUT` de [256, 128, 64] neuronas, con una capa de salida lineal de dimensión 1. El protocolo de entrenamiento incluye: función de pérdida *Huber*, optimizador AdamW ($\text{lr} = 10^{-3}$, $\text{weight decay} = 10^{-4}$), planificador `CosineAnnealingLR`, 300 épocas y tamaño de

batch de 512. Se entrena un modelo independiente por cada *target* (*log_orth_defect*, *log_norm_b1*, *target_rhf*).

3.3.2. Modelos por dimensión

Como alternativa al modelo global, se entrena un modelo independiente para cada valor de $d \in \{10, 20, 30, 50, 80, 100, 120, 150, 200\}$, de modo que cada modelo aprenda exclusivamente la distribución de retículos de una dimensión fija y no necesite generalizar entre regímenes geoméricamente distintos. En esta configuración la *feature* d se suprime del vector de entrada, por ser constante dentro de cada subconjunto, lo que reduce el vector de entrada de 30 a 29 componentes (24 características numéricas, las cuatro columnas *one-hot* del tipo de retículo y el parámetro δ). La especialización se fundamenta en el carácter dimensional del factor de Hermite señalado al comienzo de la sección y, en particular, cabe esperar que beneficie a la predicción del *root Hermite factor*, cuya baja varianza intradimensional dificulta su estimación por un modelo global.

Bajo esta misma estrategia, y con el fin de contrastar ambos paradigmas también en la modalidad por dimensión, se entrena adicionalmente un MLP por cada par (d , *target*) (esto es, $9 \times 3 = 27$ modelos) siguiendo el mismo protocolo de entrenamiento descrito para el MLP global en la Sección 3.3.1. El efecto de la especialización por dimensión sobre la capacidad predictiva de todos estos modelos se evalúa en el Capítulo 4.

3.4. Selección adaptativa del parámetro δ

El predictor descrito en la Sección 3.3 no constituye un fin en sí mismo, sino el componente sobre el que se articula la contribución aplicada de este trabajo: un *oráculo adaptativo* capaz de seleccionar, para cada base concreta, el valor del parámetro δ de LLL más conveniente sin necesidad de ejecutar el algoritmo. Esta sección presenta el diseño del oráculo siguiendo el proceso iterativo con que fue construido. Se describe primero una formulación inicial, basada en la regresión del *root Hermite factor* y una regla de umbral (Sección 3.4.1); a continuación, su reformulación como un problema de clasificación supervisada (Sección 3.4.2), motivada por el diagnóstico empírico que se expone en el Capítulo 4. Por último se definen el oráculo de referencia, las métricas y el protocolo experimental con que ambas formulaciones se evalúan; los resultados cuantitativos se difieren a la Sección 4.4.

Conviene recordar el compromiso que motiva la selección. El parámetro $\delta \in (1/4, 1]$ de LLL (Sección 2.4.1) regula el equilibrio entre calidad y coste: valores próximos a la unidad (típicamente $\delta = 0,99$) producen bases de mayor calidad a costa de un mayor número de iteraciones, mientras que valores menores reducen el coste a expensas de una reducción más débil [28]. La práctica habitual fija $\delta = 0,99$ de forma conservadora, ignorando que muchas instancias alcanzan una calidad equivalente con valores sensiblemente

inferiores. El oráculo persigue explotar esa holgura instancia a instancia.

3.4.1. Formulación inicial: regresión del RHF con regla de umbral

La formulación inicial del oráculo, anticipada en la Etapa 6 del *pipeline* (Sección 3.1), se desarrolla aquí en detalle. Dada una base $\mathbf{B}$ de dimensión d , se extraen sus características pre-reducción (Sección 3.2.3) y, empleando el regresor de *gradient boosting* por dimensión correspondiente a d (Sección 3.3.2), se predice el *root Hermite factor* (2.7) resultante para cada uno de los cuatro valores candidatos $\delta \in \{0,55, 0,75, 0,90, 0,99\}$. Sea $\hat{\delta}_0(\mathbf{B}, \delta)$ dicha predicción. La regla de selección elige entonces el menor δ cuya calidad predicha no se degrada más de un umbral relativo $\tau \geq 0$ respecto de la obtenida con $\delta = 0,99$, idéntica a la anticipada en la Etapa 6 (3.1):

$$\delta^*(\mathbf{B}) = \min\{\delta \in \{0,55, 0,75, 0,90, 0,99\} : \hat{\delta}_0(\mathbf{B}, \delta) \leq (1 + \tau) \hat{\delta}_0(\mathbf{B}, 0,99)\}. \quad (3.4)$$

La regla descansa en el supuesto de monotonía de la calidad con δ (de modo que $\delta = 0,99$ proporciona la mejor de las cuatro reducciones y actúa como referencia), supuesto razonable a la luz del comportamiento general de LLL [10], [28], si bien su validez sobre el conjunto de datos de este trabajo se contrasta en el diagnóstico de la Sección 4.4.1. Como métrica rectora se adopta el *root Hermite factor* por ser el indicador estándar de calidad de reducción, independiente de la dimensión y del volumen del retículo (Sección 3.2.4).

En coherencia con la modalidad por dimensión, el vector de entrada de cada predicción está formado por las características geométricas y algebraicas de la base junto con las cuatro columnas *one-hot* del tipo de retículo, al que se añade δ como última componente; la dimensión d no se incluye, pues queda implícita en el modelo seleccionado. Las características se transforman con el mismo *StandardScaler* ajustado durante el entrenamiento de cada modelo, lo que garantiza la consistencia entre las fases de entrenamiento e inferencia. Para dimensiones no contempladas en el conjunto de modelos por dimensión, el sistema recurre al modelo global como *fallback*, según se justifica en la Sección 4.4. El coste de evaluar esta regla se reduce a cuatro inferencias del modelo, del orden de microsegundos, frente a una única ejecución de LLL, por lo que resulta despreciable en términos prácticos.

3.4.2. Reformulación como clasificación supervisada de δ^*

La evaluación de la formulación anterior, detallada en la Sección 4.4.1, revela que el regresor del *root Hermite factor* apenas captura el efecto *intra-base* del parámetro δ : el rango de las cuatro predicciones de una misma base es comparable o inferior al propio error del modelo, de modo que la regla (3.4) compara magnitudes indistinguibles del ruido y el umbral τ deja de tener efecto. Este hallazgo no responde a un defecto de implementación, sino a una propiedad del problema, la escasa varianza intra-base del RHF, y motiva una reformulación del mecanismo de selección.

Ante este diagnóstico se plantean dos reformulaciones, que se designan en lo sucesivo como Opción A y Opción B. Ambas se describen a continuación como decisiones de diseño; su comparación empírica, que fundamenta la elección de la segunda, se desarrolla en la Sección 4.4.2.

Opción A. Regresión del efecto residual. La primera reformulación conserva el enfoque de regresión, pero reemplaza el objetivo absoluto por el efecto diferencial de δ , $\Delta\text{RHF}(\mathbf{B}, \delta) = \delta_0(\mathbf{B}, \delta) - \delta_0(\mathbf{B}, 0,99)$. Al centrar el objetivo en la diferencia intra-base se pretende aislar la señal que el RHF absoluto diluye; la regla de selección (3.4) se mantiene, sustituyendo las predicciones absolutas por las del residual.

Opción B. Clasificación directa de δ^* (adoptada). La segunda reformulación traslada el umbral desde el momento de inferencia hacia el objetivo de entrenamiento. En lugar de comparar predicciones, se define para cada base un objetivo de clasificación: el menor δ cuya calidad *real* no se aparta más de una tolerancia relativa τ del óptimo alcanzable,

$$\delta^*(\mathbf{B}; \tau) = \min\{\delta \in \{0,55, 0,75, 0,90, 0,99\} : \delta_0(\mathbf{B}, \delta) \leq (1 + \tau) \delta_0(\mathbf{B}, \delta^{\text{opt}})\}, \quad (3.5)$$

donde $\delta_0(\mathbf{B}, \delta^{\text{opt}})$ es el RHF real mínimo definido por el oráculo de referencia (3.6). Sobre este objetivo se entrena un clasificador de *gradient boosting* (`XGBClassifier`), con un modelo independiente por cada par dimensión–tolerancia (d, τ) que toma como entrada el mismo vector de características pre-reducción descrito en la Sección 3.4.1. A diferencia de la regla (3.4), que en inferencia compara cuatro predicciones absolutas, el clasificador aprende directamente la decisión y la emite en un único paso.

La Opción B corrige por diseño las dos debilidades del planteamiento inicial. En primer lugar, al aprender el objetivo δ^* en lugar de cuatro valores absolutos del RHF, evita comparar diferencias del orden del error del modelo y produce, además, matrices de confusión interpretables. En segundo lugar, dado que τ interviene en la propia definición del objetivo (3.5), distintos valores de τ generan distribuciones de entrenamiento distintas y, por tanto, el parámetro recupera un efecto medible sobre el compromiso calidad–coste. La validación completa del oráculo reformulado se presenta en la Sección 4.4.

3.4.3. Oráculo de referencia y métricas de evaluación

Para juzgar la calidad de las decisiones del oráculo se define un *oráculo de referencia* (*ground truth*) que, para cada base, ejecuta LLL con los cuatro valores de δ y selecciona el que produce el menor RHF *real*:

$$\delta^{\text{opt}}(\mathbf{B}) = \arg \min_{\delta \in \{0,55, 0,75, 0,90, 0,99\}} \delta_0(\mathbf{B}, \delta). \quad (3.6)$$

Este óptimo por fuerza bruta, inalcanzable en producción por su coste, actúa como cota superior de acierto. La evaluación se articula en torno a cuatro magnitudes:

- La fracción de ahorro es la proporción de bases para las que $\delta^*(\mathbf{B}) < 0,99$, esto es, aquellas en las que el oráculo se aparta del valor conservador por defecto.
- La exactitud (*accuracy*) es la proporción de bases para las que $\delta^*(\mathbf{B}) = \delta^{\text{opt}}(\mathbf{B})$.
- La degradación de calidad Δq es el incremento relativo del RHF real al aplicar δ^* frente a $\delta = 0,99$,

$$\Delta q(\mathbf{B}) = \frac{\delta_0(\mathbf{B}, \delta^*) - \delta_0(\mathbf{B}, 0,99)}{\delta_0(\mathbf{B}, 0,99)} \times 100, \quad (3.7)$$

de la que se reportan la media y el percentil 95.

- La aceleración (*speedup*) es el cociente entre el tiempo de ejecución de LLL con $\delta = 0,99$ y con δ^* , acompañado del porcentaje de tiempo ahorrado.

La exactitud y la degradación capturan dimensiones complementarias del problema: una exactitud baja no implica un mal resultado si la degradación asociada es marginal, pues lo relevante en la práctica no es acertar el óptimo exacto, sino no perder calidad de reducción de forma apreciable.

3.4.4. Protocolo experimental

La validación del oráculo se realiza en dos fases complementarias.

Fase 1. Evaluación sobre datos retenidos. A partir del conjunto `lattice_dataset_v2.csv` se reconstruye, agrupando las cuatro filas correspondientes a cada base, una tabla en la que cada base queda caracterizada por su vector de *features* y por el RHF real asociado a cada δ , lo que permite determinar δ^{opt} según (3.6). Se emplea exactamente la misma partición canónica por base que en el entrenamiento por dimensión (Sección 3.3): el conjunto de prueba se carga mediante `load_test_base_ids()`, de modo que las 11 200 bases de test del oráculo coinciden, base a base, con las empleadas para evaluar los modelos predictivos, y la evaluación recae exclusivamente sobre bases no vistas. El oráculo se aplica con un barrido de umbrales $\tau \in \{0; 0,01; 0,02; 0,03; 0,05; 0,10; 0,20\}$ y los resultados se estratifican por dimensión y por tipo de retículo, lo que permite localizar los regímenes dimensionales y estructurales en los que la selección adaptativa resulta más o menos fiable.

Fase 2. Validación temporal en retículos frescos. Para confirmar que la mejora predicha se traduce en un ahorro de cómputo real, se generan 480 bases nuevas, no empleadas en ninguna fase previa: 30 por cada combinación de dimensión $d \in \{50, 80, 100, 120\}$ y tipo de retículo (uniforme, gaussiano, *sparse* y *q*-ario). Sobre cada base se aplica el oráculo en su régimen estricto ($\tau = 0$) y se mide el tiempo de ejecución de LLL tanto con $\delta = 0,99$ como con δ^* , ejecutando ambas reducciones sobre copias idénticas de la

matriz para garantizar la comparabilidad y repitiendo cada medición 5 veces por base para reportar la mediana y el rango intercuartílico (IQR). De este experimento se obtienen la aceleración y el porcentaje de tiempo ahorrado, desglosados por dimensión y tipo de retículo. El análisis de los resultados de ambas fases se desarrolla en la Sección 4.4.

3.5. Entorno de desarrollo y reproducibilidad

Plataforma hardware

Todos los experimentos se realizaron en un sistema de sobremesa con procesador AMD Ryzen 5 7600X (6 núcleos, 12 hilos, frecuencia base de 4,7 GHz), 32 GB de memoria RAM DDR5 a 6000 MT/s y una tarjeta gráfica NVIDIA GeForce RTX 5070 (arquitectura Blackwell, 12 GB GDDR7). El sistema anfitrión ejecuta Windows 11, y el entorno de cómputo se desplegó íntegramente sobre *Windows Subsystem for Linux 2* (WSL2) con Ubuntu 24.04 LTS como distribución invitada. La GPU se accede desde WSL2 a través de la capa de paravirtualización de GPU de WSL2 (*GPU paravirtualization*, GPU-PV), articulada sobre el dispositivo `/dev/dxg` que gestiona el componente `dxgkrnl`; sobre dicha capa, el controlador CUDA para WSL de NVIDIA (rama 595 del controlador para Windows) expone las bibliotecas de CUDA bajo `/usr/lib/wsl/lib`. Este mecanismo es ajeno a DirectML, que constituye una vía de aceleración distinta y no interviene en la ejecución de CUDA. Sobre esa capa, tanto XGBoost como PyTorch emplean la compilación de CUDA 12.4 (cu124; Tabla 3.3), compatible con la versión de CUDA que anuncia el controlador instalado.

Entorno software

La gestión de dependencias se realizó mediante Conda, definiendo el entorno `delta_predictor_release` a través del fichero `environment.yml` incluido en el repositorio. La versión de Python utilizada es la 3.12. Las principales bibliotecas empleadas, con sus restricciones de versión, se recogen en la Tabla 3.3.

La librería `fpyl11` [9] proporciona los *bindings* Python de `fp111`, implementación de referencia de LLL y sus variantes. Se instaló desde el canal `conda-forge` para garantizar la compatibilidad con las dependencias nativas (`libfp111`, `GMP`, `MPFR`). El entrenamiento de los modelos de *gradient boosting* con XGBoost [11] y de las redes neuronales con PyTorch [18] hace uso de la GPU mediante CUDA. Los *notebooks* Jupyter se editan y ejecutan desde Visual Studio Code con la extensión oficial de Jupyter.

Biblioteca	Versión
Python	3.12
fpvll	conda-forge (última estable)
NumPy	≥ 1.24 , < 2.0
pandas	≥ 2.0 , < 3.0
scikit-learn	≥ 1.3 , < 2.0
Matplotlib	≥ 3.7 , < 4.0
XGBoost	$\geq 2.0.0$
PyTorch	cu124 (CUDA 12.4)

TABLA 3.3. DEPENDENCIAS PRINCIPALES DEL ENTORNO
DELTA_PREDICTOR_RELEASE.

Reproducibilidad

Con objeto de garantizar la reproducibilidad de los experimentos, se adoptaron las siguientes medidas a lo largo de todo el *pipeline*:

Se empleó la semilla global `SEED = 42` en todos los componentes con aleatoriedad: la generación del *dataset* (`DATASET_SEED = 42` en `gen_features_and_target.py`), la asignación de bases al conjunto de prueba (`seed = 42` en `split_utils.py`), la inicialización de pesos de las redes neuronales (`torch.manual_seed(42)`) y los generadores de NumPy (`np.random.seed(42)`).

La partición entrenamiento/prueba es canónica y única, por lo que no se recalcula en cada experimento. El módulo `split_utils.py` reconstruye el identificador de cada base (`reconstruct_base_id`), asigna el 20 % de las bases a prueba de forma estratificada por (*d*, tipo) con semilla 42 y persiste la lista resultante en `splits/canonical_test_base_ids.csv`. Todos los *notebooks* (`dim_models`, `dim_mlps` y `oracle`) cargan esa misma lista mediante `load_test_base_ids()`, lo que garantiza que el conjunto de test sea idéntico, base a base, en todos los experimentos y que ninguna base se reparta entre entrenamiento y prueba.

La generación del *dataset* paraleliza sobre varios núcleos mediante `multiprocessing.Pool`. Las semillas no se derivan del índice del proceso, sino que se asignan *por tarea* con anterioridad a la paralelización: un generador maestro `np.random.default_rng(42)` produce de forma determinista la secuencia de semillas que se reparte, una a una, entre las tareas (cada combinación de tipo de retículo y dimensión), y cada *worker* se inicializa con la semilla que su tarea ya lleva asociada. De este modo, la correspondencia tarea-semilla queda fijada con independencia del número de *workers* y del orden en que estos las procesen, lo que garantiza que el fichero `lattice_dataset_v2.csv` (224 000 filas, 30 columnas) sea idéntico entre ejecuciones, incluso al variar el grado de paralelismo del equipo.

El conjunto de datos canónico que sustenta los resultados reportados en esta memoria se generó con anterioridad a la consolidación del esquema de semillas por tarea descrito en el párrafo previo. Por este motivo, una regeneración del *dataset* con la versión actual de `gen_features_and_target.py`, pese a ser determinista, produce un conjunto distinto (aunque estadísticamente equivalente) del empleado en los experimentos. Para garantizar la reproducibilidad *exacta* de las cifras, dicho conjunto se distribuye congelado junto con el repositorio, acompañado de su huella SHA-256 para su verificación; el script de generación se aporta, por tanto, para asegurar la trazabilidad y la reproducibilidad del *método*, y no la del fichero concreto.

El fichero `README.md` del repositorio especifica el orden exacto de ejecución de los scripts y *notebooks* (siete pasos), así como los artefactos que produce cada uno (ficheros `.csv`, `.pkl`, `.pt`) y los tiempos estimados de cómputo con la configuración hardware descrita.

Diferencias de $\pm 0,01$ en el coeficiente de determinación R^2 entre ejecuciones independientes son aceptables y se deben a la no determinismo residual de operaciones de punto flotante en GPU; no implican variación estadísticamente significativa en las conclusiones.

El entorno completo puede reconstruirse ejecutando:

```
conda env create -f environment.yml
conda activate delta_predictor_release
```

lo que garantiza versiones idénticas de todas las dependencias, incluyendo las bibliotecas nativas de `fpv111`.

4. RESULTADOS Y EVALUACIÓN

4.1. Resultados de los modelos globales

La Tabla 4.1 recoge el coeficiente de determinación R^2 obtenido sobre el conjunto de test para las tres familias de modelos globales descritas en la Sección 3.3.1, entrenadas sobre el dataset completo (224 000 instancias, d incluida como característica de entrada).

Modelo	$\log \delta(\mathbf{B}_{\text{red}})$	$\log \ \mathbf{b}_1^{\text{red}}\ $	RHF
XGBRFRegressor	0,9996	0,9783	0,7531
XGBRegressor	0,9997	0,9775	0,7499
MLP global	0,9996	0,9779	0,7458

TABLA 4.1. COEFICIENTE DE DETERMINACIÓN R^2 DE LOS MODELOS GLOBALES SOBRE EL CONJUNTO DE TEST (20 % DEL DATASET V2, 224 000 INSTANCIAS TOTALES).

Los dos primeros *targets* se predicen con una fiabilidad muy elevada. El defecto de ortogonalidad alcanza $R^2 > 0,999$ en las tres arquitecturas, un resultado esperable dado que esta magnitud crece de forma marcadamente monótona con la dimensión y con el volumen del retículo, información que el modelo global captura de manera directa a través de las características d y $\log |\det \mathbf{B}|$. La log-norma del primer vector reducido se estima con $R^2 \approx 0,978$, lo que confirma que el conjunto de descriptores geométricos contiene señal suficiente para anticipar la calidad de la base desde la perspectiva del problema del vector más corto.

El *root Hermite factor*, en cambio, se sitúa en $R^2 \approx 0,75$, un valor sensiblemente inferior, obteniendo el mejor resultado en el RF. Este descenso no obedece a una limitación de capacidad de los modelos, sino a la naturaleza del propio estadístico cuando se agregan dimensiones muy dispares. El RHF está diseñado precisamente para ser *independiente de la dimensión* [10]: converge hacia un valor límite característico del algoritmo de reducción a medida que $d \rightarrow \infty$, pero presenta una varianza relativa elevada en dimensiones bajas ($d \leq 30$) y muy reducida en dimensiones altas. Al entrenar un único modelo sobre toda la distribución, esta heterogeneidad inter-dimensional se traslada a la varianza residual y degrada el R^2 del estadístico agregado.

Las tres arquitecturas producen resultados prácticamente indistinguibles ($\Delta R^2 < 0,004$ en todos los *targets*, salvo en el RHF, donde $\Delta R^2 < 0,01$), pese a tratarse de paradigmas de aprendizaje muy distintos (conjuntos de árboles frente a redes neuronales). En términos de error absoluto, las predicciones globales se mantienen del orden de una unidad logarítmica para el defecto de ortogonalidad, en torno a 0,07 para la log-norma y por debajo

de 0,01 para el RHF, magnitudes coherentes con los valores de R^2 reportados. La coincidencia entre arquitecturas indica que el factor limitante a escala global no es la familia de modelos ni su capacidad, sino la propia heterogeneidad de la relación entrada→RHF a lo largo de las dimensiones. Esta observación motiva directamente la estrategia de especialización por dimensión que se evalúa en la sección siguiente.

La Figura 4.1 ilustra gráficamente esta jerarquía de dificultad mediante la dispersión predicho–real de los tres *targets*. El defecto de ortogonalidad y la log-norma del primer vector se alinean estrechamente con la diagonal de predicción perfecta, mientras que el RHF presenta una dispersión visiblemente mayor en torno a la identidad (coherente con su $R^2 \approx 0,75$), concentrada además en la banda de valores próximos al límite teórico de LLL, donde la varianza del estadístico es menor y los pequeños errores absolutos resultan más patentes en términos relativos.

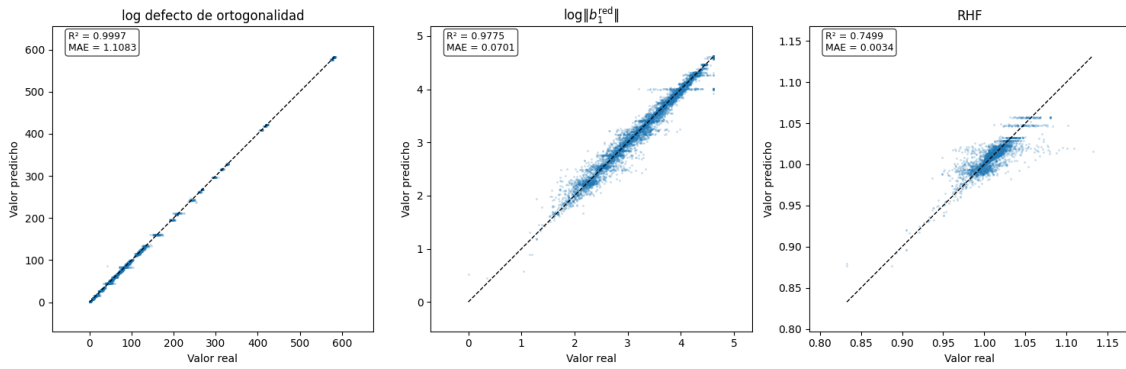


Fig. 4.1. Dispersión de valores predichos frente a valores reales del modelo global (XGBRegressor) sobre el conjunto de test, para los tres *targets*: log defecto de ortogonalidad, $\log \|b_1^{\text{red}}\|$ y RHF. La línea discontinua representa la identidad $y = x$ (predicción perfecta). La mayor dispersión del RHF respecto a la diagonal refleja su menor coeficiente de determinación a escala global.

4.2. Resultados de los modelos por dimensión

El análisis de los resultados globales identifica la mezcla de regímenes dimensionales como la principal fuente de error del modelo único para el RHF: las instancias con $d = 10\text{--}30$ (alta varianza relativa del estadístico) conviven con instancias de $d \geq 100$ (RHF convergiendo al valor teórico de LLL). Esta heterogeneidad es la que justifica la estrategia de especialización descrita en la Sección 3.3.2, en la que se entrena un modelo independiente por cada valor de d suprimiendo dicha variable de la entrada. A continuación se presentan sus resultados.

Resultados de XGBoost por dimensión

La Tabla 4.2 recoge el R^2 en test de las dos variantes de XGBoost, **XGBRFRegressor** (Random Forest, **RF**) y **XGBRegressor** (Gradient Boosting, **GB**), para cada dimensión y *target*, junto con los valores del modelo global como referencia. Ambas variantes producen resultados muy próximos en todas las dimensiones (diferencias inferiores a 0,002 en la práctica totalidad de los casos), por lo que las conclusiones cualitativas son independientes del algoritmo de conjunto empleado. El contraste cuantitativo frente al modelo global se desarrolla más adelante sobre las mismas bases de test (Tabla 4.3 y Figura 4.2), una vez justificada la métrica adecuada para realizarlo.

d	$\log \delta(\mathbf{B}_{\text{red}})$		$\log \ \mathbf{b}_1^{\text{red}}\ $		RHF	
	RF	GB	RF	GB	RF	GB
10	0,4588	0,4290	0,7510	0,7279	0,5288	0,5142
20	0,8197	0,8200	0,7902	0,7871	0,7194	0,7175
30	0,9437	0,9439	0,8417	0,8448	0,8680	0,8713
50	0,9902	0,9900	0,9340	0,9318	0,9692	0,9686
80	0,9962	0,9958	0,9598	0,9604	0,9881	0,9876
100	0,9991	0,9991	0,9722	0,9728	0,9944	0,9947
120	0,9996	0,9996	0,9802	0,9807	0,9968	0,9968
150	0,9958	0,9949	0,9776	0,9780	0,9929	0,9923
200	0,9999	0,9999	0,9707	0,9724	0,9977	0,9977
Global	0,9996	0,9997	0,9783	0,9775	0,7531	0,7499

TABLA 4.2. R^2 DE LOS MODELOS POR DIMENSIÓN PARA LAS DOS VARIANTES DE XGBOOST: RF (XGBRFREGRESSOR) Y GB (XGBREGRESSOR). LA ÚLTIMA FILA REPRODUCE EL R^2 DEL MODELO GLOBAL COMO REFERENCIA, AUNQUE NO ES DIRECTAMENTE COMPARABLE YA QUE ES UN AGREGADO.

Los resultados confirman la hipótesis de especialización con un patrón dimensión-dependiente bien diferenciado:

- Para $d \geq 30$, la estrategia por dimensión mejora al modelo global en el *target* RHF. La comparación se realiza sobre las mismas bases de test de cada dimensión: evaluados ambos modelos sobre idénticas instancias, su R^2 es muy similar en dimensiones altas (por ejemplo, 0,998 frente a 0,995 en $d = 200$), pero el error absoluto medio del modelo por dimensión es sistemáticamente menor, entre aproximadamente una cuarta parte y la mitad inferior para $d \geq 100$ (p. ej. $1,2 \times 10^{-4}$ frente a $2,3 \times 10^{-4}$ en $d = 200$) (Tabla 4.3). La diferencia frente al $R^2 \approx 0,75$ del modelo global no es directamente comparable, pues ese valor se mide sobre el conjunto de test con todas las dimensiones mezcladas, cuya varianza del RHF (denominador del

R^2) es mucho mayor que la de cada dimensión por separado. La ventaja de la especialización reside, por tanto, en la reducción del error absoluto, lo que confirma que la heterogeneidad inter-dimensional era la principal fuente de error del predictor único.

- Para $d \leq 100$, el modelo por dimensión obtiene peor R^2 en $\log \delta(\mathbf{B}_{\text{red}})$ que el modelo global. Este efecto se atribuye a que, al suprimir d de la entrada, el modelo especializado pierde la señal dominante para una magnitud que escala fuertemente con la dimensión, sin que la información geométrica restante baste para compensarla en dimensiones donde, además, la varianza intrínseca del *target* es elevada y al ser el modelo general también goza de mayor cantidad de datos.
- En el régimen asintótico ($d \geq 100$), los tres *targets* alcanzan $R^2 \geq 0,96$, de forma consistente con la convergencia teórica del factor de Hermite de LLL hacia un valor límite a medida que d crece [10]: la varianza residual del *target* disminuye y la tarea de regresión se vuelve progresivamente más sencilla.

d	MAE ($\times 10^{-3}$)		R^2	
	por dim.	global	por dim.	global
10	12,13	12,34	0,514	0,504
20	4,97	5,14	0,718	0,715
30	3,03	3,26	0,871	0,869
50	1,32	1,50	0,969	0,966
80	0,60	0,70	0,988	0,988
100	0,38	0,54	0,995	0,992
120	0,24	0,36	0,997	0,996
150	0,20	0,26	0,992	0,996
200	0,12	0,23	0,998	0,995

TABLA 4.3. MODELOS POR DIMENSIÓN Y GLOBAL DEL RHF EVALUADOS SOBRE LAS *MISMAS* BASES DE TEST DE CADA DIMENSIÓN. EL MAE SE EXPRESA EN UNIDADES DE 10^{-3} . EL R^2 ES EQUIPARABLE EN DIMENSIONES ALTAS, MIENTRAS QUE EL ERROR ABSOLUTO DEL MODELO POR DIMENSIÓN ES SISTEMÁTICAMENTE MENOR.

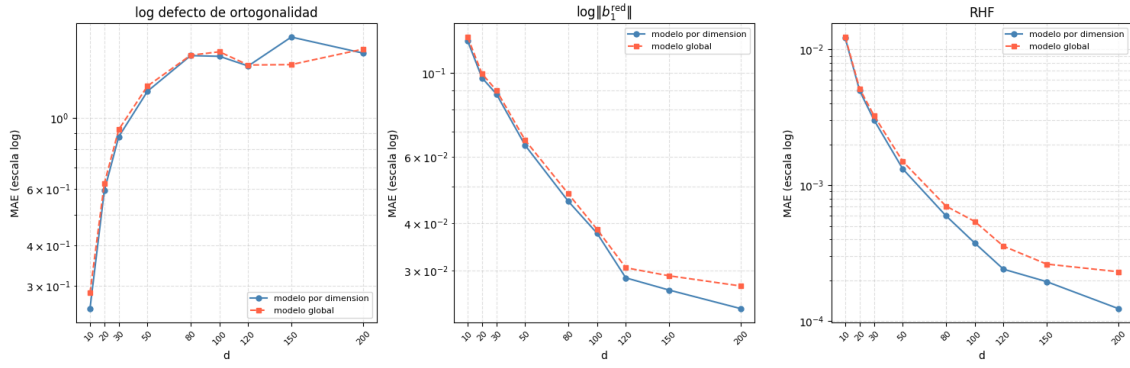


Fig. 4.2. MAE en test de los modelos por dimensión y del modelo global, evaluados sobre las *mismas* bases de test de cada dimensión, para los tres *targets* (eje vertical en escala logarítmica). Comparar sobre instancias idénticas neutraliza el efecto de la varianza y hace que las diferencias de error sean directamente interpretables.

Al extender la comparación sobre idénticas bases a los tres *targets* (Figura 4.2) se observa que la ventaja de la especialización es *dependiente del target*. En el RHF, el modelo por dimensión se separa con claridad del global a partir de $d = 80$ y la distancia crece con la dimensión, hasta reducir el MAE casi a la mitad en $d = 200$. En $\log \|b_1^{\text{red}}\|$ la mejora es más discreta, pero consistente en las dimensiones más altas. Para $\log \delta(\mathbf{B}_{\text{red}})$, en cambio, ambas curvas son prácticamente indistinguibles e incluso se cruzan en dimensiones altas: el modelo global, que conserva d entre sus entradas, no resulta superado precisamente en la magnitud que más fuertemente escala con la dimensión. Este comportamiento es coherente con el déficit de R^2 del modelo por dimensión en dicho *target* (Tabla 4.2) y delimita el alcance real de la hipótesis de especialización: beneficia a las métricas cuya estructura fina depende de la geometría de la base (señaladamente el RHF, la más relevante para caracterizar la calidad de la reducción) y resulta neutra para aquellas dominadas por un escalado dimensional que el modelo global ya captura de forma explícita.

Comparativa MLP por dimensión frente a XGBoost

Con el fin de contrastar ambos paradigmas de aprendizaje también en la modalidad especializada, se replica el experimento con un MLP por cada par (d, target) . La Tabla 4.4 compara los R^2 del RHF (el *target* más sensible a la estrategia de entrenamiento) obtenidos por XGBRFRegressor, XGBRegressor y el MLP por dimensión. La Figura 4.3 muestra la comparativa completa para los tres *targets*.

d	XGB-RF	XGB-GB	MLP	$\Delta_{\text{MLP-RF}}$
10	0,5288	0,5142	0,4838	-0,0450 ×
20	0,7194	0,7175	0,7007	-0,0187 ×
30	0,8680	0,8713	0,8667	-0,0013 ✓
50	0,9692	0,9686	0,9642	-0,0050 ✓
80	0,9881	0,9876	0,9842	-0,0039 ✓
100	0,9944	0,9947	0,9305	-0,0639 ×
120	0,9968	0,9968	0,9630	-0,0338 ×
150	0,9929	0,9923	-13,52	-14,5138 ×
200	0,9977	0,9977	-5335,6	-5336,6759 ×

TABLA 4.4. R^2 DEL RHF POR DIMENSIÓN: COMPARATIVA ENTRE XGBRFREGRESSOR, XGBREGRESSOR Y MLP POR DIMENSIÓN. SE MARCA CON ✓ CUANDO EL MLP ES COMPETITIVO ($|\Delta| < 0,01$) Y CON × CUANDO PRESENTA DEGRADACIÓN SIGNIFICATIVA.

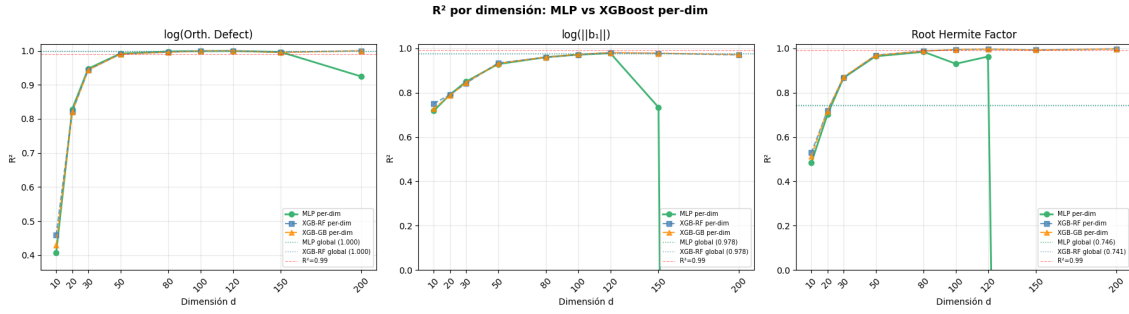


Fig. 4.3. Comparativa del R^2 por dimensión entre el MLP y las dos variantes de XGBoost por dimensión, para los tres *targets*. Para el RHF (panel derecho) se aprecia la divergencia del MLP a partir de $d = 100$, donde XGBoost mantiene $R^2 > 0,99$ mientras el MLP se degrada progresivamente.

El MLP por dimensión iguala a XGBoost solo para $d \in \{30, 50, 80\}$ ($|\Delta R^2| < 0,01$); a partir de $d = 100$ se degrada (0,9305 en $d = 100$, 0,963 en $d = 120$) y diverge para $d \geq 150$ (-13,52 en $d = 150$ y $\approx -5,33 \times 10^3$ en $d = 200$), una inestabilidad ajena a XGBoost.

Este fenómeno se explica por la combinación de dos factores. En primer lugar, el subconjunto de entrenamiento para $d \geq 100$ contiene únicamente 12 800 instancias (3 200 matrices $\times$ 4 valores de δ), la mitad que el disponible para $d \leq 80$, lo que reduce el régimen de datos del que dispone una red neuronal con cerca de $5,0 \times 10^4$ parámetros entrenables. En segundo lugar, y de forma determinante, la varianza del RHF en dimensiones altas es extremadamente reducida porque el estadístico converge a su valor teórico: dado que el R^2 normaliza el error cuadrático por la varianza del *target*, en este régimen pequeñas

desviaciones absolutas de la predicción se amplifican al dividir por una varianza casi nula. XGBoost aprovecha en estas condiciones su mayor eficiencia estadística sobre datos tabulares, mientras que el MLP muestra indicios de convergencia inadecuada en las 300 épocas de entrenamiento. La implicación práctica es directa: para la construcción del oráculo de selección de δ (Sección 3.3.2), XGBoost es la familia de modelos preferible en todo el rango dimensional.

Para los *targets* $\log \delta(\mathbf{B}_{\text{red}})$ y $\log \|\mathbf{b}_1^{\text{red}}\|$, el MLP por dimensión es estadísticamente equivalente a XGBoost hasta $d = 120$, pero el colapso en régimen de pocos datos reaparece en dimensiones altas: la log-norma cae a $R_{\text{MLP}}^2 = 0,7337$ en $d = 150$ y diverge a $-44,03$ en $d = 200$ (frente a $0,9707$ de XGB-RF), mientras que el defecto de ortogonalidad descende a $0,9245$ en $d = 200$; la degradación no se limita al RHF.

Evaluación estratificada por tipo de retículo

El elevado R^2 agregado de los modelos por dimensión en el RHF (Tabla 4.2) admite una lectura más matizada al desglosar la métrica por tipo de retículo *dentro* de cada dimensión. La Tabla 4.5 y la Figura 4.4 presentan el R^2 del mejor modelo por dimensión (XGBRegressor, *target* RHF) evaluado de forma estratificada sobre el conjunto de test.

d	<i>gaussian</i>	<i>q-ario</i>	<i>sparse</i>	<i>uniform</i>
10	0,3477	0,2211	0,7313	0,4733
20	0,6252	0,6379	0,7327	0,5986
30	0,6760	0,8120	0,7217	0,6531
50	0,6743	0,9235	0,7238	0,6375
80	0,6405	0,8728	0,6564	0,6660
100	0,6878	0,8413	0,6599	0,6433
120	0,6785	0,0474	0,6965	0,5959
150	0,6380	n.d. [†]	0,6831	0,6078
200	0,6067	n.d. [†]	0,6662	0,6115

TABLA 4.5. R^2 DEL RHF POR DIMENSIÓN Y TIPO DE RETÍCULO (XGBREGRESSOR POR DIMENSIÓN), EVALUADO DE FORMA ESTRATIFICADA SOBRE EL CONJUNTO DE TEST.

Este desglose revela dos hechos relevantes que conviene interpretar con cuidado. En primer lugar, el R^2 *intra-tipo* (en torno a $0,59$ – $0,73$ para los tipos *gaussian*, *sparse* y *uniform*) es notablemente inferior al R^2 agregado por dimensión, que supera $0,99$ a dimensiones altas. La aparente contradicción se resuelve al recordar que el R^2 se normaliza por la varianza del *target* en el conjunto sobre el que se mide: al agregar los cuatro tipos, las diferencias sistemáticas de RHF *entre* tipos generan una varianza total elevada que el modelo explica con holgura, en buena medida gracias a las columnas *one-hot* que codifican el tipo de retículo, lo que infla el R^2 conjunto. Al fijar el tipo, esa varianza entre

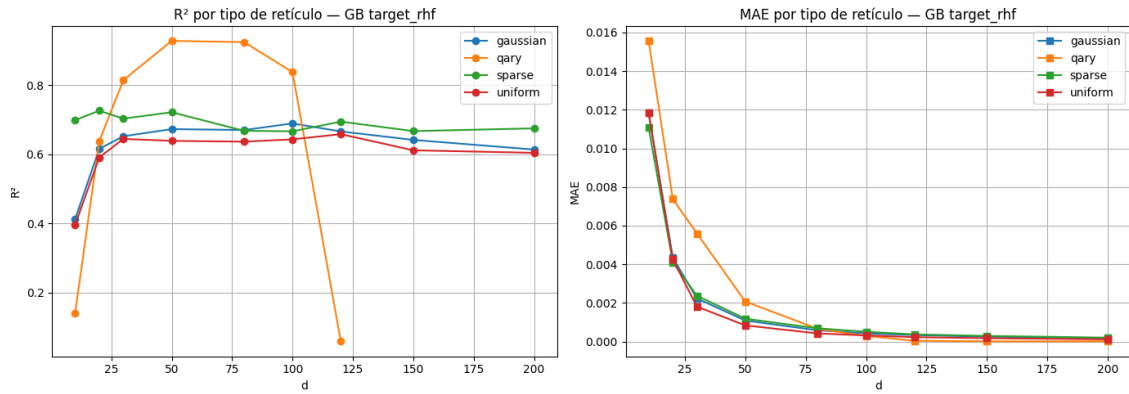


Fig. 4.4. R^2 del RHF por dimensión desglosado por tipo de retículo, para el modelo XGBRegressor por dimensión. El desglose intra-tipo revela valores sensiblemente inferiores al R^2 agregado y el colapso de la métrica en los retículos q -arios a dimensiones altas.

grupos desaparece y solo resta la dispersión residual *dentro* de cada tipo, mucho menor y, por tanto, más difícil de explicar. La predicción del RHF en su componente verdaderamente fina, a tipo y dimensión fijos, es, en consecuencia, una tarea apreciablemente más exigente de lo que sugiere la métrica agregada.

En segundo lugar, los retículos q -arios exhiben un patrón singular: su R^2 asciende hasta 0,92 en dimensiones intermedias ($d = 50-80$) y después se desploma (0,84 en $d = 100$, apenas 0,05 en $d = 120$) hasta volverse indefinido para $d \geq 150$. Este colapso no refleja un fallo del modelo, sino una inestabilidad numérica de la propia métrica: en dimensiones altas la varianza del RHF dentro del tipo q -ario es prácticamente nula (sus bases convergen al mismo factor de Hermite de forma muy concentrada), de modo que el denominador del R^2 tiende a cero y el coeficiente se vuelve no significativo. El error absoluto confirma esta lectura: el MAE de las predicciones q -arias a $d = 200$ se mantiene del orden de 10^{-5} , es decir, las predicciones son extremadamente precisas en términos absolutos pese a que el R^2 no esté definido. Por ello, las entradas marcadas con † deben entenderse como casos en los que el R^2 deja de ser una medida informativa y procede recurrir a métricas de error absoluto.

4.3. Análisis de importancia de features

Los modelos XGBoost por dimensión alcanzan $R^2 > 0,99$ para $\log \delta(\mathbf{B}_{\text{red}})$ en $d \geq 80$ y para el RHF en $d \geq 100$ (Tabla 4.2). Esta sección no se limita a constatar dicha capacidad predictiva, sino que indaga *qué* características de la base sin reducir la sustentan, con un doble propósito: validar que el modelo apoya sus predicciones en magnitudes geométricamente plausibles (y no en artefactos del conjunto de datos) y contrastar si redescubre, de forma puramente estadística, las relaciones teóricas que gobiernan la reducción de retículos.

El análisis combina dos técnicas complementarias. En primer lugar, la importancia de *ganancia* (*gain*) propia de XGBoost, que mide la reducción media del error de predicción atribuible a cada *feature* al dividir nodos del árbol. Es una primera aproximación de cálculo inmediato, pero adolece de un sesgo conocido: tiende a sobrevalorar las *features* de alto rango o de elevada cardinalidad frente a las de variación reducida. En segundo lugar, y como análisis principal, los valores SHAP (*SHapley Additive exPlanations*), que cuantifican la contribución marginal de cada *feature* a cada predicción individual a partir de los valores de Shapley de teoría de juegos, con garantías de consistencia y aditividad local de las que carece la *ganancia* [12]. Los valores SHAP se calculan con el estimador exacto para modelos de árboles (*TreeExplainer*) sobre una submuestra aleatoria del conjunto de test de cada par (*target*, *d*), y la importancia global de cada *feature* se obtiene como media ponderada, por tamaño de test, de su media de valor absoluto $\bar{\phi}_j = \frac{1}{n} \sum_i |\phi_j^{(i)}|$ sobre todas las dimensiones.

Importancia de ganancia: una primera aproximación sesgada

La Figura 4.5 recoge la importancia de *ganancia* promediada sobre dimensiones para las quince *features* más relevantes, en dos variantes: incluyendo las indicadoras *one-hot* del tipo de retículo (Figura 4.5a) y excluyéndolas (Figura 4.5b).

En la variante con tipo, el resultado más llamativo es la dominancia de las indicadoras *one-hot* del tipo de retículo y, en particular, de la del tipo *q*-ario, que alcanza una *ganancia* normalizada de 0,529 para el defecto, 0,463 para $\log \|\mathbf{b}_1^{\text{red}}\|$ y 0,433 para el RHF; en el caso de $\log \|\mathbf{b}_1^{\text{red}}\|$, la indicadora del tipo uniforme contribuye además con 0,176. Esta preponderancia no debe interpretarse como evidencia de que el tipo de retículo sea el verdadero motor de la predicción, sino como una manifestación directa del sesgo de la métrica de *ganancia*: las particiones binarias sobre una variable categórica que separa subpoblaciones de *target* muy distintas producen reducciones de error grandes y poco frecuentes que dicha métrica sobrepondera, eclipsando a las *features* geométricas continuas.

La variante sin las indicadoras de tipo despeja este artefacto y revela la jerarquía entre las *features* estrictamente geométricas. Una vez neutralizada la variable categórica, la *ganancia* identifica como predictores de cabeza, para el defecto, la pendiente del perfil GSO (0,25) y el parámetro δ (0,234), con la fracción $|\mu_{i,j}| > 0,5$ (0,201) muy próxima; para $\log \|\mathbf{b}_1^{\text{red}}\|$, la media de las normas $\|\mathbf{b}_i\|$ (0,289) y $\log |\det(\mathbf{B})|$ (0,210); y para el RHF, la fracción $|\mu_{i,j}| > 0,5$ (0,277), netamente por encima del resto, seguida de la pendiente del perfil GSO (0,142).

El contraste con SHAP es, no obstante, solo parcial, y esa discrepancia es en sí misma informativa. Ambas técnicas coinciden en señalar como relevantes un núcleo común de magnitudes geométricas (el parámetro δ y la fracción $|\mu_{i,j}| > 0,5$ figuran en cabeza tanto en *ganancia* como en SHAP para el defecto y el RHF), pero difieren en la *feature* principal y en el ordenamiento fino. La *ganancia* eleva la pendiente del perfil GSO al primer puesto del defecto, posición que SHAP le niega al relegarla a un lugar secunda-

rio. Para el RHF en cambio la *feature* con mayor relevancia, con diferencia es la fracción $|\mu_{i,j}| > 0,5$, posición que el análisis SHAP rechaza poniendo en cabeza el parámetro δ ; y para $\log \|\mathbf{b}_1^{\text{red}}\|$ corona la media de $\|\mathbf{b}_i\|$ y $\log |\det(\mathbf{B})|$, mientras que SHAP sitúa en cabeza δ y $\min_i \|\mathbf{b}_i\|$. Lejos de ser una contradicción, esta divergencia es consecuencia directa de sus definiciones operacionales. La ganancia mide la optimización local del árbol durante el entrenamiento, mientras que SHAP desglosa de manera robusta la atribución de cada variable sobre el output del modelo. De este modo, los resultados justifican el uso analítico de SHAP gracias a sus propiedades de consistencia, posicionando a la ganancia intrínseca no como una métrica definitiva, sino como un elemento de control que permite visibilizar el sesgo estructural de los árboles de decisión ante determinadas tipologías de datos.

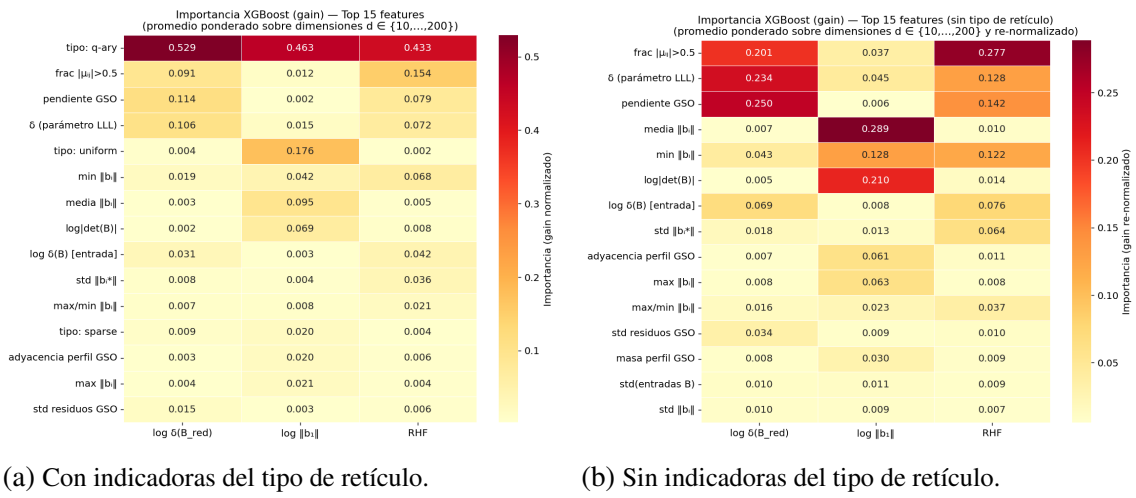


Fig. 4.5. Importancia de *ganancia* de XGBoost (*gain* normalizado), promediada de forma ponderada sobre las dimensiones $d \in \{10, \dots, 200\}$, para las quince *features* de mayor peso. (a) Al incluir las indicadores *one-hot* del tipo, la del tipo q -ario sobrepondera de forma artefactual el resto de predictores. (b) Al excluirlas y ponderar nuevamente, la jerarquía entre *features* geométricas comparte con la obtenida mediante SHAP un núcleo común de predictores, aunque no la *feature* principal, lo que evidencia el sesgo de la métrica de *ganancia* y justifica relegarla al papel de control.

Importancia SHAP global y dispersión de la atribución

La Figura 4.6 presenta la importancia SHAP global de cada *target*. Para cuantificar de forma compacta cuán concentrada o difusa es la atribución entre *features*, se calcula la entropía de Shannon $H = -\sum_j p_j \log p_j$ sobre la distribución normalizada $p_j = \bar{\phi}_j / \sum_k \bar{\phi}_k$: un valor de H bajo indica que pocas *features* explican la predicción, mientras que un valor alto refleja una contribución repartida entre muchas. Los valores obtenidos son $H = 0,908$ nats para el defecto, $H = 2,658$ nats para $\log \|\mathbf{b}_1^{\text{red}}\|$ y $H = 2,473$ nats para el RHF.

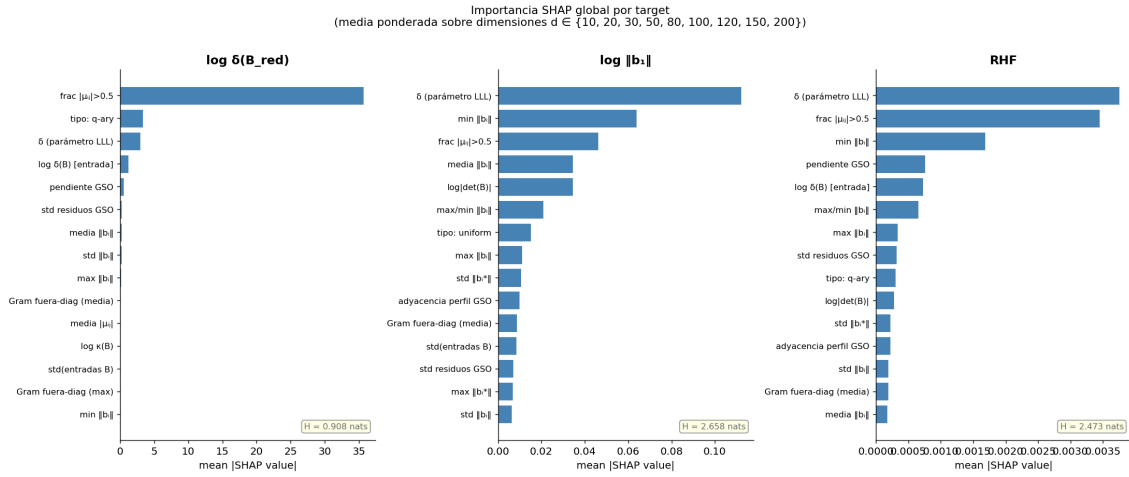


Fig. 4.6. Importancia SHAP global (media de valor absoluto, ponderada sobre dimensiones) para los tres *targets*, mostrando las quince *features* de mayor peso. El recuadro de cada panel indica la entropía de Shannon H de la distribución de importancias. El defecto se explica esencialmente por una única *feature* estructural (H baja), mientras que log $\|\mathbf{b}_1^{red}\|$ y el RHF reparten la atribución entre más predictores (H alta).

La jerarquía SHAP difiere de forma significativa de la sugerida por la *ganancia* y se aparta, además, de las expectativas teóricas más inmediatas. Para el defecto log $\delta(\mathbf{B}_{red})$, la *feature* dominante es la fracción de coeficientes de Gram-Schmidt con $|\mu_{i,j}| > 0.5$, cuyo valor medio de |SHAP| supera en cerca de un orden de magnitud al del siguiente predictor (la indicadora del tipo q -ario) seguido del parámetro δ ; esta *feature* concentra así casi toda la atribución, en coherencia con la entropía mínima del *target*. Para log $\|\mathbf{b}_1^{red}\|$, el predictor principal es el parámetro de reducción δ , seguido de $\min_i \|\mathbf{b}_i\|$ y de la fracción $|\mu_{i,j}| > 0.5$; el logaritmo del determinante log $|\det(\mathbf{B})|$, que la heurística Gaussiana situaría en cabeza, aparece relegado a la quinta posición. Para el RHF, el parámetro δ y la fracción $|\mu_{i,j}| > 0.5$ comparten la primera posición con magnitudes muy próximas, por delante de $\min_i \|\mathbf{b}_i\|$.

Dirección de los efectos: diagramas SHAP *beeswarm*

La importancia global cuantifica *cuánto* influye cada *feature*, pero no *en qué sentido*. Los diagramas *beeswarm* de las Figuras 4.7, 4.8 y 4.9, calculados para la dimensión intermedia $d = 80$ (donde $R_{defecto}^2 \approx 0.996$ y $R_{RHF}^2 \approx 0.99$), resuelven esta cuestión. Cada punto es un ejemplo del conjunto de test; su posición horizontal es la contribución SHAP a la predicción y su color, el valor de la *feature* en escala original (rojo, valores altos; azul, valores bajos).

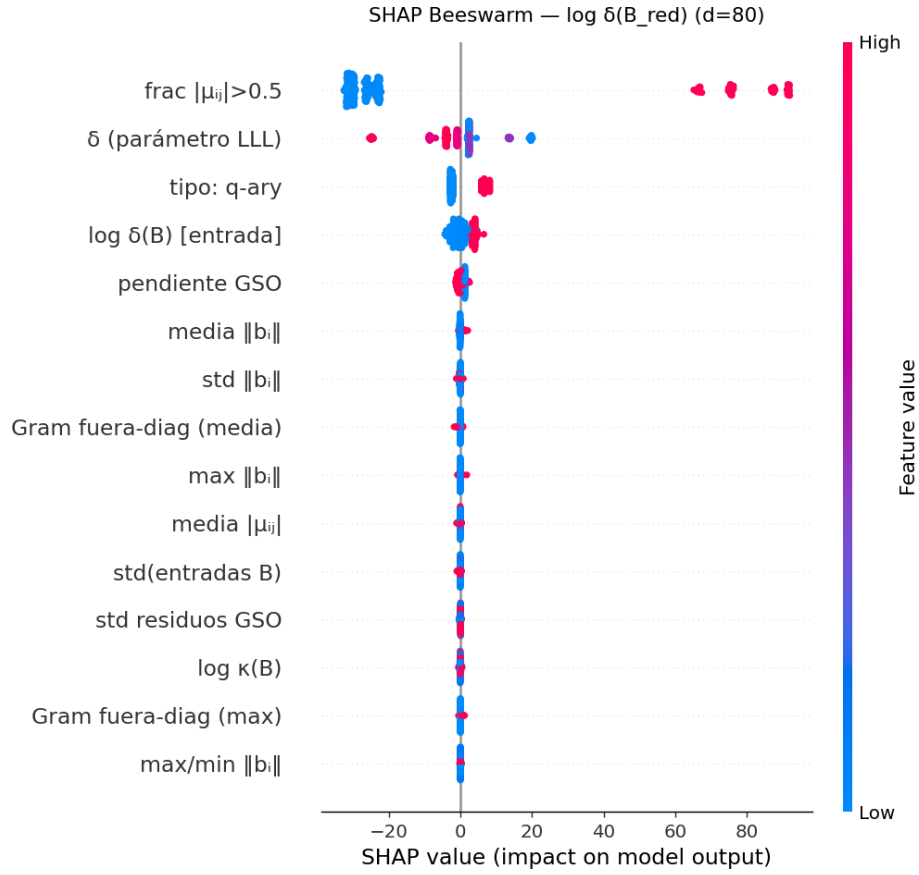


Fig. 4.7. Diagrama SHAP *beeswarm* para $\log \delta(\mathbf{B}_{\text{red}})$ en $d = 80$. La *feature* dominante es la fracción de coeficientes de Gram–Schmidt con $|\mu_{i,j}| > 0,5$: los valores altos (puntos rojos) producen contribuciones SHAP fuertemente positivas y los bajos (puntos azules), marcadamente negativas, lo que evidencia una relación monótona creciente entre la lejanía de la base de entrada respecto de la condición de *size-reduction* ($|\mu_{i,j}| \leq 1/2$, justamente la que el algoritmo LLL impone) y el defecto de ortogonalidad alcanzado tras la reducción. El modelo se apoya así en un *proxy* estructural directo de cuán reducida está la base de partida, en lugar de reconstruir término a término la identidad $\log \delta(\mathbf{B}) = \sum_i \log \|\mathbf{b}_i\| - \log |\det(\mathbf{B})|$. El parámetro de reducción δ aparece en segundo plano con un efecto *inverso* sobre el defecto: los valores altos (puntos rojos), que refuerzan la condición de Lovász y producen una base más ortogonal, desplazan la predicción hacia defectos menores (SHAP negativo), mientras que los bajos la elevan. La concentración casi total de la atribución en una única *feature* es coherente con la mínima entropía de este *target* (Figura 4.6).

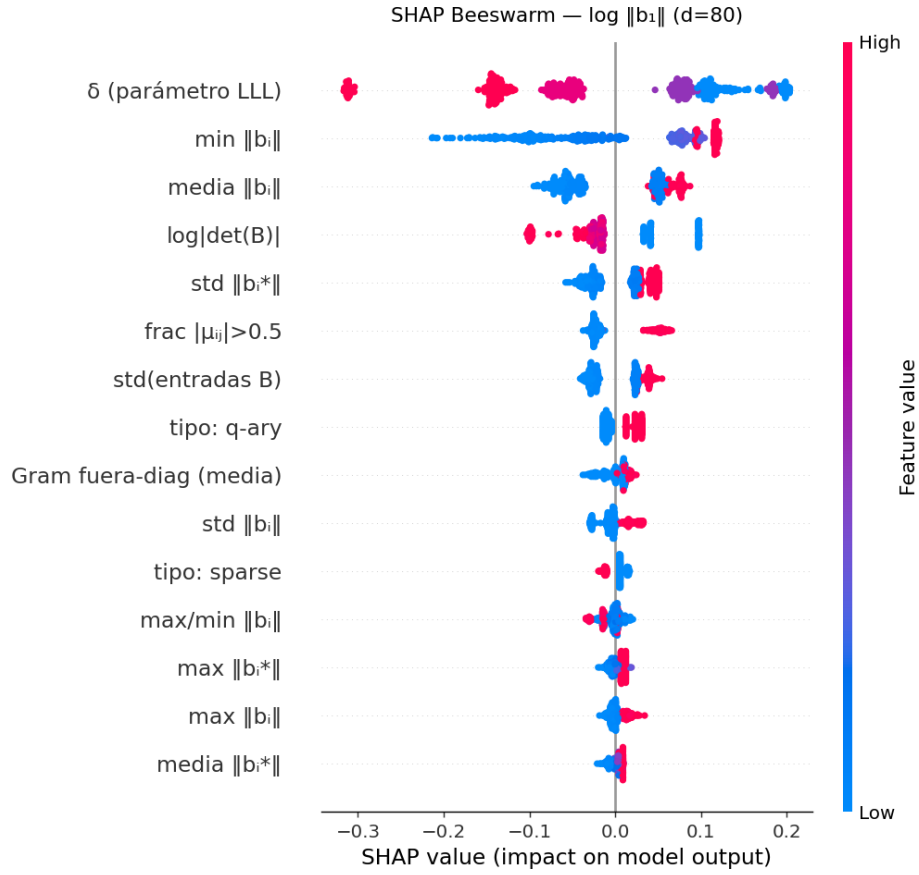


Fig. 4.8. Diagrama SHAP *beeswarm* para $\log \|b_1^{\text{red}}\|$ en $d = 80$. La atribución se reparte entre varios predictores, en coherencia con la elevada entropía de este *target*. El parámetro δ encabeza la jerarquía con un efecto *inverso*: los valores altos (puntos rojos), asociados a una reducción más agresiva, desplazan la predicción hacia log-normas menores (SHAP negativo), mientras que los valores bajos la elevan. La *feature* $\min_i \|b_i\|$ exhibe, por el contrario, una relación *directa* y nítida: una norma mínima de entrada pequeña (puntos azules) arrastra sistemáticamente la predicción a la baja, en coherencia con que un vector de partida ya corto impone una cota inferior a la longitud alcanzable tras la reducción. El logaritmo del determinante $\log |\det(\mathbf{B})|$, predictor que la heurística Gaussiana situaría en cabeza, presenta un efecto comparativamente débil y de signo poco definido, lo que refuta la forma fuerte de dicha heurística a d fijo: con el volumen del retículo de variabilidad acotada dentro de cada dimensión, la log-norma queda gobernada por la agresividad de la reducción y por la estructura métrica de la base de entrada, no por el término volumétrico.

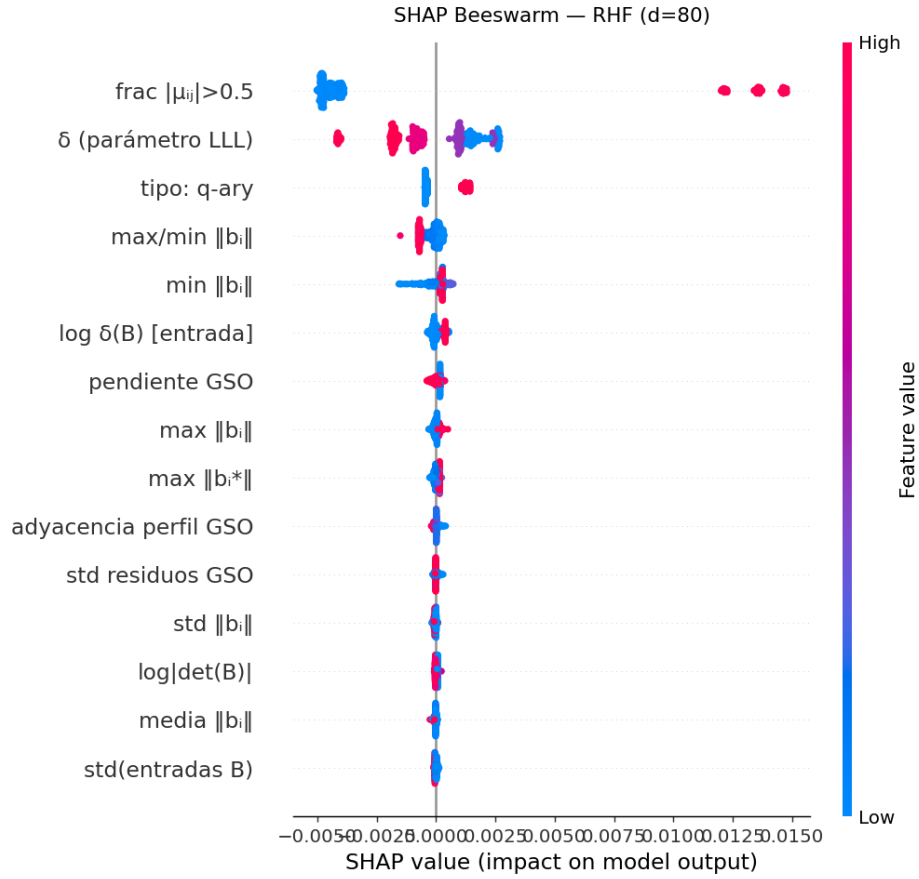


Fig. 4.9. Diagrama SHAP *beeswarm* para el RHF en $d = 80$. La fracción de coeficientes $|\mu_{i,j}| > 0,5$ encabeza la atribución con un efecto monótono creciente y claramente separado del resto: las instancias con valores altos (puntos rojos) reciben las contribuciones positivas de mayor amplitud. El parámetro δ ocupa la segunda posición, pero contribuye con ambos signos, lo que apunta a un efecto mediado por interacciones con la estructura de la base más que a una dependencia directa de signo constante. Por detrás emergen descriptores estructurales finos, la indicadora del tipo q -ario y varias magnitudes del perfil de Gram–Schmidt (pendiente, adyacencia y dispersión de residuos), en coherencia con que el RHF, al normalizar $\|\mathbf{b}_1^{\text{red}}\|$ por $|\det(\Lambda)|^{1/d}$, cancela el término volumétrico dominante y deja la varianza residual gobernada por la geometría de la base. Nótese la escala SHAP comprimida ($\approx \pm 0,015$) frente a la del *beeswarm* de la norma, reflejo de la reducida varianza intrínseca de este *target*.

Merece subrayarse que, sobre el defecto y la longitud, las dos métricas de calidad en escala absoluta, δ actúa *con el mismo sentido*: un δ mayor reduce simultáneamente el defecto de ortogonalidad y la longitud del primer vector reducido. Lejos de ser una coincidencia, esta concordancia es la firma empírica, a escala de instancia, del papel que la teoría reserva a δ : como único parámetro que gradúa la agresividad de la reducción a través de la condición de Lovász (2.12), su incremento tensa el perfil de Gram–Schmidt y produce a la vez vectores más cortos y más ortogonales, de modo que ambas métricas mejoran de forma monótona con él, el mismo comportamiento que la garantía del peor caso (2.13) encapsula en un factor $2/\sqrt{4\delta - 1}$ decreciente en δ . Esta monotonía conjunta

es, además, el supuesto sobre el que se articula el oráculo adaptativo (Sección 3.4.1): que el modelo recupere su dirección a partir únicamente de descriptores pre-reducción refuerza que lo aprendido tiene correlato geométrico y no es un artefacto del conjunto de datos. En el RHF (Figura 4.9) se reproduce el patrón monótono de la fracción $|\mu_{i,j}| > 0,5$, en tanto que δ pasa a contribuir con ambos signos; no se trata de una excepción al mecanismo anterior, sino de su consecuencia directa: al normalizar la longitud por $|\det(\Lambda)|^{1/d}$, el RHF cancela la escala volumétrica sobre la que δ ejerce su efecto monótono, y solo queda una varianza residual fina cuyo signo depende de interacciones con otras propiedades estructurales de la base.

Estabilidad de la importancia con la dimensión

Dado que se entrena un modelo independiente por dimensión, conviene examinar si los predictores dominantes son universales o varían con d . La Figura 4.10 traza la evolución de la importancia SHAP de las ocho *features* principales de cada *target* a lo largo del rango $d \in \{10, \dots, 200\}$.

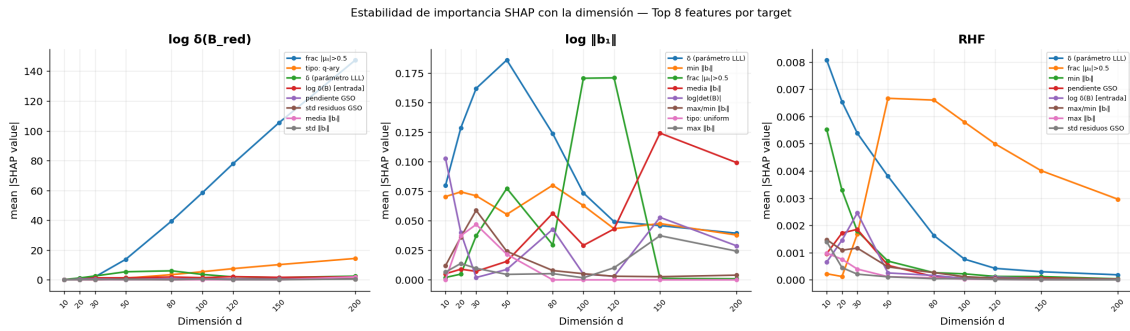


Fig. 4.10. Evolución de la importancia SHAP (media de $|\text{SHAP}|$) de las ocho *features* principales de cada *target* en función de la dimensión d . Se observa el relevo de predictores con la dimensión, especialmente acusado para $\log \|\mathbf{b}_1^{\text{red}}\|$, lo que concuerda con su mayor entropía de atribución. Nótese el contraste de escalas verticales: la atribución del defecto crece sin saturación con d , mientras que la del RHF se colapsa.

El análisis revela un *relevo de predictores* dependiente del régimen dimensional, sobre el que se superpone un fenómeno de escala que distingue netamente a los tres *targets* y que admite una lectura geométrica directa.

Para el defecto $\log \delta(\mathbf{B}_{\text{red}})$, la fracción $|\mu_{i,j}| > 0,5$ no solo conserva el protagonismo en todo el rango (tras un breve liderazgo de δ en las dimensiones más bajas) sino que su importancia *crece de forma sostenida y casi lineal* con d , hasta superar en $d = 200$ en más de dos órdenes de magnitud al resto de predictores. Este crecimiento es coherente con la naturaleza del propio *target*: el defecto $\delta(\mathbf{B}) = (\prod_i \|\mathbf{b}_i\|) / |\det(\mathbf{B})|$ es una magnitud *extensiva*, cuyo logaritmo se descompone como una suma de d términos no negativos y que, por la desigualdad de Hadamard, acumula desviación respecto de la ortogonalidad a medida que aumenta el número de vectores. Como las atribuciones SHAP se expresan en

las unidades del modelo, su escala absoluta crece de forma mecánica con la del *target*. La concentración en una única *feature* se intensifica, además, porque el número de coeficientes de Gram–Schmidt es $d(d-1)/2$: en dimensión alta, la fracción que viola $|\mu_{i,j}| \leq 1/2$, la condición de *size-reduction* (2.11), constituye un resumen estructural progresivamente más completo y fiable de cuán lejos se halla la base de entrada del régimen reducido. El defecto se vuelve así *más* explicable por este proxy a medida que crece d , en coherencia con su entropía de atribución mínima.

Para $\log \|\mathbf{b}_1^{\text{red}}\|$, la rotación de *features* es la más acusada de los tres *targets*: $\log |\det(\mathbf{B})|$ encabeza fugazmente en $d = 10$, δ alcanza su máximo en $d = 50$ y decae después, la fracción $|\mu_{i,j}| > 0,5$ toma el relevo hacia $d \in \{100, 120\}$ para desvanecerse a continuación, y en las dimensiones más altas ($d \geq 150$) encabeza la media de las normas $\|\mathbf{b}_i\|$. Esta volatilidad, sin un predictor estable que domine en todo el rango, explica directamente la entropía de atribución más elevada de este *target*.

Para el RHF, el comportamiento es el opuesto al del defecto y es, posiblemente, el más informativo desde el punto de vista teórico. La escala completa de las atribuciones se *colapsa* con la dimensión: ningún predictor mantiene un peso apreciable en régimen asintótico. Esta extinción es la firma directa de la convergencia del RHF (2.7) hacia su valor límite característico $\delta_0 \approx 1,02$, prácticamente independiente de la instancia, a medida que d crece [10]: al reducirse la varianza intrínseca del *target*, mengua de forma paralela la varianza que cualquier *feature* puede explicar. Dentro de esa tendencia se aprecia un relevo nítido: el parámetro δ domina en dimensiones bajas (donde el RHF dista aún de su asíntota y la agresividad de la reducción gobierna la variabilidad entre instancias) y decrece después de forma monótona, cediendo el liderazgo a la fracción $|\mu_{i,j}| > 0,5$ en torno a $d = 50$. La caída sostenida de la atribución de δ corrobora que, en dimensión alta, el RHF se torna esencialmente insensible al valor de δ , observación que se retoma en el diagnóstico del oráculo (Sección 4.4.1) y que es, en último término, la razón de que la holgura intra-base sobre la que aquel pretende operar resulte tan estrecha.

Interpretación teórica

Los resultados anteriores admiten una lectura coherente con la geometría de la reducción de retículos, si bien obligan a matizar las expectativas iniciales. Tres hechos vertebran esta interpretación: el *proxy* estructural que el modelo elige para cada *target* en lugar de los términos explícitos de su definición, el *sentido* de los efectos dominantes, y el modo en que la atribución *escala con la dimensión*, dictado por el carácter extensivo o intensivo de cada magnitud.

Defecto de ortogonalidad. El defecto se define como $\delta(\mathbf{B}_{\text{red}}) = \prod_i \|\mathbf{b}_i^{\text{red}}\| / |\det(\mathbf{B})| \geq 1$, con igualdad (por la desigualdad de Hadamard) únicamente en bases ortogonales, por lo que cabría anticipar que dominaran $\log |\det(\mathbf{B})|$ y las estadísticas de normas, componentes explícitos de la fórmula. El modelo, en cambio, privilegia con holgura la fracción de co-

eficientes con $|\mu_{i,j}| > 0,5$. Lejos de ser anómalo, este hallazgo es geoméricamente natural: la condición de *size-reduction* del algoritmo LLL exige $|\mu_{i,j}| \leq 1/2$ (2.11) [28], de modo que la proporción de coeficientes que violan dicho umbral en la base de entrada es un indicador estructural directo de cuán lejos está esta de estar reducida y, por tanto, de cuánto excederá el producto de sus normas al determinante tras la reducción. El modelo identifica así un predictor estructural más informativo que la descomposición algebraica de la propia definición, en lugar de reconstruir la identidad $\log \delta(\mathbf{B}) = \sum_i \log \|\mathbf{b}_i\| - \log |\det(\mathbf{B})|$ término a término. Esta concentración (reflejada en la entropía mínima del *target*, $H = 0,908$ nats) no solo se mantiene, sino que se *intensifica* con la dimensión (Figura 4.10): el defecto es una magnitud *extensiva*, cuyo logaritmo suma d contribuciones no negativas y acumula desviación respecto de la ortogonalidad a medida que crece el número de vectores; y dado que los coeficientes de Gram–Schmidt son $d(d-1)/2$, la fracción que viola la *size-reduction* constituye en dimensión alta un resumen estructural progresivamente más completo y fiable. A ello se añade, en segundo plano, el parámetro δ con un efecto *inverso*: un δ mayor refuerza la condición de Lovász (2.12), produce una base más ortogonal y reduce el defecto.

Longitud del primer vector reducido. La heurística Gaussiana establece que $\lambda_1(\Lambda) \approx \sqrt{d/(2\pi e)} \cdot |\det(\mathbf{B})|^{1/d}$, lo que para d fijo predice $\log \|\mathbf{b}_1^{\text{red}}\| \approx \frac{1}{d} \log |\det(\mathbf{B})| + c(d)$ y situaría a $\log |\det(\mathbf{B})|$ como predictor principal. Los datos refutan esta expectativa en su forma fuerte, el determinante queda relegado a la quinta posición: dentro de cada dimensión, el volumen del retículo presenta una variabilidad acotada y la log-norma alcanzada queda gobernada, ante todo, por el parámetro de reducción δ (que controla la agresividad del algoritmo a través de (2.12)) y por $\min_i \|\mathbf{b}_i\|$, pues un vector de entrada ya corto acota inferiormente lo que la reducción puede lograr. El modelo aprende, por tanto, un *refinamiento dependiente de la instancia* de la heurística asintótica: no reconstruye la ley volumétrica, sino que captura los factores operativos y estructurales que determinan la desviación de cada base respecto a ella. Conviene subrayar que δ actúa sobre esta log-norma con el *mismo sentido* que sobre el defecto (en ambas, métricas de calidad en escala absoluta, un δ mayor produce un valor menor), lo que no es sino el correlato empírico, a escala de instancia, del papel que la teoría reserva a δ como único mando del compromiso calidad–coste: la garantía del peor caso $\|\mathbf{b}_1\| \leq (2/\sqrt{4\delta-1})^{n-1} \lambda_1(\Lambda)$ (2.13) encapsula esa monotonía en un factor decreciente en δ . La atribución más repartida de este *target* ($H = 2,658$ nats, la mayor de las tres) se manifiesta además en el relevo de predictores más acusado a lo largo de la dimensión (Figura 4.10).

Root Hermite Factor. El RHF normaliza $\|\mathbf{b}_1^{\text{red}}\|$ por $|\det(\Lambda)|^{1/d}$, cancelando el término volumétrico dominante de la heurística Gaussiana. La varianza residual depende entonces de detalles estructurales finos (el valor de δ , la proximidad a la *size-reduction* y la forma del perfil GSO), lo que se traduce en la coexistencia en cabeza de δ y de la fracción $|\mu_{i,j}| > 0,5$. A diferencia de los dos *targets* anteriores, aquí δ contribuye con ambos signos:

cancelada la escala sobre la que actúa de forma monótona, su efecto pasa a estar mediado por interacciones con la estructura de la base más que por una dependencia directa de signo constante. Como magnitud *intensiva*, el RHF converge hacia un valor límite característico $\delta_0 \approx 1,02$, prácticamente independiente de la instancia, a medida que d crece [10]; de ahí que su atribución, lejos de amplificarse como la del defecto, se *extinga* con la dimensión (Figura 4.10), al menguar la varianza que cualquier *feature* puede explicar. La hipótesis de partida, que el RHF presentaría la atribución más difusa, se confirma solo parcialmente: su entropía (2,473 nats) supera con claridad a la del defecto (0,908 nats), pero es inferior a la de $\log \|\mathbf{b}_1^{\text{red}}\|$ (2,658 nats). La dificultad de predicción del RHF en dimensiones bajas, reflejada en su menor R^2 (Tabla 4.2), responde así a la combinación de una atribución repartida con una varianza intrínseca reducida del propio *target*, antes que a la dispersión de la importancia por sí sola.

Síntesis. El contraste entre las dos técnicas constituye, en sí mismo, un resultado metodológico: la importancia de *ganancia* habría conducido a atribuir el poder predictivo al tipo de retículo, mientras que SHAP revela que el modelo se apoya en propiedades geométricas interpretables (la proximidad a la *size-reduction*, la presencia de vectores cortos y el parámetro de control δ). El comportamiento opuesto de las atribuciones con la dimensión (creciente para el defecto, decreciente para el RHF) no es un artefacto del modelo, sino el reflejo fiel del carácter extensivo o intensivo de cada métrica reticular: la escala de la explicación sigue a la del fenómeno explicado. Y el que ninguno de los tres *targets* se explique por la reconstrucción literal de su fórmula teórica, sino por *proxies* estructurales más informativos, refuerza la idea de que el modelo no memoriza relaciones algebraicas, sino que aprende un refinamiento de la teoría de reducción sensible a las particularidades de cada instancia. La Tabla 4.6 resume el contraste entre las hipótesis de partida y los hallazgos.

<i>Target</i>	Predicción inicial	<i>Feature</i> SHAP dominante observada	Resultado
$\log \delta(\mathbf{B}_{\text{red}})$	$\log \det(\mathbf{B}) $ y normas	fracción $ \mu_{i,j} > 0,5$	Refutada
$\log \ \mathbf{b}_1^{\text{red}}\ $	$\log \det(\mathbf{B}) $ (heur. Gaussiana)	δ , $\min_i \ \mathbf{b}_i\ $	Refutada
RHF	atribución muy difusa	δ y fracción $ \mu_{i,j} > 0,5$	Parcial

TABLA 4.6. CONTRASTE ENTRE LAS HIPÓTESIS INICIALES SOBRE LA *FEATURE* SHAP DOMINANTE Y LOS RESULTADOS OBSERVADOS PARA CADA *TARGET*.

4.4. Evaluación de la selección adaptativa de δ

Esta sección desarrolla la evaluación empírica de las dos formulaciones del oráculo introducidas en la Sección 3.4. Se diagnostican primero las causas del fracaso de la formulación inicial (Sección 4.4.1); se contrastan a continuación ambas reformulaciones para fundamentar la finalmente adoptada (Sección 4.4.2); y se valida por último el oráculo reformulado en profundidad (Sección 4.4.3).

4.4.1. Diagnóstico de la formulación inicial

La formulación inicial (Sección 3.4.1) selecciona, a partir de las cuatro predicciones absolutas del RHF, el menor δ que satisface $\hat{\delta}_0(\mathbf{B}, \delta) \leq (1 + \tau) \hat{\delta}_0(\mathbf{B}, 0,99)$. Su evaluación sobre el conjunto de prueba revela un comportamiento patológico que invalida el diseño y cuyas causas, lejos de obedecer a un defecto de implementación, son inherentes al problema.

El síntoma más visible es la insensibilidad al umbral: el oráculo degenera a predecir sistemáticamente $\delta = 0,55$ con independencia de τ , de modo que sus decisiones (y, con ellas, el porcentaje de selección y la degradación de calidad que provocan) permanecen idénticas para todo $\tau \in \{0; 0,01; 0,02; 0,03; 0,05; 0,10; 0,20\}$. Su exactitud frente al óptimo estricto $\delta^*(\tau = 0)$ es de apenas el 8,1 % sobre el test, muy por debajo del 53,0 % del *baseline* trivial *siempre* $\delta = 0,99$. La conclusión es contundente: la selección adaptativa, tal como está formulada, rinde peor que no seleccionar en absoluto.

La causa raíz se aprecia en la figura 4.11. El rango de predicción *intra-base*, $\max_{\delta} \hat{\delta}_0(\mathbf{B}, \delta) - \min_{\delta} \hat{\delta}_0(\mathbf{B}, \delta)$, es prácticamente nulo en todas las dimensiones y resulta comparable o inferior al error absoluto medio del propio modelo. Dicho de otro modo, las cuatro predicciones que la regla (3.4) compara difieren entre sí por menos que el ruido del regresor, de manera que la comparación opera sobre ruido y el umbral τ carece de margen sobre el que actuar. Este fenómeno no es un artefacto del entrenamiento, sino el reflejo de una propiedad del problema: la varianza del RHF *dentro* de una misma base al variar δ es minúscula frente a su varianza *entre* bases, que es la que el modelo aprende con holgura.

Dos observaciones adicionales orientan el rediseño. La primera es que la oportunidad que el oráculo persigue existe realmente: al determinar por fuerza bruta el δ que minimiza el RHF de cada base, $\delta = 0,99$ resulta óptimo únicamente para el 52,9 % ³ de las instancias, mientras que el 47,1 % restante alcanza su mejor reducción con un valor estrictamente menor, repartido entre $\delta = 0,90$ (26,9 %), $\delta = 0,75$ (12,0 %) y $\delta = 0,55$ (8,1 %), según la distribución de la figura 4.12. El fracaso de la formulación inicial es, por tanto, costoso: deja sin explotar una holgura presente en casi la mitad de las bases. La segun-

³El 52,9 % se calcula sobre el conjunto de datos completo (56 000 bases), mientras que el 53 % citado como *baseline siempre* $\delta = 0,99$ mide la magnitud análoga (la fracción en que $\delta = 0,99$ es el óptimo) pero sobre el conjunto de test (11 200 bases); la leve diferencia obedece únicamente al subconjunto sobre el que se evalúan.

da es que el supuesto de monotonía $\text{RHF}(\mathbf{B}, 0,55) \geq \dots \geq \text{RHF}(\mathbf{B}, 0,99)$ (que situaría a $\delta = 0,99$ como óptimo) solo se cumple en el 82,3 % de las bases; el 17,7 % restante presenta violaciones atribuibles al ruido de precisión finita de LLL y a dependencias no lineales del proceso de reducción. Ello obliga a sustituir $\text{RHF}(\mathbf{B}, 0,99)$ por el óptimo real, alcanzado en $\delta^*(\mathbf{B}) = \arg \min_{\delta \in \mathcal{D}} \text{RHF}(\mathbf{B}, \delta)$ con $\mathcal{D} = \{0,55, 0,75, 0,90, 0,99\}$, es decir $\text{RHF}^*(\mathbf{B}) = \min_{\delta \in \mathcal{D}} \text{RHF}(\mathbf{B}, \delta)$, como referencia frente a la que medir cualquier selección.

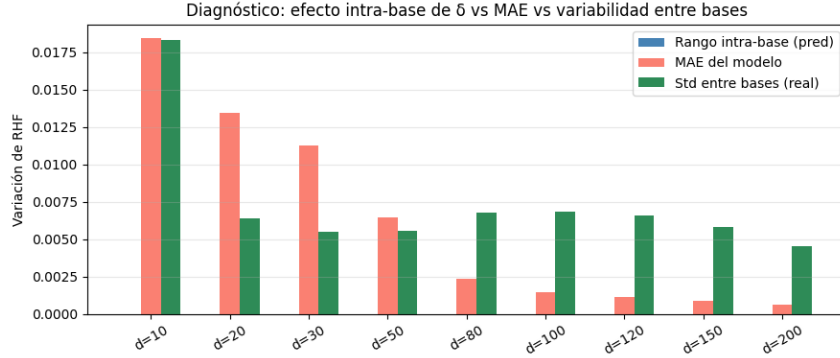


Fig. 4.11. Efecto intra-base de δ en la predicción del modelo RHF (azul), comparado con el MAE del modelo (rojo) y la desviación estándar real entre bases (verde), por dimensión. El rango intra-base es prácticamente nulo, lo que evidencia que el modelo no captura la señal de δ .

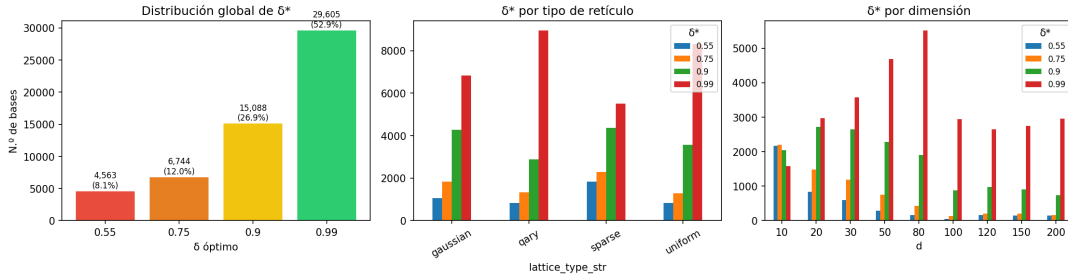


Fig. 4.12. Distribución de δ^* (global, por tipo de retículo y por dimensión). El $\delta = 0,99$ no es el óptimo para el 47,1 % de las bases.

4.4.2. Comparación de las dos formulaciones

El diagnóstico anterior descarta la formulación inicial y sugiere dos vías de corrección, formalizadas como Opción A y Opción B en la Sección 3.4.2. Esta sección las contrasta empíricamente sobre el conjunto de prueba ($n = 11\,200$ bases) para fundamentar la elección.

La Opción A (regresión del efecto residual $\Delta\text{RHF}(\delta) = \text{RHF}(\delta) - \text{RHF}(0,99)$) pretende aislar la señal intra-base que el RHF absoluto diluye. Sin embargo, hereda el problema de fondo. El error absoluto medio del modelo residual oscila entre 9×10^{-3} para $d = 10$ y $1,5 \times 10^{-4}$ para $d = 200$, magnitudes del mismo orden que el propio rango real de ΔRHF .

Al ser el error comparable a la cantidad que se desea estimar, el modelo no distingue de forma fiable si $\widehat{\Delta RHF}(\delta) \leq \tau \cdot RHF(0,99)$ y, en la evaluación, reproduce la misma degeneración que la formulación inicial. La Opción A queda así descartada por la misma razón empírica que el oráculo absoluto: la señal que necesita está por debajo del umbral de ruido del regresor.

La Opción B, clasificación directa de δ^* , evita por completo la comparación de magnitudes ruidosas, al aprender la decisión en lugar de reconstruirla a partir de predicciones del RHF. La tabla 4.7 compara las siete estrategias evaluadas con $\tau = 0$, el escenario más exigente, en el que el objetivo es exactamente δ^* .

Estrategia	Sel $\delta < 0,99$ (%)	Acc. (%)	OB1 (%)	$\overline{\Delta q_{vs099}}$	Δq_{p95}	$\overline{\Delta q_{vs\ opt}}$
Siempre $\delta = 0,99$	0.0	53.0	79.9	+0.000	+0.000	+0.006
Siempre $\delta = 0,55$	100.0	8.1	20.1	+1.080	+4.562	+1.087
Clase mayoritaria	0.0	53.0	79.9	+0.000	+0.000	+0.006
Aleatorio	75.1	24.5	59.0	+0.415	+2.374	+0.421
Fuerza bruta (cota sup.)	47.0	100.0	100.0	-0.006	+0.000	+0.000
Oráculo abs. (actual)	100.0	8.1	20.1	+1.080	+4.562	+1.087
Clasificador δ^* (Opc. B)	36.6	59.8	89.9	+0.123	+0.610	+0.129

TABLA 4.7. COMPARATIVA DE ESTRATEGIAS DE SELECCIÓN DE δ ($\tau = 0$, CONJUNTO DE TEST, $N = 11\ 200$ BASES). ACC. = FRACCIÓN DE ACIERTOS SOBRE $\delta^*(\tau = 0)$; OB1 = TOLERANCIA ± 1 NIVEL DE δ .

El clasificador δ^* supera a los cuatro baselines no triviales (siempre $\delta = 0,99$, clase mayoritaria, selección aleatoria y oráculo absoluto inicial), con una exactitud del 59,8 % frente al 53,0 % del baseline conservador. La figura 4.13 sitúa cada estrategia en el plano calidad–coste: el clasificador se ubica en la región intermedia de la frontera de Pareto, entre los baselines conservadores y la cota inalcanzable del oráculo de referencia, con una degradación muy inferior a la de la estrategia agresiva *siempre* $\delta = 0,55$.

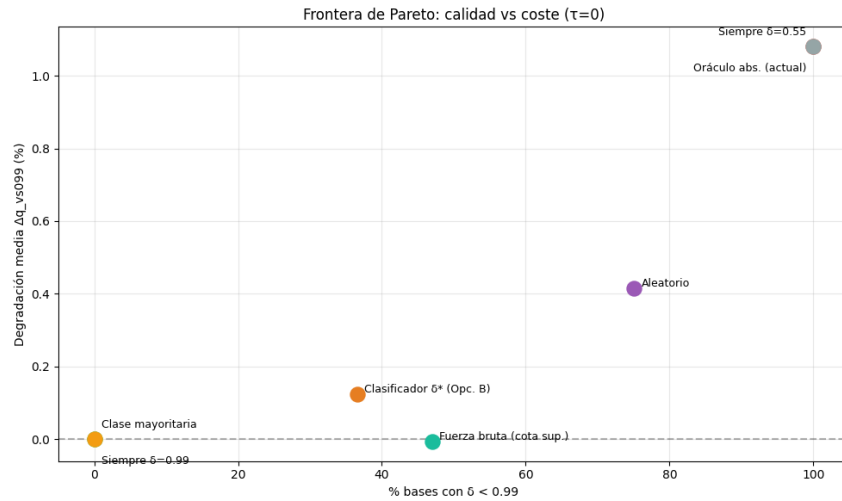


Fig. 4.13. Frontera de Pareto: degradación media $\Delta q_{vs0,99}$ frente al porcentaje de bases en que se elige $\delta < 0,99$ ($\tau = 0$). El clasificador δ^* (Opción B) se sitúa entre los baselines conservadores y la cota de fuerza bruta, con una degradación muy inferior a *siempre* $\delta = 0,55$.

La elección de la Opción B se sostiene, en suma, sobre tres criterios cuantitativos: (i) mejora la exactitud de todos los baselines no triviales (59,8 % frente a 53,0 %); (ii) selecciona un $\delta < 0,99$ en el 36,6 % de las bases con una degradación media de apenas +0,123 % respecto del valor por defecto; y (iii) a diferencia de las dos formulaciones anteriores, responde de forma medible al parámetro τ , lo que habilita el ajuste explícito del compromiso calidad–coste que se analiza a continuación.

4.4.3. Validación del oráculo reformulado

Establecida la superioridad de la Opción B, esta sección la valida en profundidad: el efecto del parámetro τ , la estructura de sus errores, su comportamiento según dimensión y tipo de retículo, y la traducción de la mejora en ahorro de cómputo real sobre bases no vistas. Las métricas de degradación $\Delta q_{vs0,99}$ y $\Delta q_{vs\text{opt}}$ empleadas son las definidas en la Sección 3.4.3.

Efecto del umbral τ . La figura 4.14 descompone el comportamiento del clasificador en función de τ en tres magnitudes (porcentaje de bases con $\delta < 0,99$, exactitud frente al objetivo $\delta^*(\tau)$ y degradación media $\Delta q_{vs0,99}$), contrastadas en cada caso con los dos baselines triviales y con la cota de fuerza bruta. En el punto de operación estricto $\tau = 0$ el clasificador es genuinamente selectivo: reserva $\delta < 0,99$ para el 36,6 % de las bases, acierta el δ^* exacto en el 59,8 % de los casos y lo hace con una degradación media de apenas +0,123 %, muy próxima al óptimo alcanzable por fuerza bruta (−0,006 % con un 47,0 % de selección). La pequeña brecha entre ambos (+0,123 % frente a −0,006 %) cuantifica el coste de los errores de clasificación: el oráculo ideal nunca degrada la cali-

dad por debajo de $\delta = 0,99$, mientras que el clasificador incurre ocasionalmente en esa penalización al confundir clases adyacentes.

Al relajar el umbral, las tres magnitudes evolucionan de forma acoplada y revelan la naturaleza del compromiso. La selectividad se desploma casi de inmediato: el porcentaje de bases con $\delta < 0,99$ salta del 36,6 % al 99,8 % en $\tau = 0,01$ y satura en el 100 % desde $\tau = 0,02$. La exactitud, por su parte, crece de forma monótona, 59,8 % ($\tau = 0$), 79,2 % (0,01), 88,6 % (0,02), 91,4 % (0,03), 95,9 % (0,05) y 99,9 % (0,10), pero esta mejora es en buena medida aparente: a medida que τ crece, el propio objetivo $\delta^*(\tau)$ se desplaza hacia $\delta = 0,55$ (que pasa de ser óptimo en el 8,1 % de las bases a serlo en el 96,0 % en $\tau = 0,05$), de modo que acertar se reduce a predecir sistemáticamente el valor más agresivo. Lo confirma el cruce de los baselines en el panel central: la exactitud de *siempre* $\delta = 0,99$ se hunde del 53,0 % a cero, mientras que la de *siempre* $\delta = 0,55$ asciende del 8,1 % al 100 %. El tercer panel desvela el precio de esa exactitud: la degradación media del clasificador crece en paralelo, +0,477 % ($\tau = 0,01$), +0,841 % (0,03), +1,036 % (0,05), hasta converger al techo de +1,08 % que impone la estrategia *siempre* $\delta = 0,55$. Dicho de otro modo, la exactitud solo alcanza el 100 % cuando la selección ha degenerado por completo y la calidad se ha sacrificado al máximo; el clasificador traza así, desde arriba, la frontera de fuerza bruta, manteniéndose en todo el recorrido por debajo de la degradación de la estrategia agresiva trivial.

Esta transición abrupta (concentrada en el entorno de $\tau \approx 0,01$) es la firma directa de la escasa varianza intra-base del RHF diagnosticada en la Sección 4.4.1: la diferencia entre $\text{RHF}(\mathbf{B}, 0,55)$ y el óptimo $\text{RHF}^*(\mathbf{B})$ es tan reducida que una tolerancia relativa de apenas el 1 % basta para que $\delta = 0,55$ resulte admisible en la práctica totalidad de las bases. Lejos de invalidar el diseño, esta sensibilidad confirma que, en la Opción B, τ opera como una palanca efectiva, aunque de recorrido corto, del compromiso calidad–coste: $\tau = 0$ proporciona el régimen útil, selectivo y de degradación casi nula, mientras que cualquier $\tau \gtrsim 0,01$ colapsa hacia la estrategia agresiva trivial. El rango de τ con valor práctico queda así acotado a la vecindad de cero.

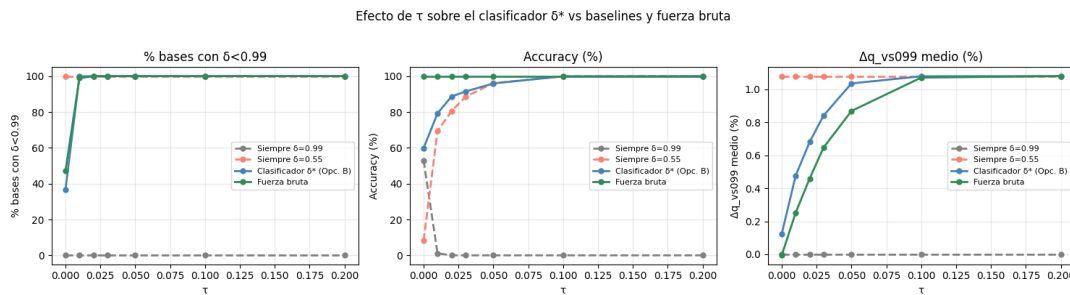


Fig. 4.14. Efecto de τ sobre el porcentaje de bases con $\delta < 0,99$, la exactitud y la degradación de calidad del clasificador δ^* , comparado con la cota de fuerza bruta y los baselines triviales. El parámetro τ produce una transición abrupta en $\tau \approx 0,01$.

Análisis de errores. La figura 4.15 recoge la matriz de confusión del clasificador para $\tau = 0$. Los errores se concentran en la confusión entre $\delta = 0,90$ y $\delta = 0,99$, las dos clases más numerosas y más difíciles de distinguir a partir de las características pre-reducción. La tolerancia *off-by-one* (± 1 nivel de δ) alcanza el 89,9 %, lo que indica que, incluso cuando el clasificador falla, el error es por lo general de una sola clase adyacente y la consiguiente degradación de calidad permanece acotada.

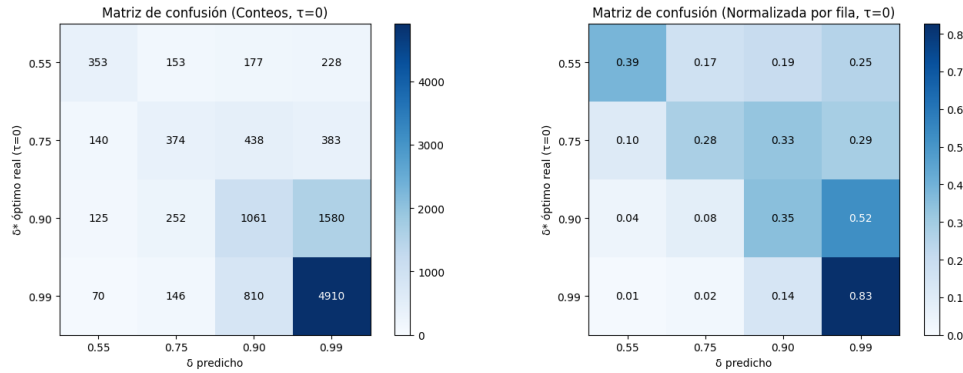


Fig. 4.15. Matrices de confusión del clasificador δ^* ($\tau = 0$): por conteos (izquierda) y normalizada por fila (derecha). Los errores se concentran entre las clases $\delta = 0,90$ y $\delta = 0,99$.

Desglose por dimensión y tipo de retículo. La figura 4.16 desglosa la exactitud y la degradación media por dimensión y tipo de retículo. El patrón es marcadamente heterogéneo. Los retículos q -arios alcanzan la mayor exactitud en dimensiones intermedias (por encima del 90 % para $50 \leq d \leq 100$) pero ésta cae a valores en torno al 40 % para $d \geq 120$; los gaussianos mejoran de forma monótona con la dimensión (de ~ 28 % en $d = 10$ a ~ 80 % en $d = 200$), los uniformes mantienen una exactitud elevada en casi todo el rango y los dispersos un comportamiento intermedio. La degradación de calidad asociada es en todos los casos pequeña y decreciente con la dimensión, por debajo del 0,03 % para $d \geq 50$. Este comportamiento dispar por tipo condiciona la aceleración real que se analiza a continuación.

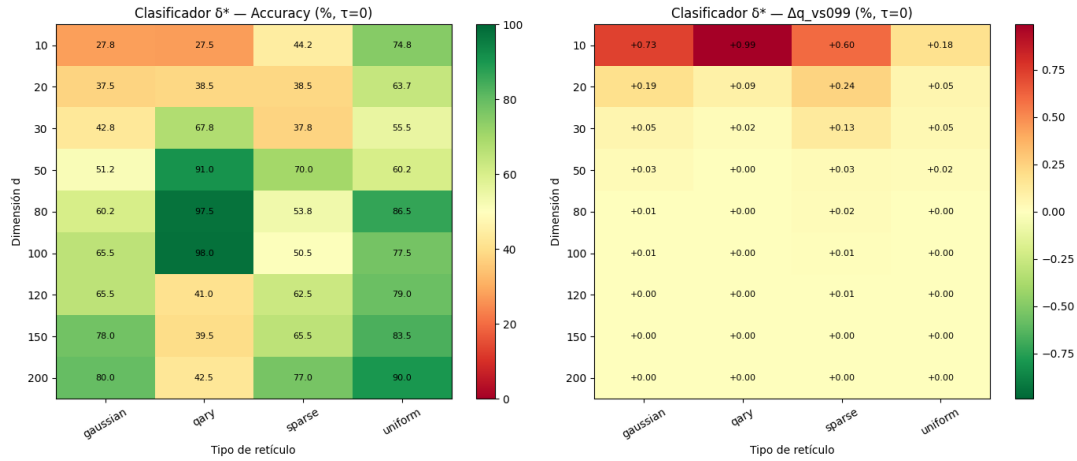


Fig. 4.16. Mapa de calor de la exactitud (izquierda) y la degradación media Δq_{vs099} (derecha) del clasificador δ^* por dimensión d y tipo de retículo ($\tau = 0$).

Validación temporal en retículos frescos. Para confirmar que la mejora predicha se traduce en ahorro de cómputo real, se generaron bases nuevas, no empleadas en ninguna fase de entrenamiento, para $d \in \{50, 80, 100, 120\}$, con 30 bases por par (d , tipo) y 5 repeticiones de LLL por base, reportando la mediana y el rango intercuartílico (IQR) del tiempo de ejecución real frente a la ejecución con $\delta = 0,99$.

El ahorro observado es modesto y fuertemente dependiente de la dimensión (figura 4.17). Para $d \in \{50, 80, 100\}$ las cuatro familias se mantienen muy próximas a la paridad (aceleraciones medianas entre $\approx 1,0\times$ y $\approx 1,09\times$, con ahorros de tiempo de un solo dígito porcentual e incluso ligeramente decrecientes hasta $d = 100$), de modo que en el rango intermedio el beneficio temporal del oráculo es marginal. El comportamiento cambia de forma acusada en $d = 120$: los retículos gaussiano, disperso y uniforme exhiben un salto nítido hasta aceleraciones de $\approx 1,23$ – $1,28\times$, equivalentes a un ahorro mediano del orden del 19–22 %, que constituye el máximo de todo el barrido. Los retículos q -arios, en cambio, no acompañan ese salto: su aceleración mediana permanece en $\approx 1,0\times$ en todas las dimensiones, y en $d = 120$ (pese a que el clasificador selecciona $\delta < 0,99$ en el 30 % de las bases) el ahorro mediano sigue siendo nulo y el IQR se dispara, abarcando desde ralentizaciones marcadas hasta aceleraciones cercanas a $1,8\times$. Sobre estas bases fuertemente estructuradas, por tanto, la elección de un δ menor no produce un ahorro fiable: el tiempo de LLL está dominado por factores ajenos al parámetro de reducción y la señal de ahorro queda enmascarada por el ruido de ejecución.

En todos los casos la degradación mediana del RHF es indistinguible de cero, de modo que el ahorro de tiempo, allí donde existe, se obtiene sin coste apreciable de calidad. Conviene distinguir esta aceleración selectiva del simple hecho de que LLL con $\delta = 0,55$ se ejecute más rápido que con $\delta = 0,99$: una estrategia que fijara $\delta = 0,55$ de forma indiscriminada alcanzaría aceleraciones mayores, señaladamente sobre las familias más estructuradas, pero a costa de la degradación de calidad recogida en la tabla 4.7 para *siempre* $\delta = 0,55$ (+1,08 % de media). El mérito del clasificador reside en que, en su

régimen selectivo ($\tau = 0$), reserva $\delta < 0,99$ para las bases en las que existe holgura y conserva $\delta = 0,99$ donde no la hay (señaladamente sobre los q -arios, donde apenas la selecciona y sin efecto temporal apreciable), renunciando a esa aceleración espuria a cambio de no comprometer la calidad de la reducción.

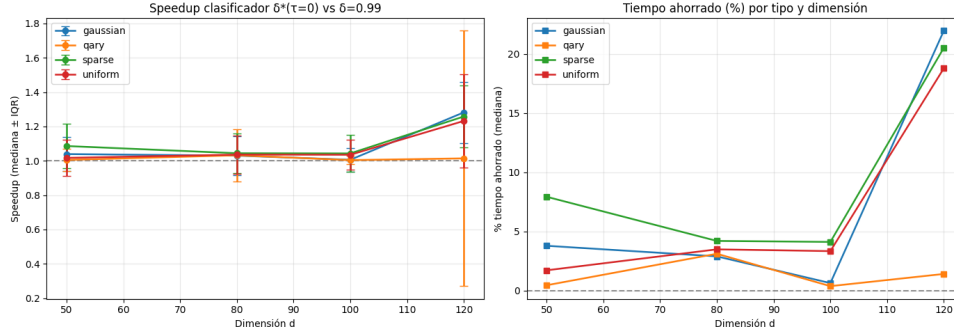


Fig. 4.17. Aceleración (mediana $\pm$ IQR, izquierda) y porcentaje de tiempo ahorrado (mediana, derecha) del clasificador δ^* frente a $\delta = 0,99$ sobre retículos frescos, por tipo y dimensión.

4.4.4. Limitaciones y trabajo futuro

A pesar de las mejoras que la reformulación como clasificación supervisada aporta sobre la regla de umbral inicial, el oráculo definitivo presenta limitaciones que delimitan su aplicabilidad práctica y que conviene explicitar antes de proyectar sus extensiones.

En primer lugar, el techo de precisión del clasificador lo fija la propia estructura del problema y no la capacidad del modelo. Como las diferencias de calidad entre los valores de δ son del orden del 1 % de media (la misma escasa varianza intra-base del RHF diagnosticada en la Sección 4.4.1) ni siquiera un clasificador perfecto dispondría de mucha más señal que explotar. Ello se manifiesta en la estructura de los errores, concentrados en la confusión entre las clases adyacentes $\delta = 0,90$ y $\delta = 0,99$ (Figura 4.15); que la tolerancia *off-by-one* alcance el 89,9 % acota la degradación incluso cuando el clasificador yerra, pero ninguna mejora del modelo podría ensanchar una señal que es intrínsecamente estrecha.

En segundo lugar, la transferibilidad del oráculo entre distribuciones distintas no ha sido evaluada. El clasificador se entrena con un modelo independiente por cada par dimensión–tolerancia (d, τ), que recibe el tipo de retículo como una característica codificada *one-hot* (Sección 3.4.2) y se ajusta exclusivamente sobre las cuatro familias generadas para este estudio. No puede garantizarse, por tanto, sin experimentación adicional, que la decisión aprendida se mantenga sobre distribuciones no observadas durante el entrenamiento (señaladamente sobre retículos dotados de estructura algebraica, como los *NTRU*), cuya dinámica de reducción difiere de la de las familias empleadas. La extensión correspondiente se desarrolla en la Sección 6.2.

En tercer lugar, el rango dimensional estudiado se acota a $d \leq 200$, muy por debajo de las dimensiones criptográficamente relevantes. El diseño por dimensión obliga, además,

a disponer de un modelo entrenado para cada d , de modo que su aplicación fuera de la rejilla considerada exige regenerar el conjunto etiquetado (cuyo coste, dominado por la ejecución de LLL sobre cada base, crece de forma acusada con la dimensión) o recurrir a un modelo menos especializado. A ello se añade que, en el extremo superior del rango, el R^2 deja de ser informativo por la varianza casi nula del *target* y la validación debe apoyarse en el error absoluto, tal como se observó en los retículos q -arios (Sección 4.3).

En cuarto lugar, el conjunto de datos no incorpora ninguna medida del coste computacional de LLL como variable objetivo, de manera que el oráculo aproxima el compromiso calidad–coste de forma *indirecta*, a través de la elección de δ , y el ahorro de tiempo real solo puede validarse empíricamente sobre bases frescas. Dicho ahorro resulta, además, modesto y fuertemente dependiente del tipo y la dimensión: nulo sobre los retículos q -arios (donde el clasificador conserva $\delta = 0,99$) y limitado a aceleraciones medianas de hasta $\approx 1,3\times$ sobre las demás familias en $d = 120$ (Figura 4.17), sin que la selección produzca una mejora temporal fiable en el rango intermedio.

Estas limitaciones perfilan con nitidez la línea de mejora más inmediata del oráculo: incorporar el número de operaciones de intercambio (*swaps*) de LLL como característica o como variable objetivo adicional del conjunto de datos. Disponer de un *proxy* directo del coste de reducción permitiría entrenar un oráculo que optimizase de forma explícita el compromiso calidad–coste, en lugar de aproximararlo mediante δ , y dotaría de fundamento cuantitativo a la interpretación del ahorro temporal observado. Esta y otras extensiones de mayor alcance (la generalización a BKZ, el escalado a dimensiones superiores y el estudio de transferibilidad) se desarrollan de forma sistemática en la Sección 6.2.

4.5. Discusión de resultados

Elección del modelo para el oráculo

A la vista de los resultados anteriores, se adoptan los modelos XGBoost por dimensión como componente del oráculo de selección adaptativa de δ . Esta elección está justificada por tres razones: (i) son competitivos o superiores al MLP en todo el rango (diferencias $< 0,01$) y claramente superiores en alta dimensión, donde el MLP diverge; (ii) son robustos en el régimen de baja varianza de *target* ($d \geq 100$), que corresponde precisamente al rango criptográficamente relevante; y (iii) no requieren ajuste de hiperparámetros de optimización (tasa de aprendizaje, número de épocas), lo que reduce el riesgo de convergencia inadecuada. El modelo global se conserva como *fallback* para la inferencia sobre dimensiones no contempladas en el entrenamiento.

Qué aprende realmente el sistema

El hilo conductor que recorre los resultados de las Secciones 4.1–4.3 es que los modelos no reconstruyen las relaciones algebraicas que definen cada métrica de calidad, sino

que aprenden *proxies* estructurales más informativos de la geometría de la base. El análisis SHAP lo evidencia de forma sistemática: las tres hipótesis de partida sobre la *feature* dominante quedan refutadas o confirmadas solo en parte (Tabla 4.6). El defecto de ortogonalidad no se explica por $\log |\det(\mathbf{B})|$ ni por las normas, los términos explícitos de su definición, sino por la fracción de coeficientes de Gram-Schmidt con $|\mu_{i,j}| > 0,5$, es decir, por una medida directa de cuán lejos se halla la base de entrada de la condición de *size-reduction* que el algoritmo LLL impone [28]. La log-norma del primer vector reducido no queda gobernada por la ley volumétrica de la heurística Gaussiana, sino por el parámetro δ y por $\min_i \|\mathbf{b}_i\|$, esto es, por la agresividad de la reducción y por la cota inferior que un vector ya corto impone al resultado.

La lectura conjunta es que el sistema funciona como un *refinamiento dependiente de la instancia* de la teoría asintótica de reducción de retículos. Donde la heurística Gaussiana ofrece una estimación promedio que ignora la estructura concreta de cada base, y donde las cotas del peor caso resultan excesivamente conservadoras (Sección 2.5.4), los modelos capturan la desviación de cada instancia respecto a ese comportamiento medio a partir de descriptores geométricos plausibles. Que las predicciones se apoyen en magnitudes interpretables (y no en artefactos del conjunto de datos, como la indicadora *q*-aria que la importancia de *ganancia* sobrepondera de forma espuria) es un resultado en sí mismo: respalda la validez del enfoque más allá de su mera precisión numérica y sugiere que lo aprendido tiene un correlato geométrico transferible al razonamiento sobre la reducción.

Dos lecturas de la capacidad predictiva

La fiabilidad de los modelos admite una doble lectura que conviene explicitar para no sobreinterpretar las cifras agregadas. En su modalidad por dimensión, XGBoost alcanza $R^2 > 0,99$ para el RHF a partir de $d \geq 100$ (Tabla 4.2). La superioridad sobre el modelo global no debe leerse a partir del salto frente a su $R^2 \approx 0,75$ agregado (medido sobre el test con todas las dimensiones mezcladas, cuya mayor varianza infla la diferencia), sino sobre las mismas bases: evaluados ambos modelos sobre idénticas instancias (Tabla 4.3), su R^2 converge en dimensiones altas mientras que el error absoluto del modelo por dimensión se mantiene entre aproximadamente una cuarta parte y la mitad por debajo. Esa reducción del error, y no la diferencia de R^2 agregado, valida la hipótesis de especialización: la heterogeneidad inter-dimensional era, en efecto, la principal fuente de error del predictor único. Sin embargo, el desglose intra-tipo (Tabla 4.5) matiza de forma sustancial ese optimismo. Una vez fijados dimensión y tipo, el R^2 del RHF desciende al rango 0,59–0,73 para los retículos gaussianos, dispersos y uniformes: buena parte del R^2 agregado proviene de la varianza *entre* tipos, que las columnas *one-hot* explican con holgura, mientras que la señal verdaderamente fina, la que distingue bases de un mismo tipo y dimensión, es apreciablemente más difícil de capturar.

Esta observación obliga, además, a leer con cautela el propio coeficiente de determinación en el régimen criptográficamente relevante. Al converger el RHF a su valor teórico

en dimensiones altas, su varianza tiende a cero y el R^2 , que normaliza el error por dicha varianza, se vuelve inestable o incluso queda indefinido, como ocurre con los retículos q -arios para $d \geq 150$. El error absoluto desmiente cualquier alarma: el MAE de esas predicciones se mantiene del orden de 10^{-5} , de modo que el modelo es extremadamente preciso en términos absolutos justo donde la métrica relativa deja de ser informativa. La conclusión metodológica es que la evaluación de estos predictores no puede descansar en una sola figura de mérito: el R^2 resume bien la modalidad global, pero en el límite de varianza nula debe complementarse necesariamente con el error absoluto para no extraer conclusiones erróneas.

Alcance y límites de la selección adaptativa

El recorrido del oráculo, desde su formulación inicial hasta la versión adoptada, constituye en sí mismo un hallazgo metodológico antes que una mera secuencia de ajustes. El fracaso de la regresión absoluta del RHF (que degeneraba a predecir $\delta = 0,55$ y alcanzaba un 8,1 % de exactitud, muy por debajo del 53,0 % del *baseline* conservador) y el de su variante residual no obedecen a un defecto de implementación, sino a una propiedad estructural del problema: la varianza *intra-base* del RHF al variar δ es comparable o inferior al ruido del propio regresor, de manera que cualquier regla que compare predicciones opera sobre ruido. Reformular la selección como una clasificación directa de δ^* traslada el umbral del momento de inferencia al objetivo de entrenamiento y elude esa comparación de magnitudes indistinguibles. El resultado es un clasificador que supera a los cuatro *baselines* no triviales (59,8 % de exactitud frente a 53,0 %), selecciona un $\delta < 0,99$ en el 36,6 % de las bases con una degradación media de tan solo +0,123 % respecto del valor por defecto y, a diferencia de las formulaciones previas, responde de forma medible al parámetro τ (Sección 4.4, Figura 4.13).

Ahora bien, la misma propiedad que explica el fracaso inicial fija el techo del diseño definitivo. Como las diferencias de calidad entre valores de δ son del orden del 1 % de media, ni siquiera un clasificador perfecto dispondría de mucha más señal que explotar; de ahí que la estructura de errores se concentre en la confusión entre las clases adyacentes $\delta = 0,90$ y $\delta = 0,99$, con una tolerancia *off-by-one* del 89,9 % que acota la degradación incluso cuando el clasificador yerra. El verdadero valor del oráculo no es, por tanto, acertar el óptimo exacto, sino no perder calidad de forma apreciable mientras explota la holgura presente en casi la mitad de las bases, y hacerlo con un coste de inferencia despreciable. La validación temporal sobre retículos frescos confirma que esa holgura se traduce en un ahorro real, si bien modesto: en su régimen estricto el clasificador conserva $\delta = 0,99$ sobre los retículos q -arios (por lo que su tiempo de ejecución no cambia) y solo selecciona un $\delta < 0,99$ de forma parcial en los demás tipos, con aceleraciones moderadas de hasta $\approx 1,3\times$ y sin coste apreciable de calidad (Figura 4.17). Las limitaciones que delimitan esta aplicabilidad (el techo de precisión por baja varianza, la falta de garantía de transferibilidad entre tipos, la escasez de datos en dimensiones muy altas y la ausencia de una

medida directa del coste de LLL como objetivo) se han analizado de forma específica en la Sección 4.4.4.

Posicionamiento frente al estado del arte

Situados estos resultados frente a la literatura revisada en el estado del arte, su aportación se delimita con precisión por contraste. La familia de ataques SALSA [38], [39] emplea el aprendizaje automático para *recuperar* secretos LWE, y sus propios autores identifican la reducción de bases del preprocesado como el principal cuello de botella del método (Sección 2.5.1). El presente trabajo no compite con esa línea, sino que aborda la magnitud que la condiciona: la calidad de reducción alcanzable, que aquí se *predice* sin ejecutar el algoritmo. De modo análogo, el trabajo de Marchetti *et al.* [25] utiliza redes neuronales para *ejecutar* la reducción y no logra superar a LLL de forma consistente al crecer la dimensión, mientras que esta memoria orienta el aprendizaje a *predecir* el resultado de un algoritmo clásico maduro (Sección 2.5.2). La distinción entre ejecutar y predecir es la que define la originalidad del enfoque: hasta donde alcanza la revisión realizada, ninguno de los trabajos previos aborda la predicción de la calidad de reducción a partir de la base sin reducir (Sección 2.5.4).

Esta delimitación obliga también a ser honesto sobre el alcance de lo conseguido. Los resultados se circunscriben al algoritmo LLL y a las distribuciones de retículos generadas para este estudio; no constituyen un ataque criptoanalítico completo, sino una prueba de viabilidad de la predicción de calidad como herramienta auxiliar. La extensión a BKZ (más relevante para los niveles de seguridad estandarizados) y la evaluación de la transferibilidad entre distribuciones quedan fuera del alcance actual. Aun así, la viabilidad demostrada es coherente con el marco conceptual de Laarhoven [21], que legitima la incorporación de técnicas de inteligencia computacional al criptoanálisis de retículos, y abre una vía complementaria a las dos grandes líneas de aplicación del aprendizaje automático en este dominio.

Implicaciones para la estimación de seguridad

El comportamiento de los retículos q -arios merece una lectura final por su relación (si bien indirecta y cualitativa, dada la naturaleza simplificada de la familia generada (Sección 3.2.1)) con la clase de problemas LWE que sustenta los estándares post-cuánticos basados en retículos recientemente aprobados [2], [3]. Precisamente sobre estas bases, las más regulares del conjunto experimental, el sistema alcanza su mejor precisión absoluta en la predicción del RHF, con un MAE del orden de 10^{-5} , justo donde la varianza casi nula del *target* vuelve inestable al R^2 (Sección 4.3). Conviene matizar, no obstante, que esa fortaleza reside en la *predicción* de la calidad y no en la *aceleración* de la reducción: sobre los retículos q -arios la selección adaptativa no halla holgura que explotar, conserva $\delta = 0,99$ y no produce un ahorro temporal apreciable (Sección 4.4.3, Figura 4.17). La

capacidad de anticipar la calidad de reducción de una clase de retículos a un coste despreciable encaja con la función de las herramientas de calibración de seguridad, del tipo del *lattice estimator* [36], que deben evaluar la dificultad efectiva de los ataques sobre un gran número de configuraciones de parámetros.

Esta misma utilidad encierra, no obstante, la ambivalencia característica de la investigación en criptoanálisis: las herramientas que permiten estimar con mayor finura la fortaleza de un esquema pueden, en principio, orientar también la selección de instancias sobre las que concentrar un ataque. Esta tensión de uso dual, junto con el encuadre normativo del trabajo, se aborda de forma específica en la Sección 5.3; baste señalar aquí que el destino natural de estos resultados es reforzar la calibración de la seguridad post-cuántica, una necesidad de alcance global para la infraestructura digital que ha de migrar a estos esquemas, antes que facilitar su vulneración.

5. PLANIFICACIÓN Y PRESUPUESTO

5.1. Planificación temporal

El desarrollo de este Trabajo Fin de Grado se extendió desde el 1 de febrero hasta su entrega el 12 de junio de 2026, esto es, algo más de cuatro meses de trabajo efectivo. Como asignatura de 12 ECTS del Grado en Ingeniería Informática, el trabajo lleva asociada una dedicación estimada de entre 300 y 360 horas de trabajo del estudiante; sobre esa base se planificó una dedicación media en torno a las 350 horas, cuyo desglose económico se detalla en la sección de presupuesto. La distribución concreta de esas horas entre las distintas actividades del proyecto se resume en el diagrama de Gantt de la Figura 5.1 y se justifica en los párrafos siguientes.

La planificación se articuló siguiendo un enfoque *iterativo e incremental* en lugar de un modelo en cascada estricto. Esta elección no es accidental, sino que responde a la naturaleza misma del problema abordado: el sistema se construyó como una sucesión de incrementos en los que cada familia de modelos se apoya en los hallazgos del incremento anterior. Así, los modelos globales (Sección 3.2 y siguientes) precedieron a los modelos por dimensión, y la observación del comportamiento de estos motivó a su vez el diseño del oráculo de selección adaptativa del parámetro δ (Sección 3.4). Cada iteración cerraba un ciclo de diseño, implementación y evaluación cuyos resultados realimentaban las decisiones de la siguiente, de modo que la frontera entre fases es deliberadamente difusa y varias actividades se solapan en el tiempo. La redacción de la memoria, en particular, se mantuvo activa en paralelo al desarrollo desde la mitad del proyecto, con objeto de documentar las decisiones de diseño mientras el contexto permanecía fresco y de reducir el riesgo de concentrar la escritura en las últimas semanas, especialmente crítico dado el carácter improrrogable de la fecha de entrega.

El trabajo se organizó en seis fases, que se describen a continuación junto con su ubicación temporal aproximada y sus principales entregables:

1. La revisión bibliográfica y el estudio del estado del arte (1 de febrero a principios de marzo) abarcaron los fundamentos de la criptografía basada en retículos, de los problemas computacionales asociados (SVP, CVP, LWE) y de los algoritmos de reducción de bases (LLL y BKZ), así como de la literatura sobre aplicación de aprendizaje automático al criptoanálisis de retículos. Esta fase fijó el marco teórico del Capítulo 2 y delimitó el alcance del problema de ingeniería a resolver.
2. El diseño e implementación del *pipeline* de características (segunda quincena de febrero a finales de marzo) comprendió la implementación del módulo de extracción de las 25 características algebraicas y geométricas a partir del perfil GSO (`gen_features_and_target.py`), incluyendo la integración con `fpylll` como

oráculo experimental de la reducción LLL y la construcción de la estructura *multi-delta* del conjunto de datos.

3. La construcción del *dataset* y de los modelos globales (finales de marzo a mediados de abril) comprendió la generación reproducible y paralelizada del conjunto de datos *multi-delta* (224 000 filas) y entrenamiento de los primeros modelos globales de *Random Forest* y *gradient boosting* (`dim_models.ipynb`), que sirvieron de línea base para el resto del trabajo.
4. Los modelos por dimensión y el oráculo de δ (abril a mediados de mayo) comprendieron el entrenamiento de modelos especializados por dimensión (`dim_models.ipynb`), exploración de redes neuronales multicapa (`dim_mlps.ipynb`) y diseño, implementación y validación del oráculo de selección adaptativa del parámetro δ (`oracle.ipynb`), núcleo de la contribución del trabajo.
5. El análisis y las extensiones (mediados de abril a finales de mayo) abarcaron el estudio de la importancia de las características e interpretación criptográfica de los modelos, extensión de los experimentos a dimensiones altas y a retículos q -arios, y validación temporal del coste real de LLL frente al δ predicho. Los resultados de esta fase se recogen en el Capítulo 4.
6. La redacción de la memoria (finales de marzo a la entrega del 12 de junio) consistió en la documentación progresiva de la metodología y los resultados, revisión iterativa con el tutor, y maquetación final del documento en $\text{\LaTeX}$ conforme a la plantilla y al estilo IEEE de la titulación.

Los hitos que estructuraron el seguimiento del proyecto se recogen, de forma sintética, en la Tabla 5.1.

Hito	Descripción	Fecha aprox.
H1	Consolidación del <i>dataset</i> v2 (224 000 filas)	Finales de marzo
H2	Primeros modelos globales evaluados	Mediados de abril
H3	Oráculo de selección de δ operativo	Mediados de mayo
H4	Borrador completo de la memoria	Principios de junio
H5	Entrega del TFG	12 de junio de 2026

TABLA 5.1. HITOS PRINCIPALES DEL PROYECTO.

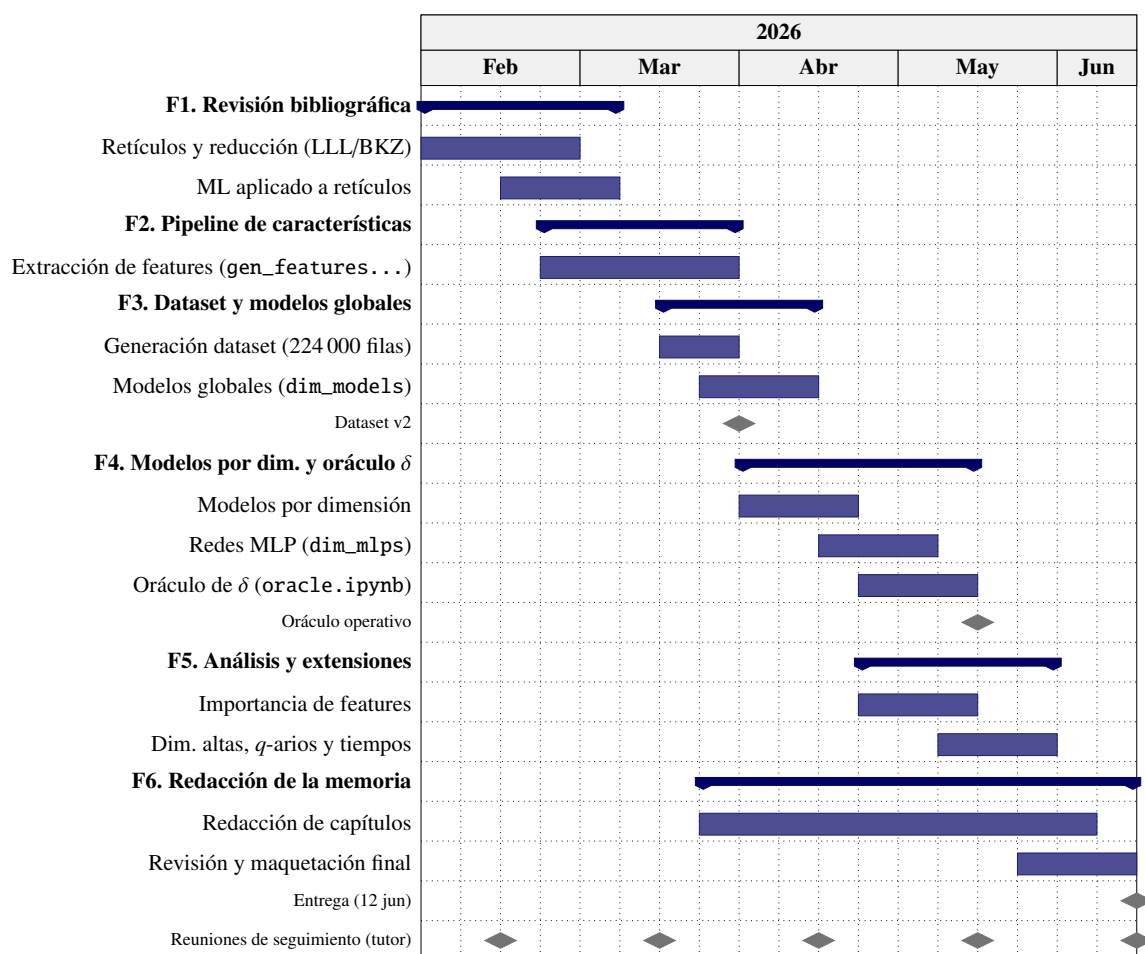


Fig. 5.1. Diagrama de Gantt de la planificación temporal del proyecto (1 de febrero a 12 de junio de 2026; el mes de junio aparece truncado en la fecha de entrega). Las barras representan actividades; los rombos, los hitos principales y las reuniones de seguimiento con el tutor. El solapamiento entre fases refleja el carácter iterativo e incremental del desarrollo.

La adopción de un modelo iterativo permitió absorber sin desviaciones críticas los principales riesgos identificados durante el desarrollo, factor especialmente relevante al disponer de una fecha de entrega fija e improrrogable. El coste computacional de la generación del conjunto de datos se contuvo mediante la paralelización determinista descrita en la Sección 3.2, lo que evitó que la Fase 3 se convirtiera en un cuello de botella. La variabilidad temporal observada al cronometrar la ejecución de LLL se gestionó incorporando, ya en la planificación de la Fase 5, repeticiones de medida y el uso de estadísticos robustos. Finalmente, la limitada transferibilidad de los modelos entre tipos de retículo se anticipó reservando tiempo específico para el reentrenamiento sobre retículos q -arios. Esta capacidad de reajuste dentro de cada iteración, frente a la rigidez de una planificación en cascada, fue determinante para completar el alcance previsto dentro del marco temporal disponible, comprendido entre el 1 de febrero y la entrega del 12 de junio de 2026.

5.2. Presupuesto

El presupuesto que se detalla a continuación cuantifica el coste de ejecución de este Trabajo Fin de Grado, basado en la imputación de costes directos (personal, amortización de equipos, licencias de software y otros costes directos) a los que se añade una partida de costes indirectos calculada como un porcentaje fijo de los primeros. La dedicación del personal se expresa en *hombres-mes* (HM), siguiendo la convención de equivalencia 1 HM = 131,25 horas. Por tratarse de un proyecto de investigación de carácter académico, sin finalidad de explotación comercial, no se aplican márgenes de riesgo ni de beneficio industrial; el Impuesto sobre el Valor Añadido (IVA) se consigna únicamente al final, a título informativo, sobre el coste total de ejecución.

Costes de personal. La partida más relevante corresponde al coste del personal involucrado. El autor del proyecto se imputa con la categoría de *Ingeniero*, a la tarifa de referencia de 2 694,39 €/HM, y con una dedicación de 350 horas, que es la prevista en la planificación temporal de la Sección 5.1 para una asignatura de 12 ECTS; dichas horas equivalen a $350/131,25 \approx 2,67$ HM. La labor de supervisión del tutor se imputa con la categoría de *Ingeniero Senior*, a 4 289,54 €/HM, y con una dedicación estimada de 25 horas ($\approx 0,19$ HM), coherente con las reuniones periódicas de seguimiento reflejadas en el diagrama de Gantt de la Figura 5.1 y con el tiempo asociado de revisión del trabajo. El desglose se recoge en la Tabla 5.2.

Rol	Categoría	Ded. (HM)	Coste/HM (€)	Coste (€)
Autor del TFG	Ingeniero	2,67	2.694,39	7.194,02
Tutor	Ingeniero Senior	0,19	4.289,54	815,01
Total costes de personal				8.009,03

TABLA 5.2. COSTES DE PERSONAL.

Amortización de equipos. El único recurso *hardware* empleado fue la estación de trabajo de sobremesa descrita en el capítulo de metodología (CPU AMD Ryzen 5 7600X, 32 GB de memoria DDR5 y tarjeta gráfica NVIDIA GeForce RTX 5070), cuyo coste de adquisición asciende a 1 600 €. Dado que el equipo no se consume íntegramente durante el proyecto, se imputa únicamente la fracción correspondiente a su amortización mediante la expresión

$$C_{\text{imp}} = \frac{M}{D} \cdot C \cdot u, \quad (5.1)$$

donde C es el coste de adquisición, M el número de meses de uso, D el periodo de depreciación y u el porcentaje de uso dedicado al proyecto. Se considera un periodo de depreciación de 48 meses (cuatro años), criterio habitual para equipos de proceso de información, un uso de 5 meses (los meses naturales de febrero a junio que abarca el proyecto)

y una dedicación del 100 %, al tratarse del equipo de cómputo destinado específicamente a la experimentación. El coste imputable resultante es de 166,67 € (Tabla 5.3).

Concepto	Coste (€)	Uso (meses)	Deprec. (meses)	Dedic. (%)	Imputable (€)
Estación de trabajo	1.600,00	5	48	100	166,67
Total amortización					166,67

TABLA 5.3. AMORTIZACIÓN DEL EQUIPO INFORMÁTICO.

Costes de software. La totalidad del instrumental software utilizado es de código abierto y acceso gratuito, circunstancia ya señalada en la Sección 2.6 y que, más allá de garantizar la reproducibilidad de los experimentos, elimina cualquier coste de licencia y cualquier restricción de propiedad intelectual sobre las herramientas. El lenguaje Python y las bibliotecas científicas (NumPy, pandas, Matplotlib) se distribuyen bajo licencias permisivas de tipo PSF y BSD; `scikit-learn` bajo licencia BSD; XGBoost bajo licencia Apache 2.0; PyTorch bajo licencia BSD; y `fp1ll/fpy1ll` bajo licencias LGPL/GPL. El entorno de edición (Visual Studio Code), el sistema de gestión de dependencias (Conda), la capa de virtualización (WSL2 con Ubuntu) y el sistema de composición tipográfica (T_EX Live/L^AT_EX) son asimismo gratuitos. El único componente propietario es el sistema operativo anfitrión Windows 11, cuya licencia OEM viene incluida en el precio del equipo y queda, por tanto, absorbida en la partida de amortización. En consecuencia, el coste de software imputable al proyecto es nulo, por lo que esta partida figura con valor cero en el presupuesto total (Tabla 5.4).

Otros costes directos. Se imputa como coste directo el consumo eléctrico asociado a la ejecución de los experimentos. Considerando una potencia media del sistema en torno a 0,2 kW a lo largo de las horas de trabajo activo, junto con las ejecuciones desatendidas de generación del *dataset* y de entrenamiento de los modelos, se estima un consumo total aproximado de 100 kWh. A un precio medio de la energía de 0,18 €/kWh, ello supone un coste de 18,00 €. La modestia de esta cifra es, en sí misma, un resultado relevante para el análisis del impacto ambiental del trabajo (Sección 5.3): el coste energético de entrenar los modelos predictivos resulta marginal frente al que conllevaría la ejecución masiva de algoritmos de reducción de retículos en dimensiones elevadas.

Costes indirectos y presupuesto total. A los costes directos se añade una partida de costes indirectos, fijada en el 20 % de aquellos, que cubre conceptos no imputables de forma directa pero generados en relación con el proyecto, tales como el espacio de trabajo, la conexión a Internet y los gastos administrativos generales. La agregación de todas las partidas conduce al presupuesto total de ejecución que recoge la Tabla 5.4: el coste del proyecto asciende a 9 832,44 € sin impuestos. A efectos informativos, la aplicación del IVA al tipo general del 21 % elevaría la cifra hasta 11 897,25 €.

Concepto	Coste (€)
Costes de personal	8.009,03
Amortización de equipos	166,67
Licencias de software	0,00
Otros costes directos (electricidad)	18,00
Total costes directos	8.193,70
Costes indirectos (20 %)	1.638,74
Total (sin IVA)	9.832,44
IVA (21 %)	2.064,81
Total presupuesto	11.897,25

TABLA 5.4. PRESUPUESTO TOTAL DEL PROYECTO.

5.3. Impacto social y ambiental

El presente trabajo tiene un carácter eminentemente metodológico: no entrega un sistema desplegable, sino una técnica (la predicción mediante aprendizaje automático de la calidad de reducción de una base de retículo) y la evidencia empírica de su comportamiento. Conforme a los criterios de evaluación de la Escuela Politécnica Superior, el análisis de impacto de un trabajo de esta naturaleza ha de precisar en qué aplicaciones podría emplearse el resultado y qué efectos cabe esperar en su sector de aplicación, antes que postular un beneficio inmediato. Esta sección examina, en ese orden, la dimensión social, la ambiental y la del uso dual y la responsabilidad ética asociadas al trabajo. El marco normativo formal en el que se inscribe (protección de datos, Esquema Nacional de Seguridad y licencias de las herramientas empleadas) se desarrolla en la Sección 1.3, al que se remite para evitar duplicidades.

Impacto social. El sector natural de aplicación de este trabajo es la calibración de la seguridad de la criptografía post-cuántica. La amenaza que el cómputo cuántico representa para los criptosistemas de clave pública en uso [1] ha motivado un proceso de normalización y migración de alcance mundial, materializado en los estándares post-cuánticos basados en retículos FIPS 203 y 204 [2], [3] y en las hojas de ruta de transición promovidas por los organismos de estandarización [5], [24]. La seguridad de los esquemas basados en retículos que sustituirán a los actuales no se demuestra de forma cerrada, sino que se *calibra*: herramientas como el *lattice estimator* [36] estiman la dificultad efectiva de los mejores ataques conocidos sobre un número muy elevado de configuraciones de parámetros. En ese contexto, un predictor capaz de anticipar la calidad de reducción alcanzable sobre una clase de retículos a un coste despreciable, en lugar de ejecutar el algoritmo de reducción, contribuye a que dicha calibración pueda realizarse de forma más exhaustiva y económica. El impacto social es, por tanto, indirecto pero alineado con un bien público de primer orden: la robustez de la infraestructura digital (protocolos de transporte seguro, infraestructuras de clave pública y comunicaciones institucionales) que ha de migrar a

estos esquemas. Esta orientación conecta de forma directa con el Objetivo de Desarrollo Sostenible núm. 9 (industria, innovación e infraestructura resiliente) y, por su incidencia en la confidencialidad de las comunicaciones y en la confianza en las instituciones, con el núm. 16.

Impacto ambiental. La huella ambiental del trabajo admite una doble lectura. En cuanto a su coste directo, el consumo eléctrico de la generación del *dataset* y del entrenamiento y evaluación de los modelos se estimó en torno a 100 kWh (Tabla 5.4). Aplicando el factor de emisión del mix eléctrico peninsular sin garantías de origen publicado por el MITECO para el cálculo de la huella de carbono, del orden de 0,28 kg CO₂eq/kWh [41], dicho consumo equivale a unos 28 kg CO₂eq; bajo el mix real de generación reportado por la CNMC ($\approx 0,10$ kg CO₂eq/kWh) la cifra descendería a unos 10 kg CO₂eq. Se trata, en cualquiera de los dos supuestos, de un impacto marginal, comparable al de un consumo doméstico de pocos días.

La lectura más pertinente, sin embargo, no es la del coste absoluto del proyecto, sino la del balance energético que su resultado induce en el sector de aplicación. El entrenamiento de los modelos por dimensión es un coste *único* y la inferencia es prácticamente gratuita, mientras que el coste que se trata de evitar (la ejecución repetida de algoritmos de reducción de retículos [28], [35] en dimensiones elevadas durante un barrido de parámetros) crece de forma acusada con la dimensión. La evaluación del oráculo (§4.4) cuantifica esta holgura: la selección adaptativa del parámetro δ elige un $\delta < 0,99$ en el 36,6 % de las bases con una degradación media de calidad de tan solo el +0,123 % respecto del valor conservador por defecto. Conviene, no obstante, no sobreestimar el ahorro de tiempo asociado: en su régimen estricto el oráculo conserva $\delta = 0,99$ sobre los retículos q -arios y solo logra aceleraciones moderadas (de hasta $\approx 1,3\times$) sobre el resto de tipos, sin pérdida apreciable de calidad (Figura 4.17). El efecto ambiental neto es, en consecuencia, favorable pero acotado: la técnica permite recortar cómputo redundante en flujos de calibración intensivos, sin que ello suponga una reducción uniforme aplicable a cualquier escenario. En este sentido, el trabajo se inscribe en la línea de la computación energéticamente consciente, en coherencia con el espíritu de eficiencia de recursos que subyace a los Objetivos de Desarrollo Sostenible.

Uso dual y responsabilidad ética. La investigación en criptoanálisis presenta la ambivalencia característica de toda herramienta que mide la fortaleza de un sistema: el mismo conocimiento que permite calibrar con mayor finura la seguridad de un esquema puede, en principio, orientar la selección de instancias sobre las que concentrar un ataque. La literatura reciente que aplica aprendizaje automático al problema LWE (singularmente la familia SALSA [38], [39] y las aproximaciones neuronales [25]) ilustra esta tensión. Frente a ella, varios rasgos del presente trabajo acotan el riesgo de uso indebido. En primer lugar, su objetivo es *predecir* una magnitud (la calidad de reducción) y no *recuperar* un secreto: no constituye un ataque, sino un instrumento de estimación cuyo destino natural es de-

fensivo. En segundo lugar, los experimentos operan en dimensiones muy inferiores a las criptográficamente relevantes, por lo que no aportan ventaja práctica frente a esquemas en producción. En tercer lugar, el trabajo se apoya exclusivamente en algoritmos y bibliotecas de dominio público y documenta sus resultados de forma reproducible, en línea con el principio criptográfico de rechazo de la seguridad por oscuridad: el conocimiento que mejora la estimación de seguridad beneficia, antes que nada, a quien diseña y despliega los esquemas. La responsabilidad del investigador se concreta, así, en encuadrar el resultado dentro de su finalidad de calibración y en someterlo al marco normativo y deontológico descrito en la Sección 1.3.

Valoración global. En conjunto, el balance de impacto es positivo, si bien moderado y delimitado con precisión. La contribución social es real pero indirecta, en tanto que habilitadora de una calibración de seguridad más exhaustiva; el impacto ambiental directo es despreciable y el inducido es favorable aunque desigual entre tipos de retículo; y el riesgo de uso dual, intrínseco al campo, queda mitigado por la naturaleza predictiva del trabajo, su régimen dimensional y su transparencia. Más que un beneficio incondicional, lo que este análisis pone de manifiesto es la coherencia entre los objetivos del trabajo y un uso responsable de sus resultados.

6. CONCLUSIONES Y TRABAJO FUTURO

6.1. Conclusiones

Este Trabajo Fin de Grado se propuso diseñar, implementar y evaluar empíricamente un sistema de aprendizaje automático capaz de predecir la calidad de la reducción LLL de un retículo a partir únicamente de su base sin reducir, y de aprovechar esa capacidad para seleccionar de forma adaptativa el parámetro δ . Una vez completada la evaluación del Capítulo 4, procede contrastar cada uno de los objetivos específicos fijados en la Sección 1.2 con la evidencia obtenida.

El primer objetivo, construir un *pipeline* reproducible de generación de datos, se ha cumplido en su totalidad. El sistema desarrollado genera bases de cuatro familias estructurales (uniforme, gaussiana, *sparse* y *q*-aria) sobre nueve dimensiones, las reduce con cuatro valores de δ mediante `fpylll` [9] y conforma un conjunto etiquetado de 224 000 instancias (Sección 3.2). El uso de semillas fijadas y de una generación paralela determina garantiza que el conjunto de datos sea idéntico entre ejecuciones, condición necesaria para la trazabilidad y la verificación independiente de los resultados.

El segundo objetivo, definir una representación geométrica de la base sin reducir, también se alcanzó. Se diseñó y extrajo un vector de descriptores algebraicos y geométricos calculado exclusivamente sobre la base original, antes de aplicar reducción alguna, fundamentado en el perfil de Gram-Schmidt y en magnitudes de ortogonalidad (Sección 3.2.3). Este vector constituye una entrada de baja dimensionalidad y, como confirmó el análisis posterior, admite una interpretación criptográfica directa.

El tercer objetivo, establecer variables objetivo criptográficamente significativas, se concretó en tres métricas de calidad: el defecto de ortogonalidad, la norma del primer vector reducido y el *root Hermite factor* (RHF). Este último se adoptó como métrica rectora del sistema por tratarse de una medida normalizada e independiente de la dimensión [10], lo que la hace comparable entre las distintas escalas del conjunto de datos (Sección 3.2.4).

El cuarto objetivo (entrenar, comparar y validar distintas familias de modelos) se logró, si bien su consecución obligó a matizar el alcance de la métrica de evaluación. Bajo la estrategia global, el defecto y la log-norma del primer vector se predicen con fiabilidad muy elevada ($R^2 \approx 0,9997$ y $R^2 \approx 0,98$, respectivamente; Tabla 4.1), mientras que el RHF no supera $R^2 \approx 0,75$. La especialización por dimensión resuelve precisamente esta limitación: para el RHF alcanza $R^2 \approx 0,97$ en $d = 50$, $R^2 \approx 0,99$ en $d = 80$ y $R^2 > 0,99$ a partir de $d \geq 100$ (Tabla 4.2). Comparados ambos modelos sobre las mismas bases de test, su R^2 es equiparable en dimensiones altas, pero el error absoluto del modelo por dimensión es entre aproximadamente una cuarta parte y la mitad menor (Tabla 4.3). Este resultado confirma que la heterogeneidad inter-dimensional, y no una incapacidad intrín-

seca del modelo, era la fuente dominante de error para el RHF. En la comparativa entre paradigmas, los modelos de *gradient boosting* [11] se mostraron competitivos o superiores al perceptrón multicapa (diferencias $< 0,01$) y claramente superiores en dimensiones altas, donde el MLP diverge, por lo que se adoptaron como componente predictivo del sistema, reservando el modelo global como *fallback* para dimensiones no contempladas en el entrenamiento.

El quinto objetivo, diseñar y evaluar un oráculo de selección adaptativa de δ , se cumplió, y su consecución constituye la aportación aplicada central del trabajo. El recorrido que condujo a la versión definitiva es, en sí mismo, un hallazgo metodológico. La formulación inicial, basada en la regresión absoluta del RHF y una regla de umbral, fracasó: degeneraba a predecir sistemáticamente $\delta = 0,55$ y alcanzaba apenas un 8,1 % de exactitud, muy por debajo del 53,0 % de la estrategia trivial *siempre* $\delta = 0,99$. El diagnóstico reveló que la causa no era un defecto de implementación, sino una propiedad estructural del problema: la varianza del RHF *dentro* de una misma base al variar δ es comparable o inferior al ruido del propio regresor, de modo que cualquier regla que compare predicciones opera sobre ruido. Reformular la selección como una clasificación supervisada directa de δ^* trasladó el umbral del momento de inferencia al objetivo de entrenamiento y eludió esa comparación de magnitudes indistinguibles (Sección 4.4). El clasificador resultante supera a los baselines no triviales (59,8 % de exactitud frente al 53,0 %), selecciona un $\delta < 0,99$ en el 36,6 % de las bases con una degradación media de calidad de tan solo +0,123 % respecto del valor por defecto, y acota su error mediante una tolerancia *off-by-one* del 89,9 % que confina las confusiones a clases de δ adyacentes. La validación sobre retículos frescos confirmó que esta holgura se traduce en un ahorro real de tiempo, si bien modesto: en su régimen estricto el oráculo conserva $\delta = 0,99$ sobre los retículos q -arios y obtiene aceleraciones moderadas (de hasta $\approx 1,3\times$) sobre el resto de tipos, sin coste apreciable de calidad (Figura 4.17).

El sexto objetivo, interpretar el comportamiento del sistema en clave de seguridad, se abordó mediante técnicas de atribución basadas en valores de Shapley (Sección 4.3). El hallazgo transversal es que los modelos no reconstruyen las relaciones algebraicas que definen cada métrica, sino que aprenden *proxies* estructurales más informativos de la geometría de la base: las tres hipótesis de partida sobre la característica dominante quedaron refutadas o confirmadas solo en parte (Tabla 4.6). Para el defecto, el predictor dominante resultó ser la fracción de coeficientes de Gram-Schmidt con $|\mu_{i,j}| > 0,5$, un indicador directo de cuán lejos está la base de cumplir la condición de *size-reduction* de LLL [28], y no la descomposición de su propia definición. Para la norma del primer vector reducido, el modelo relegó $\log|\det(\mathbf{B})|$ a una posición secundaria, refutando la forma fuerte de la heurística Gaussiana y aprendiendo, en su lugar, un refinamiento dependiente de la instancia gobernado por δ y por la norma mínima de entrada. El sistema, por tanto, no se limita a memorizar las leyes asintóticas conocidas, sino que captura los factores estructurales que determinan la desviación de cada base respecto a ellas.

En conjunto, los resultados sostienen la tesis de viabilidad que motivó el trabajo: el

aprendizaje automático puede actuar como un predictor rápido y preciso de la calidad de la reducción LLL, sensible a la geometría concreta de cada instancia y con un coste de inferencia despreciable frente a la ejecución directa del algoritmo. Con ello se ofrece una respuesta concreta al compromiso entre precisión, sensibilidad a la instancia y coste computacional identificado como problema abierto en la Sección 1.1. El sistema no sustituye a los métodos consolidados de estimación de seguridad post-cuántica, sino que los complementa como instrumento de calibración rápida; y sus límites quedan bien delimitados, pues la pequeña varianza intrínseca del RHF acota el techo de precisión alcanzable y restringe el margen real de la selección adaptativa al subconjunto de bases que toleran un δ inferior al conservador.

Por último, el trabajo satisface el objetivo transversal de carácter profesional que lo acompaña. El sistema desarrollado es de naturaleza *predictiva*, orientado a estimar una magnitud de calidad y no a recuperar un secreto, por lo que su destino natural es defensivo; opera, además, en dimensiones muy inferiores a las criptográficamente relevantes y se apoya exclusivamente en algoritmos y bibliotecas de dominio público, documentando sus resultados de forma reproducible en coherencia con el principio de rechazo de la seguridad por oscuridad. Encuadrado en el contexto de la migración hacia la criptografía post-cuántica y sometido al marco normativo y deontológico descrito en la Sección 1.3, el riesgo de uso dual, intrínseco a todo trabajo de criptoanálisis, queda razonablemente mitigado, de modo que el balance de impacto del proyecto resulta positivo, si bien moderado y delimitado con precisión.

6.2. Trabajo Futuro

El alcance de este trabajo se acotó de forma deliberada al algoritmo LLL, a nueve dimensiones no superiores a $d = 200$ y a las cuatro familias de retículos generadas para el estudio (Sección 6.1). Esa delimitación, lejos de cerrar el problema, perfila con nitidez las extensiones que lo prolongan de manera natural: unas amplían el rango operativo del sistema hacia los regímenes criptográficamente significativos; otras enriquecen la información que predice o la robustez con que lo hace; y la última proyecta el predictor sobre su destino aplicado como instrumento de calibración de seguridad. Las líneas que siguen no son un catálogo de posibilidades abstractas, sino consecuencias directas de los resultados y de las limitaciones identificadas en los Capítulos 4 y 6.

Extensión a BKZ y selección adaptativa del parámetro de bloque β . La extensión de mayor relevancia consiste en trasladar la metodología desde LLL hacia el algoritmo BKZ [35], que es el que gobierna los niveles de seguridad efectivamente estandarizados y el que constituye el cuello de botella del preprocesado en los ataques de aprendizaje automático al problema LWE (Sección 2.5.1) [39]. La arquitectura desarrollada se presta a esa extensión sin cambios conceptuales: del mismo modo que aquí se predice la calidad de la reducción LLL a partir de la base sin reducir, cabe predecir la calidad alcanzable por

BKZ y, sobre esa predicción, seleccionar de forma adaptativa el parámetro de bloque β . La reformulación del oráculo como una clasificación supervisada directa del parámetro óptimo, que resolvió el fracaso de la regla de umbral (Sección 4.4), es directamente transferible a un parámetro β discreto y acotado, lo que convierte la selección adaptativa de β en el análogo natural de la selección de δ abordada en esta memoria.

Escalado a dimensiones criptográficamente relevantes. Los experimentos se circunscriben a $d \leq 200$, un régimen muy inferior a las dimensiones de retículo de 512 y 1024 empleadas en la práctica [39] y a las que sustentan los esquemas estandarizados ML-KEM y ML-DSA [2], [3]. Escalar el estudio hacia esas dimensiones plantea dos tensiones complementarias. Por un lado, la convergencia asintótica del factor de Hermite hacia un valor límite [10] hace que el *target* se vuelva, en términos absolutos, progresivamente más sencillo de predecir, tal como anticipan los $R^2 > 0,99$ alcanzados en el extremo superior del rango estudiado (Tabla 4.2). Por otro, el coste de generar el conjunto etiquetado, que exige ejecutar la reducción sobre cada base, crece de forma acusada con la dimensión, y la evaluación debe apoyarse en el error absoluto allí donde el R^2 deja de ser informativo, como evidenció el colapso de la métrica en los retículos q -arios a dimensiones altas (Sección 4.3). El cuello de botella es, por tanto, la generación de datos y no la inferencia, lo que sugiere explorar la aceleración por GPU y bibliotecas de reducción paralelas o alternativas a `fpv111`.

Extensión a retículos estructurados: el caso de los retículos NTRU. La familia q -aria del conjunto experimental se genera a partir de una matriz $\mathbf{A}$ uniforme (Sección 3.2.1), es decir, en su variante *no estructurada*. Una extensión de claro interés criptográfico consiste en incorporar los *retículos NTRU* [32] como una quinta familia, presentada ya en la Sección 2.2.6: un caso particular de retículo q -ario dotado de estructura algebraica adicional, cuyas bases adoptan una forma de bloques construida sobre polinomios de un anillo cociente y exhiben así una periodicidad cíclica de la que carecen los q -arios uniformes. Estos retículos pertenecen a la familia de retículos estructurados sobre la que se asientan las variantes *Ring-LWE* [31] y *Module-LWE* y, con ellas, los esquemas post-cuánticos estandarizados [2], [3], por lo que su incorporación aproximaría el estudio a las instancias que aparecen en la práctica. El interés de esta línea, no es solo cuantitativo: los descriptores geométricos de este trabajo (Sección 3.2.3) se concibieron y validaron sobre familias densas y sobre el q -ario uniforme, de modo que queda abierta la pregunta de si conservan su poder predictivo ante una base cuya estructura algebraica no fueron diseñados para capturar. Los retículos NTRU, que exhiben bajo reducción un comportamiento característico, asociado a la presencia de un subretículo de vectores anormalmente cortos, ausente en los q -arios uniformes, constituyen además una prueba especialmente exigente de si el predictor capta dinámicas de reducción dependientes de la estructura, y no únicamente estadísticas geométricas genéricas. Por ello esta extensión se entrelaza con dos de las líneas ya enunciadas: con el estudio de transferibilidad entre distribuciones, pues evalua-

ría la generalización desde familias no estructuradas hacia otras estructuradas, y con el aprendizaje profundo geométrico, dado que la invariancia cíclica propia de los retículos NTRU es precisamente el tipo de simetría que una arquitectura con el sesgo inductivo adecuado [25] podría aprovechar y que los descriptores manuales solo aproximan.

De las métricas escalares al perfil GSO completo. El sistema predice tres magnitudes escalares de calidad (el defecto de ortogonalidad, la log-norma del primer vector reducido y el RHF), que resumen la reducción pero descartan información estructural. Una extensión de calado consiste en predecir el perfil de Gram-Schmidt completo, esto es, la secuencia $\{\log \|\mathbf{b}_i^*\|\}_{i=1}^d$, reformulando el problema como una regresión multisalida. El interés de esta línea es doble: el perfil GSO contiene una caracterización mucho más rica de la base reducida que cualquier estadístico agregado y es, además, la entrada que consumen los simuladores de BKZ y los estimadores concretos de seguridad, por lo que predecirlo conecta de manera directa con la línea de integración aplicada descrita más adelante.

Incorporación del coste de reducción como objetivo de aprendizaje. Tal como se señaló al delimitar las limitaciones del oráculo (Sección 4.4.4), el conjunto de datos no incluye ninguna medida del coste computacional de la reducción como variable objetivo, de modo que el oráculo aproxima el compromiso calidad–coste *indirectamente*, a través de la elección de δ , y el ahorro de tiempo de ejecución real solo se valida de forma empírica sobre bases frescas. Una mejora natural es incorporar el número de operaciones de intercambio (*swaps*) de LLL como característica o como variable objetivo adicional del conjunto de datos. Disponer de un *proxy* directo del coste permitiría entrenar un oráculo que optimice de forma explícita el compromiso entre calidad de reducción y coste de cómputo, en lugar de aproximarlos mediante la selección de δ , y permitiría interpretar con mayor rigor el ahorro de tiempo, modesto y dependiente del tipo de retículo, observado en la validación temporal (Figura 4.17).

Aprendizaje profundo geométrico e invariancia ante el cambio de base. La representación de la base empleada en este trabajo descansa en un vector de descriptores diseñados *a mano*, y en la comparativa entre paradigmas los conjuntos de árboles superaron o igualaron al perceptrón multicapa en todas las dimensiones y *targets* [11]. Una vía prometedora para capturar relaciones no lineales más complejas es el aprendizaje profundo geométrico, en la línea de Marchetti *et al.* [25], mediante redes que operen directamente sobre la matriz de Gram y que incorporen como sesgo inductivo las simetrías propias del problema, en particular la invariancia ante el cambio de base unimodular, una propiedad que los descriptores manuales solo aproximan. Conviene, no obstante, mantener la cautela que imponen los propios resultados: el modelo neuronal no superó a los conjuntos de árboles sobre datos tabulares, por lo que el beneficio de un enfoque profundo dependería de que las simetrías geométricas incorporadas aportaran una señal de la que las características agregadas carecen, y no debe presuponerse sin verificación empírica.

Transferibilidad entre distribuciones y robustez frente a bases adversariales. Los modelos por dimensión se entrenaron y evaluaron sobre las mismas familias estructurales de retículos, sin que se haya cuantificado su transferibilidad entre distribuciones distintas (Sección 4.4.4). Un estudio sistemático debería determinar en qué medida un modelo entrenado sobre una familia (por ejemplo, los retículos q -arios *planos*, los más afines a las instancias LWE no estructuradas) conserva su capacidad predictiva al aplicarse sobre otra, pues de esa transferibilidad depende la utilidad del sistema fuera de las condiciones controladas del estudio. A esta cuestión se añade la de la robustez frente a bases deliberadamente adversariales, construidas para inducir al predictor a sobrestimar o subestimar la calidad de reducción: en un instrumento concebido para la calibración de seguridad, la fiabilidad ante entradas manipuladas no es un detalle accesorio, sino una condición de su empleo responsable.

Integración en herramientas de estimación de seguridad. La culminación aplicada de estas líneas es la integración del predictor en las herramientas de estimación concreta de seguridad, del tipo del *lattice estimator* [36], que han de evaluar la dificultad efectiva de los ataques sobre un gran número de configuraciones de parámetros. Un predictor de coste de inferencia despreciable, máxime una vez extendido a BKZ y a la predicción del perfil GSO completo, encaja con naturalidad como módulo de estimación rápida dentro de ese flujo. Esta proyección refuerza, antes que diluye, la orientación *defensiva* del trabajo establecida en la Sección 1.3: su finalidad es calibrar con mayor finura la fortaleza de los esquemas post-cuánticos, y no facilitar su vulneración. A medida que las extensiones anteriores aproximen el sistema a los regímenes criptográficamente relevantes, el carácter de uso dual intrínseco al criptoanálisis [21] exigirá mantener vigente el encuadre normativo y deontológico descrito en dicha sección, de modo que cada avance en capacidad predictiva venga acompañado de la correspondiente reflexión sobre su uso responsable.

En conjunto, estas líneas comparten un hilo conductor: prolongar lo que esta memoria establece como una prueba de viabilidad, la predicción de la calidad de reducción a partir de la base sin reducir, hasta un instrumento desplegable de calibración defensiva de la seguridad, ampliando su alcance algorítmico y dimensional, enriqueciendo la información que predice y robusteciendo sus garantías, sin abandonar en ningún momento el marco de responsabilidad profesional que corresponde a un trabajo de esta naturaleza.

BIBLIOGRAFÍA

- [1] P. W. Shor, «Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,» *SIAM Journal on Computing*, vol. 26, n.º 5, pp. 1484-1509, 1997. doi: [10.1137/S0097539795293172](https://doi.org/10.1137/S0097539795293172)
- [2] National Institute of Standards and Technology, «FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM),» NIST, inf. téc., 2024. doi: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203)
- [3] National Institute of Standards and Technology, «FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA),» NIST, inf. téc., 2024. doi: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204)
- [4] National Institute of Standards and Technology, «FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA),» NIST, inf. téc., 2024. doi: [10.6028/NIST.FIPS.205](https://doi.org/10.6028/NIST.FIPS.205)
- [5] National Institute of Standards and Technology, «Post-Quantum Cryptography Standardization,» NIST, inf. téc., 2022. [En línea]. Disponible en: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [6] D. Micciancio y O. Regev, «Lattice-based Cryptography,» en *Post-Quantum Cryptography*, D. J. Bernstein, J. Buchmann y E. Dahmen, eds., Berlin, Heidelberg: Springer, 2009, pp. 147-191. doi: [10.1007/978-3-540-88702-7_5](https://doi.org/10.1007/978-3-540-88702-7_5)
- [7] O. Regev, «On lattices, learning with errors, random linear codes, and cryptography,» en *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC '05)*, 2005, pp. 84-93. doi: [10.1145/1060590.1060603](https://doi.org/10.1145/1060590.1060603)
- [8] C. Peikert, «A Decade of Lattice Cryptography,» *Found. Trends Theor. Comput. Sci.*, vol. 10, n.º 4, pp. 283-424, mar. de 2016. doi: [10.1561/04000000074](https://doi.org/10.1561/04000000074)
- [9] The FPLLL development team, *fpLLL, a lattice reduction library*, Software, versión 0.6.1, 2024. [En línea]. Disponible en: <https://github.com/fplll/fpylll>
- [10] N. Gama y P. Q. Nguyen, «Predicting Lattice Reduction,» en *Advances in Cryptology — EUROCRYPT 2008*, ép. Lecture Notes in Computer Science, vol. 4965, Springer, 2008, pp. 31-51. doi: [10.1007/978-3-540-78967-3_3](https://doi.org/10.1007/978-3-540-78967-3_3)
- [11] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» en *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785-794. doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)

- [12] S. M. Lundberg y S.-I. Lee, «A Unified Approach to Interpreting Model Predictions,» en *Advances in Neural Information Processing Systems*, I. Guyon et al., eds., vol. 30, Curran Associates, Inc., 2017. [En línea]. Disponible en: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd2Paper.pdf
- [13] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD)*, Diario Oficial de la Unión Europea, L 119/1, 2016. [En línea]. Disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>
- [14] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, Boletín Oficial del Estado, núm. 294, 2018. [En línea]. Disponible en: <https://www.boe.es/eli/es/lo/2018/12/05/3>
- [15] Gobierno de España, *Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad*, Boletín Oficial del Estado, núm. 106, 2022. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2022/05/03/311>
- [16] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2021/821 por el que se establece un régimen de la Unión de control de las exportaciones, el corretaje, la asistencia técnica, el tránsito y la transferencia de productos de doble uso (versión refundida)*, Diario Oficial de la Unión Europea, L 206/1, 2021. [En línea]. Disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32021R0821>
- [17] F. Pedregosa et al., «Scikit-learn: Machine learning in Python,» *Journal of Machine Learning Research*, vol. 12, n.º 85, pp. 2825-2830, 2011. [En línea]. Disponible en: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [18] A. Paszke et al., «PyTorch: An Imperative Style, High-Performance Deep Learning Library,» en *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox y R. Garnett, eds., vol. 32, Curran Associates, Inc., 2019. [En línea]. Disponible en: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
- [19] IEEE, *IEEE Code of Ethics*, Disponible en: <https://www.ieee.org/about/corporate/governance/p7-8.html>, 2020.
- [20] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, Disponible en: <https://www.acm.org/code-of-ethics>, 2018.
- [21] T. Laarhoven, «Evolutionary techniques in lattice sieving algorithms,» *arXiv pre-print arXiv:1907.04629*, 2019. [En línea]. Disponible en: <https://doi.org/10.48550/arXiv.1907.04629>

- [22] W. Diffie y M. E. Hellman, «New directions in cryptography,» *IEEE Transactions on Information Theory*, vol. 22, n.º 6, pp. 644-654, 1976. doi: [10.1109/TIT.1976.1055638](https://doi.org/10.1109/TIT.1976.1055638)
- [23] R. L. Rivest, A. Shamir y L. Adleman, «A method for obtaining digital signatures and public-key cryptosystems,» *Communications of the ACM*, vol. 21, n.º 2, pp. 120-126, 1978. doi: [10.1145/359340.359342](https://doi.org/10.1145/359340.359342)
- [24] European Telecommunications Standards Institute, «Quantum-Safe Cryptography (QSC); Quantum-Safe Threat Taxonomy,» ETSI, inf. téc. GR QSC 003 V1.1.1, 2016. [En línea]. Disponible en: https://www.etsi.org/deliver/etsi_gr/QSC/001_099/003/01.01.01_60/gr_QSC003v010101p.pdf
- [25] G. L. Marchetti, G. Cesa, P. Kumar y A. Behboodi, «Neural Lattice Reduction: A Self-Supervised Geometric Deep Learning Approach,» *Transactions on Machine Learning Research*, 2025. [En línea]. Disponible en: <https://openreview.net/forum?id=YxXyRSlZ4b>
- [26] X. Deng, *An Introduction to Lenstra-Lenstra-Lovász Lattice Basis Reduction Algorithm*, Undergraduate paper, Massachusetts Institute of Technology, 18.204: Seminar in Mathematical Analysis, 2016. [En línea]. Disponible en: https://math.mit.edu/~apost/courses/18.204-2016/18.204_Xinyue_Deng_final_paper.pdf
- [27] D. Micciancio, *The LLL Algorithm*, Notas de clase, CSE 206A: Lattice Algorithms and Applications, University of California, San Diego, 2012. [En línea]. Disponible en: <https://cseweb.ucsd.edu/classes/wi10/cse206a/lec3.pdf>
- [28] A. K. Lenstra, H. W. Lenstra y L. Lovász, «Factoring polynomials with rational coefficients,» *Mathematische Annalen*, vol. 261, n.º 4, pp. 515-534, 1982. doi: [10.1007/BF01457454](https://doi.org/10.1007/BF01457454)
- [29] P. Q. Nguyen, «Hermite's Constant and Lattice Algorithms,» en *The LLL Algorithm: Survey and Applications*, ép. Information Security and Cryptography, P. Q. Nguyen y B. Vallée, eds., Springer, 2009, pp. 19-69. doi: [10.1007/978-3-642-02295-1_2](https://doi.org/10.1007/978-3-642-02295-1_2)
- [30] C. P. Schnorr, «Lattice Reduction by Random Sampling and Birthday Methods,» en *STACS 2003: 20th Annual Symposium on Theoretical Aspects of Computer Science*, ép. Lecture Notes in Computer Science, vol. 2607, Springer, 2003, pp. 145-156. doi: [10.1007/3-540-36494-3_14](https://doi.org/10.1007/3-540-36494-3_14)
- [31] O. Regev, «The Learning with Errors Problem (Invited Survey),» pp. 191-204, 2010. doi: [10.1109/CCC.2010.26](https://doi.org/10.1109/CCC.2010.26)
- [32] J. Hoffstein, J. Pipher y J. H. Silverman, «NTRU: A Ring-Based Public Key Cryptosystem,» en *Algorithmic Number Theory (ANTS-III)*, ép. Lecture Notes in Computer Science, vol. 1423, Springer, 1998, pp. 267-288. doi: [10.1007/BFb0054868](https://doi.org/10.1007/BFb0054868)

- [33] S. Khot, «Hardness of approximating the shortest vector problem in lattices,» *Journal of the ACM*, vol. 52, n.º 5, pp. 789-808, 2005. doi: [10.1145/1089023.1089027](https://doi.org/10.1145/1089023.1089027)
- [34] M. Ajtai, «Generating hard instances of lattice problems (extended abstract),» en *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ép. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 99-108. doi: [10.1145/237814.237838](https://doi.org/10.1145/237814.237838)
- [35] Y. Chen y P. Q. Nguyen, «BKZ 2.0: Better lattice security estimates,» en *Advances in Cryptology — ASIACRYPT 2011*, ép. Lecture Notes in Computer Science, vol. 7073, Springer, 2011, pp. 1-20. doi: [10.1007/978-3-642-25385-0_1](https://doi.org/10.1007/978-3-642-25385-0_1)
- [36] M. R. Albrecht et al., *The Lattice Estimator*, Software, 2024. [En línea]. Disponible en: <https://github.com/malb/lattice-estimator>
- [37] A. Becker, L. Ducas, N. Gama y T. Laarhoven, «New directions in nearest neighbor searching with applications to lattice sieving,» en *Proceedings of the 27th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*, 2016, pp. 10-24. doi: [10.1137/1.9781611974331.ch2](https://doi.org/10.1137/1.9781611974331.ch2)
- [38] E. Wenger, M. Chen, F. Charton y K. E. Lauter, «SALSA: Attacking Lattice Cryptography with Transformers,» en *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho y A. Oh, eds., vol. 35, Curran Associates, Inc., 2022, pp. 34981-34994. [En línea]. Disponible en: https://proceedings.neurips.cc/paper_files/paper/2022/file/e28b3369186459f57c94a9ec9137fac9-Paper-Conference.pdf
- [39] S. Stevens et al., *SALSA FRESCA: Angular Embeddings and Pre-Training for ML Attacks on Learning With Errors*, arXiv preprint, 2024. arXiv: [2402.01082](https://arxiv.org/abs/2402.01082) [cs.CR]. [En línea]. Disponible en: <https://arxiv.org/abs/2402.01082>
- [40] M. R. Albrecht, R. Player y S. Scott, «On the concrete hardness of Learning with Errors,» *Journal of Mathematical Cryptology*, vol. 9, n.º 3, pp. 169-203, 2015. doi: [10.1515/jmc-2015-0016](https://doi.org/10.1515/jmc-2015-0016)
- [41] Ministerio para la Transición Ecológica y el Reto Demográfico, *Factores de emisión. Registro de huella de carbono, compensación y proyectos de absorción de dióxido de carbono*, Oficina Española de Cambio Climático (OECC), Consultado en 2026, 2025. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2025/03/18/214/con>

ANEXO A: CÓDIGO FUENTE PRINCIPAL

El desarrollo de este Trabajo de Fin de Grado se ha materializado en un conjunto de *scripts* de Python y cuadernos de Jupyter que implementan la generación del conjunto de datos, el entrenamiento y la evaluación de los modelos, y el análisis de interpretabilidad. Con el fin de garantizar la trazabilidad y la reproducibilidad de los experimentos, la totalidad del código fuente se encuentra disponible públicamente en el repositorio del proyecto, en la dirección:

https://github.com/fernandomartinarencibia/ml_lattice_delta_predictor_release

Por motivos de extensión, este anexo no reproduce el código de forma íntegra, sino una selección de fragmentos representativos de cada componente del sistema, elegidos por concentrar las decisiones de diseño y los aspectos metodológicos más relevantes. Cada fragmento se acompaña de una breve descripción de su función dentro del flujo de trabajo global. El código completo, incluyendo las dependencias, los *scripts* auxiliares y las instrucciones de ejecución, puede consultarse en el repositorio enlazado.

El módulo `lattice_utils.py` (Anexo A.2) fue desarrollado por el tutor del proyecto, Javier Blanco Romero, y se reutiliza en este trabajo como base para la generación de retículos. Los experimentos se apoyan asimismo en las bibliotecas `fpvlll` y `fp1lll`, distribuidas bajo licencias GPL-2.0 y LGPL-2.1 respectivamente, cuyos términos se respetan en el uso académico realizado.

La reproducción de los experimentos requiere un entorno `conda` con las dependencias especificadas en el fichero `environment.yml` del repositorio. Una vez clonado este, el entorno se crea y activa con las órdenes siguientes:

CÓDIGO 6.1. Creación del entorno de ejecución.

```
1 conda env create -f environment.yml
2 conda activate delta_predictor_release
```

Los experimentos deben ejecutarse en el orden indicado, pues cada etapa consume los artefactos generados por las anteriores. El conjunto de datos que sustenta los resultados presentados en esta memoria se distribuye congelado en el fichero comprimido `lattice_dataset_v2.zip`; la vía de ejecución recomendada consiste en descomprimirlo (`unzip lattice_dataset_v2.zip`) para obtener `lattice_dataset_v2.csv` (aproximadamente 224 000 filas), cuya integridad puede verificarse frente a la huella SHA-256 publicada en el repositorio. De forma alternativa, puede regenerarse otro conjunto de datos distinto al que contiene el zip ejecutando `python gen_features_and_target.py`, que lo produce de manera determinista gracias a la semilla maestra `DATASET_SEED = 42`.

Con el fichero de datos ya disponible, se ejecuta el cuaderno `dim_models.ipynb`, que entrena y serializa los regresores XGBoost por dimensión junto con el fichero de métricas `xgb_results.json`, requisito previo tanto para la comparativa con redes neuronales (`dim_mlps.ipynb`) como para el oráculo de selección de δ (`oracle.ipynb`). Estos dos últimos cuadernos pueden ejecutarse de forma independiente una vez disponibles los modelos por dimensión.

Debe precisarse que el conjunto de datos congelado se generó con anterioridad a la consolidación del esquema de semillas por tarea actualmente implementado en `gen_features_and_target.py`. En consecuencia, la regeneración del conjunto, aun siendo determinista y reproducible entre ejecuciones, produce un fichero distinto, si bien estadísticamente equivalente, del empleado en los experimentos. Por este motivo, la reproducción *exacta* de las cifras reportadas requiere emplear la vía recomendada (el conjunto congelado); el script de generación se aporta para garantizar la trazabilidad y la reproducibilidad del método de construcción del conjunto, y no la del fichero concreto.

La reproducibilidad de las restantes etapas queda garantizada por el uso de la semilla 42 en todas las particiones y procesos estocásticos (`random_state`, `np.random` y `torch.manual_seed`). Sobre el conjunto congelado, diferencias de hasta $\pm 0,01$ en el coeficiente R^2 entre ejecuciones se consideran tolerables y obedecen al no determinismo residual de las operaciones de punto flotante en GPU; en caso de regenerar el conjunto, las discrepancias pueden ser algo mayores por tratarse de un fichero distinto, aunque equivalente, sin que ello altere las conclusiones. El entrenamiento de los modelos XGBoost emplea aceleración por GPU mediante `device='cuda'`; en ausencia de hardware compatible con CUDA, basta con sustituir dicho valor por `device='cpu'` en los cuadernos correspondientes.

A.1 Extracción de características (`gen_features_and_target.py`)

El núcleo del proceso de generación del conjunto de datos es la función `extract_features`, que recibe una base reticular y devuelve el vector de características derivadas del perfil de Gram–Schmidt. Se reproduce a continuación una versión abreviada (el código completo está disponible en el repositorio del proyecto).

CÓDIGO 6.2. Extracción de características a partir del perfil GSO de una base reticular.

```
1 def extract_features(B):
2     M = GSO.Mat(B)
3     M.update_gso()
4     d = B.nrows
5     B_np = np.array([list(B[i]) for i in range(d)], dtype=float)
6
7     # get_log_det devuelve el log del determinante AL CUADRADO ->
8     # /2.0
9     log_det = M.get_log_det(0, d) / 2.0
```

```

9     norms_b = np.array([B[i].norm() for i in range(d)])
10
11     # get_r devuelve el CUADRADO de la norma GSO -> raiz para ||b_i
12     # ^*||
13     norms_b_star = np.sqrt(np.array([M.get_r(i, i) for i in range(d)
14     ])))
15     log_norms_b_star = np.log(norms_b_star)
16
17     # Pendiente del perfil GSO (ajuste lineal log||b_i^*|| vs. i)
18     indices = np.arange(d)
19     slope, intercept = np.polyfit(indices, log_norms_b_star, 1)
20     residuals = log_norms_b_star - (slope * indices + intercept)
21
22     # Defecto de ortogonalidad (en escala logaritmica)
23     log_orth_defect = np.sum(np.log(norms_b)) - log_det
24
25     # Coeficientes mu del triangulo inferior
26     mu_vals = np.array([abs(M.get_mu(i, j))
27     for i in range(1, d) for j in range(i)])
28
29     features = {
30         'd': d,
31         'log_det': log_det,
32         'norm_mean': np.mean(norms_b),
33         'norm_ratio': np.max(norms_b) / np.min(norms_b),
34         'gso_slope': slope,
35         'gso_res_std': np.std(residuals),
36         'log_orth_defect': log_orth_defect,
37         'mu_mean': np.mean(mu_vals),
38         'mu_frac_gt_05': np.sum(mu_vals > 0.5) / len(mu_vals),
39         # [...] resto de descriptores estadísticos y funcionales del
40         # perfil
41         'profile_entropy': profile_entropy(log_norms_b_star),
42     }
43     return features

```

A.2 Generadores de retículos (lattice_utils.py)

El módulo `lattice_utils.py`, desarrollado por el tutor del proyecto, reúne las utilidades compartidas para los experimentos de reducción. Su componente esencial son los cuatro generadores de familias de retículos empleados en la construcción del conjunto de datos. Cada generador reintentará hasta obtener una base de rango completo, salvo el caso q -ario, cuya inicialización es determinista por diseño (los retículos q -arios son de rango completo por construcción).

CÓDIGO 6.3. Generadores de las cuatro familias de retículos: uniforme, gaussiano, disperso y q -ario.

```

1  def gen_lattice_uniform(d, rng=10, max_attempts=500):
2      """Entradas ~ Uniforme({-rng, ..., rng})."""
3      for _ in range(max_attempts):
4          B = np.random.randint(-rng, rng + 1, (d, d))
5          if _is_nonsingular(B):
6              return B
7      raise ValueError(f"No se pudo generar retICulo no singular en d
      = {d}")
8
9
10 def gen_lattice_gaussian(d, sigma=5.0, max_attempts=500):
11     """Entradas ~ round(Normal(0, sigma))."""
12     for _ in range(max_attempts):
13         B = np.round(np.random.randn(d, d) * sigma).astype(int)
14         if _is_nonsingular(B):
15             return B
16     raise ValueError(f"No se pudo generar retICulo gaussiano en d={d
    }")
17
18
19 def gen_lattice_sparse(d, density=0.5, rng=10, max_attempts=500):
20     """RetICulo disperso (diagonal forzada no nula)."""
21     for _ in range(max_attempts):
22         B = np.random.randint(-rng, rng + 1, (d, d))
23         mask = np.random.rand(d, d) > density
24         B[mask] = 0
25         np.fill_diagonal(B, np.random.randint(1, rng + 1, d))
26         if _is_nonsingular(B):
27             return B
28     raise ValueError(f"No se pudo generar retICulo disperso en d={d}
    ")
29
30
31 def gen_lattice_qary(d, q=101):
32     """RetICulo q-ario de medio rango:  $[q \cdot I_k, 0; A, I_{d-k}]$ .  $\det = q^k$ ."""
33     k = d // 2
34     B = np.zeros((d, d), dtype=int)
35     np.fill_diagonal(B[:k, :k], q)
36     B[k:, :k] = np.random.randint(0, q, (d - k, k))
37     np.fill_diagonal(B[k:, k:], 1)
38     return B
39
40
41 GENERATORS = {
42     "uniform": gen_lattice_uniform,
43     "gaussian": gen_lattice_gaussian,
44     "sparse": gen_lattice_sparse,
45     "qary": gen_lattice_qary,
46 }

```

A.3 Partición canónica anti-fuga (split_utils.py)

Para evitar la fuga de información derivada del diseño multi- δ (cada base genera cuatro filas, una por valor de δ), la partición de entrenamiento/prueba se realiza agrupando por base, nunca por fila. El identificador `base_id` se reconstruye de forma determinista y el reparto se estratifica por $(d, \text{lattice_type})$ con semilla fija, garantizando reproducibilidad y representación de todos los estratos.

CÓDIGO 6.4. Reconstrucción del `base_id` y partición de prueba agrupada por base y estratificada por estrato.

```
1  def reconstruct_base_id(df: pd.DataFrame) -> pd.Series:
2      """base_id canonico = 'd|lattice_type|occ', alineado con las
        filas de df.
3
4      occ = cumcount() // 4 dentro de cada grupo (d, lattice_type),
5      replicando el criterio de dim_models / dim_mlps / oracle.
6      """
7      occ = df.groupby(['d', 'lattice_type']).cumcount() // 4
8      base_id = (df['d'].astype(str) + '|'
9                 + df['lattice_type'].astype(str) + '|'
10                + occ.astype(str))
11     return base_id
12
13
14  def build_test_base_ids(df: pd.DataFrame, test_size: float = 0.2,
15                          seed: int = 42) -> set:
16      """Asigna el test_size de las bases a test, estratificado por
17      (d, lattice_type). Evita estratos vacios o de varianza nula en
18      test.
19      """
20      base_id = reconstruct_base_id(df)
21      bases = (pd.DataFrame({'base_id': base_id,
22                             'd': df['d'].values,
23                             'lt': df['lattice_type'].values})
24                .drop_duplicates('base_id'))
25      rng = np.random.default_rng(seed)
26      test_ids = []
27      for (_d, _lt), grp in bases.groupby(['d', 'lt'], sort=True):
28          ids = rng.permutation(grp['base_id'].to_numpy())
29          n_test = round(test_size * len(ids))
30          test_ids.extend(ids[:n_test].tolist())
31      return set(test_ids)
```

A.4 Entrenamiento y evaluación de los regresores (`dim_models.ipynb`)

Los modelos de regresión se entrenan mediante la función `train_evaluate`, que ajusta las dos variantes basadas en XGBoost (bosque aleatorio (`XGBRFRegressor`) y *gradient boosting* (`XGBRegressor`)) sobre las mismas particiones. El escalado se ajusta únicamente con los datos de entrenamiento y se aplica al conjunto de prueba, evitando así cualquier fuga de información. Como métricas se reportan el coeficiente de determinación (R^2) y el error absoluto medio (MAE).

CÓDIGO 6.5. Función de entrenamiento y evaluación de las variantes XGBoost (RF y GB).

```
1  def train_evaluate(X_train, X_test, y_train, y_test, label=''):
2      scaler = StandardScaler()
3      # ajusta y transforma solo en train
4      X_tr = scaler.fit_transform(X_train)
5      # en test solo transforma, evitando data leakage
6      X_te = scaler.transform(X_test)
7
8      rf = XGBRFRegressor(
9          n_estimators=200, max_depth=15,
10         device='cuda', tree_method='hist',
11         subsample=0.8, colsample_bynode=0.8,
12         random_state=42
13     )
14     gb = XGBRegressor(
15         n_estimators=200, max_depth=5, learning_rate=0.1,
16         device='cuda', tree_method='hist',
17         subsample=0.8, colsample_bytree=0.8,
18         random_state=42
19     )
20
21     results = {}
22     for name, model in [('RF', rf), ('GB', gb)]:
23         model.fit(X_tr, y_train)
24         y_pred = model.predict(X_te)
25         results[name] = {
26             'r2': r2_score(y_test, y_pred),
27             'mae': mean_absolute_error(y_test, y_pred)
28         }
29         print(f" {label} | {name}: R_squared={results[name]['r2']:.4f} "
30               f"MAE={results[name]['mae']:.4f}")
31     return results, {'RF': rf, 'GB': gb}, scaler
```

A.5 Arquitectura del perceptrón multicapa (dim_mlps.ipynb)

Como alternativa a los modelos basados en árboles, se evalúa un perceptrón multicapa parametrizable, `ConditionedMLP`, implementado en PyTorch. Cada capa oculta sigue el patrón `Linear` $\rightarrow$ `BatchNorm` $\rightarrow$ `GELU` $\rightarrow$ `Dropout`: la normalización por lotes estabiliza el gradiente ante características de escalas muy dispares y `GELU` evita las neuronas muertas, preferible a `ReLU` en regresión continua. Los pesos se inicializan con el método de Kaiming y la capa de salida con Xavier.

CÓDIGO 6.6. Arquitectura del perceptrón multicapa parametrizable `ConditionedMLP`.

```
1 # Capas: Linear -> BatchNorm -> GELU -> Dropout.
2 # BatchNorm estabiliza gradientes ante features de escalas dispares.
3 # GELU evita neuronas muertas; preferible a ReLU en regresion
  continua.
4
5 class ConditionedMLP(nn.Module):
6     def __init__(self, n_features: int, hidden_dims: list,
7                   dropout_p: float = 0.1):
8         super().__init__()
9         layers = []
10        in_dim = n_features
11        for out_dim in hidden_dims:
12            layers += [
13                nn.Linear(in_dim, out_dim),
14                nn.BatchNorm1d(out_dim),
15                nn.GELU(),
16                nn.Dropout(p=dropout_p),
17            ]
18            in_dim = out_dim
19        layers.append(nn.Linear(in_dim, 1))
20        self.network = nn.Sequential(*layers)
21        self._init_weights()
22
23    def _init_weights(self):
24        for m in self.modules():
25            if isinstance(m, nn.Linear):
26                nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
27                nn.init.zeros_(m.bias)
28        last_linear = [m for m in self.modules()
29                        if isinstance(m, nn.Linear)][-1]
30        nn.init.xavier_normal_(last_linear.weight)
31
32    def forward(self, x: torch.Tensor) -> torch.Tensor:
33        return self.network(x)
```


A.6 Oráculo de selección de δ (oracle.ipynb)

El oráculo se formula como un problema de clasificación directa (Opción B): para cada base se determina el valor óptimo δ^* como el menor δ cuyo RHF no degrada más de una fracción τ respecto al mejor alcanzable. Se entrena un clasificador `XGBClassifier` independiente por cada par (d, τ) , incorporando el tipo de retículo como variable codificada en *one-hot*. Esta granularidad permite analizar la transición abrupta del comportamiento del oráculo en torno a $\tau \approx 0,01$.

CÓDIGO 6.7. Etiquetado de δ^* y entrenamiento de un clasificador por par (d, τ) .

```
1 def compute_delta_star(df_bases, tau):
2     """delta* = menor delta cuyo RHF no degrada mas de (1+tau)
3     respecto al mejor."""
4     results = []
5     for _, row in df_bases.iterrows():
6         ref = float(row['best_rhf'])
7         chosen = 0.99
8         for delta in [0.55, 0.75, 0.90, 0.99]:
9             if float(row[delta]) <= ref * (1 + tau):
10                 chosen = delta
11                 break
12         results.append(chosen)
13     return pd.Series(results, index=df_bases.index)
14
15 # Un clasificador independiente por cada par (d, tau);
16 # el tipo de reticulo va incluido en FEAT_CLF como one-hot.
17 for d in DIMENSIONS:
18     tr_d = df_train[df_train['d'] == d]
19     X_tr = tr_d[FEAT_CLF].values
20     for tau_tr in THRESHOLDS:
21         y_tr = compute_delta_star(tr_d, tau_tr).map(DELTA_TO_IDX).
22             values.astype(int)
23         scaler_c = StandardScaler()
24         model_c = XGBClassifier(
25             n_estimators=300, max_depth=6, learning_rate=0.1,
26             subsample=0.8, colsample_bytree=0.8,
27             eval_metric='mlogloss', random_state=42,
28             n_jobs=-1, device='cpu'
29         )
30         model_c.fit(scaler_c.fit_transform(X_tr), y_tr)
31         models_clf[(d, tau_tr)] = model_c
32         scalers_clf[(d, tau_tr)] = scaler_c
```

A.7 Importancia de características con SHAP (`feature_importance.ipynb`)

La interpretabilidad de los modelos se analiza mediante valores SHAP calculados con `TreeExplainer` sobre los regresores *gradient boosting* por dimensión. Para cada par (objetivo, d) se explican exclusivamente muestras del conjunto de prueba —seleccionadas con la partición canónica— y se promedia el valor absoluto de SHAP por característica. La importancia global de cada objetivo se obtiene como media ponderada por el número de muestras de prueba de cada dimensión, garantizando que las dimensiones con mayor representación contribuyan proporcionalmente.

CÓDIGO 6.8. Cálculo de la importancia de características mediante valores SHAP agregados por dimensión.

```
1 SHAP_SUBSAMPLE = 300 # muestras por (target, d)
2
3 shap_global = {t: np.zeros(len(FEAT_COLS_PER_DIM)) for t in TARGETS
4               }
5 total_w_shap = {t: 0.0 for t in TARGETS}
6 test_ids_set = load_test_base_ids()
7
8 for target in TARGETS:
9     for d in DIMS:
10         df_d = df_enc[df_enc['d'] == d]
11         X_all = df_d[FEAT_COLS_PER_DIM].values
12
13         # Solo muestras de test (particion canonica por base_id)
14         is_test_mask = df_d['base_id'].isin(test_ids_set).to_numpy()
15         X_test = X_all[is_test_mask]
16
17         model = joblib.load(f'models/per_dim/gb_{target}_d{d}.pkl')
18         scaler = joblib.load(f'models/per_dim/scaler_{target}_d{d}.pkl')
19         X_scaled = scaler.transform(X_test)
20
21         # Submuestreo reproducible
22         rng = np.random.default_rng(42)
23         idx = rng.choice(len(X_scaled),
24                           size=min(SHAP_SUBSAMPLE, len(X_scaled)),
25                           replace=False)
26
27         explainer = shap.TreeExplainer(model)
28         shap_vals = explainer.shap_values(X_scaled[idx],
29                                           check_additivity=False)
30         mean_abs = np.abs(shap_vals).mean(axis=0)
31
32         # Agregacion global ponderada por num de muestras de test
33         w = is_test_mask.sum()
34         shap_global[target] += mean_abs * w
35         total_w_shap[target] += w
```

34

35

```
shap_global[target] /= total_w_shap[target]
```

ANEXO B: DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG)

A continuación se cumplimenta la declaración de uso de Inteligencia Artificial Generativa exigida por la Universidad Carlos III de Madrid para la entrega del Trabajo de Fin de Grado.

He usado IAG en mi TFG: ☒ SÍ ☐ NO

B.1 Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFG puede verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad)

Pregunta 1. En mi interacción con herramientas de IAG he facilitado **datos de carácter confidencial** contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).

☐ SÍ, he usado estos datos con la autorización de los interesados.

☒ NO, no he usado datos de carácter confidencial.

Pregunta 2. En mi interacción con herramientas de IAG he facilitado **materiales protegidos por derechos de autoría** contando siempre con la autorización de los respectivos titulares.

☒ SÍ, he usado estos materiales con autorización de los titulares de los derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley (obra en dominio público, obra licenciada —p. ej. licencias *Creative Commons*— o uso de fragmentos con fines de investigación, derecho de cita).

☐ NO, no he usado materiales protegidos por derechos de autoría.

Pregunta 3. En mi interacción con herramientas de IAG he facilitado **datos de carácter personal** con la debida autorización de los interesados.

☐ SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad.

☒ NO, no he usado datos de carácter personal.

Pregunta 4. Mi utilización de la herramienta de IAG ha **respetado sus términos de uso**, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.

☒ SÍ ☐ NO

B.2 Parte 2: declaración de uso técnico

Se detallan a continuación los tipos de interacción mantenidos con herramientas de IAG, acompañados de un ejemplo representativo de cada uno.

B.2.1 Documentación y redacción

Declaro haber hecho uso del sistema de IAG *Gemini Flash 3.5 Educativo* para las tareas que se enumeran:

☒ **Soporte a la reflexión** sobre el desarrollo del trabajo (análisis iterativo de alternativas y enfoques).

Ejemplo: empleando el *prompt* «¿Cómo se puede modelar un problema de machine learning donde la variable a predecir tiene una varianza muy baja?», teniendo como interacción la discusión de compromisos que orientó la separación entre el regresor por dimensión y el oráculo de selección recogidos en el Sección 3.4.2.

Ejemplo: empleando el *prompt* «¿Qué riesgos de fuga de información existen al particionar un conjunto en el que cada base genera varias filas con distintos valores de δ ?», teniendo como interacción la confirmación conceptual del problema, cuya solución (partición agrupada por *base_id*) fue diseñada e implementada de forma independiente por el autor.

☒ **Revisión o reescritura** de párrafos redactados previamente.

Ejemplo: empleando el *prompt* «Revisa el registro académico del siguiente párrafo sobre la partición canónica anti-fuga sin alterar las cifras», teniendo como interacción la incorporación de las correcciones de estilo manteniendo intactos los valores verificados frente a los cuadernos.

☒ **Búsqueda de información** o respuesta a preguntas concretas.

Ejemplo: empleando el *prompt* «¿Cómo se relacionan el δ aplicado a una reducción y el factor de Hermite de la base reducida?», teniendo como interacción la clarificación conceptual posteriormente contrastada con la bibliografía citada.

Ejemplo: empleando el *prompt* «¿Qué convención sigue la función `get_r` de `fpyl` respecto a las normas de Gram–Schmidt?», teniendo como interacción una orientación inicial

que fue verificada directamente sobre la documentación oficial de la biblioteca antes de su uso en el código.

☒ **Resumen de bibliografía consultada.**

Ejemplo: empleando el *prompt* «Resume las ideas principales de este artículo sobre aplicación de aprendizaje automático al criptoanálisis reticular», teniendo como interacción una síntesis que sirvió de punto de partida para la lectura crítica del trabajo, posteriormente verificada sobre la fuente original.

☐ Búsqueda de bibliografía.

☐ Traducción de textos consultados.

B.2.2 Desarrollo de contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

☒ **Asistencia en el desarrollo de líneas de código** (programación).

Ejemplo: empleando el *prompt* «Ayúdame a depurar la partición agrupada por `base_id` para garantizar que ninguna base aparezca a la vez en entrenamiento y test», teniendo como interacción la revisión de las aserciones anti-fuga del módulo `split_utils.py`, cuya lógica final fue diseñada, probada y validada por el autor.

Ejemplo: empleando el *prompt* «Ayúdame a escribir el código `matplotlib` para un gráfico de dispersión de valores predichos frente a reales, con la diagonal $y = x$ de referencia y los ejes etiquetados», teniendo como interacción una plantilla de trazado que el autor adaptó y rellenó con las predicciones reales de los regresores, comprobando que la figura reproduce fielmente los resultados del cuaderno `dim_models.ipynb`.

Ejemplo: empleando el *prompt* «Necesito trazar varias curvas del ahorro temporal frente al umbral τ , una por dimensión, en unos mismos ejes con leyenda», teniendo como interacción la estructura del trazado, que el autor completó con los resultados del cuaderno `oracle.ipynb` y revisó para confirmar la transición observada en torno a $\tau \approx 0,01$.

Ejemplo: empleando el *prompt* «¿Cómo guardo y vuelvo a cargar un modelo y su `scaler` con `joblib` respetando una convención de nombres por objetivo y dimensión?», teniendo como interacción el patrón de persistencia que el autor adaptó a la estructura de directorios `models/per_dim/` del proyecto.

Ejemplo: empleando el *prompt* «Indícame todas las semillas que debo fijar para que un experimento con NumPy, scikit-learn y PyTorch sea reproducible», teniendo como interacción una lista de verificación (`random_state`, `np.random`, `torch.manual_seed`) que el autor aplicó de forma sistemática con la semilla 42.

Ejemplo: empleando el *prompt* «¿Cuál es la sintaxis correcta para instanciar un Gradient Boosting Regressor con aceleración por GPU mediante `device='cuda'?`», teniendo

como interacción la confirmación de la API XGBoost que ha permitido acelerar enormemente los experimentos. Igualmente, los hiperparámetros concretos y la decisión de entrenar un modelo por dimensión fueron definidos por el autor en `dim_models.ipynb`.

- Generación de esquemas, imágenes, audios o vídeos.
- Procesos de optimización.
- Tratamiento de datos (recogida, análisis, cruce de datos).
- Inspiración de ideas en el proceso creativo.
- Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo.

B.3 Parte 3: reflexión sobre utilidad

El uso de herramientas de IAG durante este Trabajo de Fin de Grado se ha mantenido deliberadamente en un papel de apoyo, nunca de sustitución del criterio propio. Entre sus **fortalezas**, ha resultado especialmente útil para acelerar tareas mecánicas y de bajo valor cognitivo: la revisión de estilo, el formateo de tablas y especialmente para el uso de $\text{\LaTeX}$ pudiendo aprovechar las ventajas que ofrece esta herramienta evitando sus desventajas (desventajas como conocer o investigar la sintaxis de $\text{\LaTeX}$ antes de haber hecho la memoria). Para depuración puntual de código y una primera exploración de la bibliografía que agilizó la posterior lectura crítica de las fuentes. Este apoyo permitió dedicar más tiempo a las decisiones de diseño experimental y a la interpretación de los resultados, que constituyen el núcleo del trabajo.

Entre sus **debilidades**, la limitación más relevante ha sido la tendencia de estas herramientas a producir afirmaciones plausibles pero incorrectas, particularmente en datos numéricos y citas bibliográficas. Por ello se adoptó como norma metodológica verificar toda cifra contra las salidas reales de los cuadernos, el fichero `xgb_results.json` y el código fuente, sin tomar en ningún caso el texto generado como fuente de verdad. Asimismo, la herramienta carece del conocimiento específico del dominio y del contexto experimental concreto, por lo que las contribuciones conceptuales, las decisiones metodológicas y las conclusiones son responsabilidad exclusiva del autor.

En conjunto, la IAG ha contribuido principalmente al *proceso de redacción y depuración*, con un papel menor en el aprendizaje conceptual y nulo en la extracción de conclusiones, que se han derivado directamente del análisis de los resultados experimentales propios.